



Departamento de Ingeniería de Computadores

Facultad de Informática de A Coruña

UNIVERSIDADE DA CORUÑA

MASTER'S THESIS

INTERUNIVERSITY MASTER'S DEGREE IN

HIGH PERFORMANCE COMPUTING

Design and implementation of a multithreaded Armv8-A operating system kernel

Student: Unai Iborra Albéniz

Supervisor: Emilio José Padrón González

A Coruña, August 13, 2026.

*To my classmates in this master's degree, for making the journey as rewarding as it has been.
To Emilio, for his guidance and support throughout this project.*

Abstract

This project presents the design and implementation of a multithreaded, multicore operating system kernel for the Armv8-A architecture, running on real hardware. The kernel targets the AArch64 execution state of the NXP i.MX 8M Plus System on a Chip, written primarily in C with Assembly where required. It implements the full stack of a monolithic kernel: MMU-based virtual memory with multilevel page tables, a layered memory allocator, a preemptive multicore scheduler with load balancing, hardware abstraction through a device registration system, interrupt-driven I/O, and a system call interface supporting real userspace programs.

Resumen

Este proyecto presenta el diseño e implementación de un kernel de sistema operativo multihilo y multinúcleo para la arquitectura Armv8-A, ejecutado en hardware real. El kernel implementa los subsistemas habituales en un kernel monolítico: memoria virtual basada en MMU, un sistema de allocación de memoria en capas, un planificador multinúcleo con balanceo de carga, abstracción de hardware mediante un sistema de registro de dispositivos, comunicación con periféricos dirigida por interrupciones, e interfaz de llamadas al sistema que permite ejecutar programas reales en espacio de usuario.

Keywords:

- Operating system kernel
- Armv8-A
- Multithreading
- Virtual memory
- Memory allocator
- Multicore scheduler
- Bare-metal
- Embedded systems

Palabras clave:

- Kernel de sistema operativo
- Armv8-A
- Multihilo
- Memoria virtual
- Asignador de memoria
- Planificador multinúcleo
- Bare-metal
- Sistemas embebidos

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Objectives	2
1.3	Document Structure	3
2	State of the art	5
2.1	Reference Kernels	5
2.2	The Armv8-A architecture	7
2.2.1	Execution States	7
2.2.2	Registers	7
2.2.3	System Registers and PSTATE	8
2.2.4	Processing Elements (PEs)	8
2.2.5	Exception Levels	8
2.2.6	Hardware Exceptions and DAIF	10
2.2.7	Memory Management Unit (MMU)	12
2.2.8	Memory Model	17
2.2.9	Common Extensions	18
2.3	Targeted Hardware	19
3	Kernel Design and Implementation	23
3.1	Design Choices	23
3.2	Project Structure	25
3.3	Kernel Boot Stage	25
3.4	Kernel Initialization	34
3.5	Architecture Modules	36
3.5.1	MMU	36
3.5.2	Exception Handling	39
3.6	Common Modules	42

3.7	Memory Management Subsystem	46
3.7.1	Memory Layout	47
3.7.2	Early Allocator	49
3.7.3	Memory Management Memory Management Unit (MMU) submodule	53
3.7.4	Identity Mapping	54
3.7.5	Kernel Relocation	57
3.7.6	Physical Memory Management	58
3.7.7	Virtual Memory Management	63
3.7.8	Reserve Allocator	66
3.7.9	Raw Kernel Memory Allocator	67
3.7.10	Cache Allocator	69
3.7.11	Kernel Memory Allocator	71
3.7.12	Userspace Memory Management	72
3.7.13	Binary loading	76
3.8	Scheduler	76
3.8.1	Processes	77
3.8.2	Threads	78
3.8.3	Context Switching	78
3.8.4	Runqueue	79
3.8.5	Scheduling Algorithm	80
3.8.6	Preemptive Scheduling	81
3.8.7	Scheduler Loop	82
3.8.8	Thread Lifecycle	82
3.8.9	Parallelism	84
3.9	Device Control Subsystem	85
3.9.1	Device Classes	86
3.9.2	Implemented Drivers Overview	87
3.10	Time Subsystem	88
3.11	Virtual File System (VFS) Subsystem	88
3.12	I/O Subsystem	89
3.13	System Calls	90
4	Results	95
4.1	Userspace Execution	95
4.2	Experiment Setup	96
4.3	Context Switch Latency	99
4.4	Syscall Latency	102
4.5	Allocator Latency	104

CONTENTS

4.6 Experiments Conclusions	106
5 Conclusions	107
A Figures	111
B Listings	113
Acronyms	149
Glossary	151
Bibliography	155

List of Figures

2.1	Translation table walk example for 48-bit addresses with 4 KiB granularity. . .	16
2.2	MMU page table descriptor bit fields.	17
2.3	FRDM i.MX 8M Plus Development Board mounted peripherals	20
3.1	Project subsystem dependencies	26
3.2	mmu_map simplified flowchart	40
3.3	Kernel memory layout	50
3.4	Early allocator initialization and allocation process	52
3.5	MMU mappings state after calling early_identity_mapping	57
3.6	Order 4 page allocator example	61
3.7	Page Allocator initialization state	62
3.8	Direct map and non-direct map regions on IMX 8M Plus	64
3.9	17 virtual contiguous pages non-direct mapping	65
3.10	Cache allocator 64-byte arena data structure example	71
3.11	Userspace physical memory mapping example	75
3.12	Thread state transition diagram	83
A.1	Project subsystem dependencies related to project structure	112

List of Tables

2.1	Armv8-A exception Vector Table format.	11
2.2	Simplified Cortex-A53 memory map of the i.MX 8M Plus.	20
3.1	Top-level project directory structure.	25
3.2	AArch64 Vector Table entries and their handlers	42
3.3	Synchronous exception classes and their handlers	43
3.4	Advantages and disadvantages of the buddy page allocator	62
3.5	Implemented system calls	91
4.1	Performance monitor fence overhead (N = 20,000 samples). The wall-clock resolution is limited by the 24 MHz system counter (41.7 ns per tick), which explains the zero values at low percentiles.	99
4.2	Preemptive context switch latency breakdown (N = 18,318 samples, IRQs masked during measurement). <i>Store</i> : <code>scheduler_ectx_store()</code> . <i>Load</i> : <code>runqueue_schedule() + scheduler_ectx_load()</code>	100
4.3	<code>ectx_store</code> and <code>ectx_load</code> macro overhead in isolation (N = 50,000, IRQs masked).	102
4.4	System call latency (N = 20,000 per syscall unless noted, IRQs masked, -O2). .	103
4.5	Memory allocator latency (N = 40,000 per operation, IRQs masked, -O2). Cycles represent the mean. Page sizes: 1p = 4 KiB, 2p = 8 KiB, 3p = 12 KiB, 5p = 20 KiB.	105

Listings

2.1	msr and mrs instructions	8
3.1	daifset instruction	27
3.2	.bss initialization	28
3.3	Cache flush function	29
3.4	Early stack initialization	30
3.5	Vector Table implementation	30
3.6	_switch_to_el1 function implementation	32
3.7	Kernel boot entry code	33
3.8	kernel_entry initialization calls	34
3.9	safe_early macro definition	35
3.10	MMU core-local API main functions	36
3.11	MMU mapping-level API main functions	37
3.12	arm_ctx_t struct definition	41
3.13	Red black tree node header struct	44
3.14	mm_mmu main functions and variables	53
3.15	early_identity_mapping MMU core configuration.	55
3.16	Page configurations used for identity mapping of Memory-Mapped I/O (MMIO) and Double Data Rate (category of RAM) (DDR) regions.	56
3.17	raw_kmalloc direct-mapped allocation core logic	68
3.18	raw_kmalloc non-direct mapped path summarized	68
3.19	kmalloc implementation	71
3.20	Task struct definition	77
3.21	Thread struct definition	78
3.22	runqueue_t struct definition	79
3.23	syscall_dispatch() function implementation	90
4.1	Process loading from the kernel	95
4.2	Hello world example	96
4.3	Hello world userspace logs	96

4.4	Performance monitor module main API functions	97
4.5	Performance monitor module main API functions	98
4.6	Assembly context switch experiment code	101
4.7	C context switch experiment code	101
B.1	ectx_store and ectx_load assembly macros	113
B.2	Kernel exception vector configuration and exception context store/load	117
B.3	Scheduler load and store functions	119
B.4	runqueue_schedule function implementation	119
B.5	Device control public API exposed through device.h	122
B.6	Context switch experiment userspace tested code	123
B.7	Syscall benchmarking userspace code	124
B.8	Syscall benchmarking kernel_entry function	125
B.9	Syscall benchmarking logs	126
B.10	Allocator benchmarking logs	131
B.11	Allocator benchmark source code	144

Introduction

OPERATING system kernels are the pieces of software that sit between hardware and user applications, and every abstraction that software relies on, like virtual memory, scheduling, I/O and isolation is provided by the kernel. Designing and coding a kernel exposes hardware behaviors, and low-level programming techniques that are rarely found in userspace development, where the kernel already handles them. This contributes to understanding how computers internally work, and why userspace programs are structured in the way they are.

This academic work addresses the implementation of an operating system kernel from scratch for an embedded system, specifically an NXP i.MX 8M Plus [System On a Chip \(SoC\)](#) built on top of an ARM Cortex-A53 processor.

1.1 Motivation

The [Armv8-A](#) architecture is a relevant target for this kind of project. It is the architecture used by modern mobile devices, embedded [System On a Chip \(SoC\)](#)s, and a big percentage of server and high-performance computing hardware. Its 64-bit execution state (AArch64) provides a well documented privilege model through [Exception Levels](#), a flexible MMU with multilevel page tables, and a big set of system registers. This makes the architecture a realistic and challenging platform for kernel development.

Kernel development through QEMU emulation is a common target. In practice, however, emulation is not as strict as direct hardware execution, and masks hardware quirks that only appear on real silicon. Since one of the main motivations of this project is understanding these low-level hardware details, development targets real hardware.

The decision to target the NXP i.MX 8M Plus [SoC](#) was influenced by the quality and completeness of NXP's public hardware documentation, which is a prerequisite for this kind of low-level development.

1.2 Objectives

The main goal of this project is the design and implementation from scratch of a multithreaded, multicore operating system kernel for the Armv8-A architecture, running on real hardware. The kernel targets the AArch64 execution state and is developed primarily in C, with Assembly used when necessary.

The following concrete objectives were defined at the start of the project and describe the standard components a kernel should implement:

- **Boot and initialization:** implement the kernel boot sequence in Assembly, setting up a valid C execution environment from the state left by the [bootloader](#), and initialize all kernel subsystems in the correct order.
- **Exception and interrupt handling:** configure the exception [Vector Table](#) and correctly handle synchronous exceptions and hardware interrupts, including integration with the [Generic Interrupt Controller \(GIC\)](#).
- **Virtual memory:** implement MMU-based virtual memory management, providing kernel and per-process address space isolation through dynamic multilevel page tables configurations.
- **Memory management:** design and implement a memory allocation system, covering physical page management, kernel dynamic allocation, and userspace memory management.
- **Process and thread execution:** implement user space execution through a thread model, with support for the [Exception Level 0 \(User-space privileged level\) \(EL0\)](#) privilege level and a system call interface.
- **Multicore scheduler:** develop a preemptive, multicore thread scheduler capable of distributing threads across the available CPU cores.
- **Driver abstractions:** implement platform-specific drivers behind a [Hardware Abstraction Layer](#) that allows portability across Armv8-A platforms.
- **Performance analysis:** measure and analyze the performance of key kernel operations like [context switches](#), system calls, and memory allocations. Relate the results to the design decisions taken throughout the project.

1.3 Document Structure

The rest of this document is organized as follows. Chapter 2 covers the reference kernels that inspired the design, a summary of relevant aspects of the Armv8-A architecture, and the specific hardware used. Chapter 3 describes the design and implementation of each kernel subsystem in detail. Chapter 4 analyzes the performance of the main kernel functionalities and the relation with their design decisions. Finally, Chapter 5 presents the conclusions of the project, evaluating the objectives achieved and outlining directions for future work.

State of the art

THE development of a kernel does not come from nowhere. Many operating systems and kernels have been previously developed and studied, and have defined the modern principles of what a kernel should do. Before examining these references, it is worth explaining what a kernel actually is and how it relates to the concept of an operating system.

An operating system is the software layer that sits between hardware and user applications, providing abstractions that make hardware usable in a portable and general way. The kernel is the core component of the operating system. It is the part that runs with full hardware privileges, and it is responsible for the most fundamental tasks: managing physical and virtual memory, scheduling the execution of processes, handling hardware exceptions (such as interrupts), and providing an interface for the user space programs via system calls. Everything else, such as shells, graphical interfaces, or standard libraries are part of the operating system, but not part of the kernel itself.

The line between where the kernel starts and where the operating system continues, is not shared among the operating systems. Monolithic kernels, such as Linux, include drivers and other services directly inside the kernel itself. It attempts to trade simplicity of design for performance. Other kernels, however, such as seL4, keep the privileged core very minimal and try to move as much as possible to the user space, attempting to prioritize isolation.

2.1 Reference Kernels

The following kernels have been studied as part of the background research for this project. Each one represents a different point in the design space of operating system kernels, and their analysis allow the architectural decisions made throughout development to be informed and influenced by smart kernel development decisions.

Linux

The most widely deployed kernel on Armv8-A hardware is, without a doubt Linux [1]. It is present in Android devices, embedded systems, servers, and supercomputers. Linux represents the state of the art in monolithic general purpose kernels. Its memory management subsystem, its scheduler, and its driver model are standard references in any serious study of modern kernel design. Support for Armv8-A is mature and makes Linux the standard to which any new kernel targeting this architecture will be compared against. Because of this, most of the kernels, in some way or another, are influenced by Linux design and implementation.

xv6

xv6 [2] is a small teaching kernel developed at MIT, originally written as a reimplementation of Unix Sixth Edition (V6) for x86, and later ported to RISC-V. It has a codebase of only a few tens of thousands of lines and it exposes the fundamental subsystems of a kernel. Memory management through paging, process scheduling, a basic filesystem, and system calls, in a way that is easy to read and understand.

It is important to denote that the architectural differences between x86/RISC-V and Armv8-A are substantial. The memory model, exception handling mechanisms and hardware initialization sequence differs significantly. Because of this, xv6 has been useful for guiding higher level decisions, but not for understanding architectural implementations.

seL4

seL4 [3] is a microkernel known for being the first general purpose kernel to be formally verified for functional correctness. Unlike monolithic kernels, seL4 keeps the privileged part minimal, delegating services such as drivers and filesystems to user space. This approach maximizes isolation and auditability at the cost of more complexity in other areas.

The kernel developed in this project does not follow a microkernel architecture, but the study of seL4 is still relevant as it one of the open-source kernel developments whose code can be analyzed.

Zephyr and FreeRTOS

Two real-time operating systems are also worth considering. Zephyr and FreeRTOS, both are designed for SoCs and microcontrollers where resources such as memory are scarce, and both offer native support for Armv8-A targets.

Their scheduling models are oriented towards real-time guarantees rather than fairness, and their interrupt handling is optimized for low latency. It is more ambitious than a typical

Real-Time Operating System (RTOS) in terms of memory management and user space support, but more focused and constrained than a general-purpose kernel like Linux.

2.2 The Armv8-A architecture

The kernel developed in this project targets the **Armv8-A Instruction Set Architecture (ISA)** [4], the first generation of the Arm architecture to introduce 64-bit computing while maintaining backward compatibility with 32-bit instruction sets. It serves as the foundational 64-bit architecture used in most modern mobile devices, multipurpose SoCs and servers, making it the natural target for a general-purpose kernel.

This architecture must not be confused with the **Armv8-M ISA** [5], which shares the Armv8 generation name but targets an entirely different domain: embedded systems and microcontrollers. Armv8-M omits the MMU and the infrastructure required to run a full operating system, prioritizing instead low power consumption, deterministic latency, and minimal silicon footprint.

2.2.1 Execution States

The Armv8-A architecture has two primary states. AArch64, which allows for 64-bit processing and the A64 instruction set, and AArch32, which allows running legacy 32-bit processing using A32 and T32 instruction sets. AArch64 instructions always have a size and alignment of 4 bytes.

This project will only target AArch64 to simplify the development and explore the standard state of the architecture.

2.2.2 Registers

Armv8-A uses 31 general purpose 64-bit registers, which define the “word” registers of the architecture. They are named with the x prefix, and range from x0 to x30. The x30 register is special, as it is used as the link register (sometimes used with the alias `lr`). It is used to store the return address of a function, and it is automatically used by the hardware when executing the `ret` instruction. The x29 register is generally used as the frame pointer, however, it does not have different hardware characteristics from the other x registers.

The architecture also supports 32-bit registers (“halfwords”), but share hardware with the lower four bytes of the x registers. This has to be taken into consideration when using them with x registers, as storing data in the same numbered x or w register will change its counterpart’s data.

Another useful register is the zero register. It is called with the alias `xzr` and `wzr`, depending on the bit width. It is always zero and discards any value written to it.

The stack pointer, called with the alias `sp`, serves as the pointer that indicates the current stack position in memory. The architecture provides four stack pointer registers, each with different characteristics and use cases. This is further explained in the section 2.2.5.

In [Armv8-A](#), the program counter is not directly accessible, so it cannot be manually modified nor read without the use of specific instructions.

2.2.3 System Registers and PSTATE

The configuration of how the processor hardware should behave is usually done via *system registers*, commonly referred as “[sysregs](#)”. Sysregs in Armv8-A are special 32 or 64 bit registers that are accessed via the privileged instructions MSR and MRS, as seen in Listing 2.1.

Listing 2.1: msr and mrs instructions

```
msr <sysreg_name>, <x/w_register> // write to sysreg
mrs <x/w_register>, <sysreg_name> // read from sysreg
```

There are hundreds of [sysregs](#) defined in the architecture, that allow many operations such as MMU configuration, trap enabling, interrupt configuration, getting the ID of the processor core, etc.

[PSTATE](#) is the architectural notion that represents the processor state visible to software. It includes essential system configuration fields that control execution. PSTATE is not a single register, but the sum of different sysreg bits stored across several dedicated [sysregs](#). PSTATE contains condition flags, interrupt mask bits, execution state, and the current [Exception Level](#).

2.2.4 Processing Elements (PEs)

The Armv8 architecture defines the concept of a [Processing Element \(PE\)](#) as the abstract unit that executes instructions and maintains architectural state. For practical purposes, a PE can be regarded as equivalent to a core, and both terms will be used interchangeably throughout this report.

2.2.5 Exception Levels

In Armv8-A, privilege is established through a hierarchy of [Exception Levels \(ELs\)](#), which determine what instructions, system registers and hardware resources the core (PE) can access. This is similar to the ring model used in x86-64 architecture, where ring 0 is the most privileged and ring 3 the least, although the Armv8-A model has a different structure and purpose for each level.

Armv8-A defines four [Exception Levels](#), from least to most privileged:

- **Exception Level 0 (User-space privileged level) (EL0):** It is the level where user applications run. Instructions executing at **EL0** have the most restricted access. It cannot access most system registers, cannot configure the MMU, and can only interact with the kernel through system calls or when hardware exceptions happen (e.g., a **translation fault**).
- **Exception Level 1 (Kernel/Supervisor privileged level) (EL1):** The level where operating system kernels usually run. **Exception Level 1 (Kernel/Supervisor privileged level) (EL1)** has access to the full set of system registers for its level, including the ones that control the MMU, exception handling and cache behavior.
- **Exception Level 2 (Hypervisor privileged level) (EL2):** It is intended for hypervisors and virtualization software. **Exception Level 2 (Hypervisor privileged level) (EL2)** can manage multiple **EL1** operating systems, trapping and emulating privileged operations. It is also usually the level where **bootloaders** run.
- **Exception Level 3 (Firmware privileged level) (EL3):** The highest privilege level, used by firmware and TrustZone secure monitor code. It controls the separation between the Secure and **Non-Secure worlds** and manages transitions between them.

Each **Exception Level** has its own set of **sysregs**, specified by the `_EL<n>` suffix. For example `SCTLR_EL1` and `SCTLR_EL2` are independent sysregs of the System Control Register, for **EL1** and **EL2** respectively. A CPU running at a given **Exception Level**, can only access its own levels and lower levels sysregs. For example **EL2** can access **EL2** and **EL1** system registers, while **EL1** can only access **EL1** sysregs but not **EL2**'s ones. Attempting to access a sysreg from a disallowed level will cause a synchronous exception.

Transitions between **Exception Levels** follow specific rules. A lower EL can only raise its privilege level by triggering an exception, such as a system call or an interrupt. However, for many exception types, the higher EL must have previously explicitly enabled the trap for those types, or the EL will not raise. Transitioning to a lower level is done through the `ERET` instruction, which restores the execution state from the `ELR_EL<n>` and `SPSR_EL<n>` system registers.

Exception Level Stack Pointers (**SP_EL<x>**)

The architecture provides one stack pointer for each **Exception Level**. ELs ranging from 3 to 1 can use either their own stack pointers (`SP_EL3` for **Exception Level 3 (Firmware privileged level) (EL3)**, `SP_EL2` for **EL2**, etc.), or `SP_EL0`. **EL0**, can only use `SP_EL0`.

The stack pointer only has one instruction encoding code, so the selection of what stack pointer is used happens via the `SPSEL_EL<n>` **sysreg**. When executing instructions that

involve the use of the stack pointer, the architecture selects the used register depending on the SPSEL_EL<n> configuration.

When using the current EL stack pointer, the letter “h” is appended to the [Exception Level](#) name, referring to the *host* stack pointer. For example, [EL1](#) using its own stack pointer is referred to as EL1h, while EL1 using SP_EL0 is referred to as [EL1t](#) (*thread*).

2.2.6 Hardware Exceptions and DAIF

An exception is any event that can cause the currently executing program to be suspended and cause a change in state to execute code to handle that exception. This is sometimes defined as interrupts by other architectures, but in Armv8-A, interrupts are a type of externally generated exception. The exception types defined by the architecture are the following:

- **Synchronous (Sync):** Exceptions caused directly by the instruction being executed. Examples include system calls triggered by the SVC instruction, memory access faults caused by the MMU ([translation faults](#), permission faults), undefined instruction exceptions and [alignment faults](#).
- **Interrupt Request (IRQ):** Asynchronous exceptions generated by external hardware peripherals to signal that they require attention. Common sources include timers, [Universal Asynchronous Receiver-Transmitter \(UART\)](#) controllers and other devices. IRQs are routed through the [Generic Interrupt Controller \(GIC\)](#) [6] in most of the [Armv8-A](#) architecture implementations, which manages priority and distribution across cores.
- **Fast Interrupt Request (FIQ):** Similar to IRQs but with higher priority and lower latency. FIQs are intended for time-critical interrupt sources that require immediate handling. In practice, FIQs are commonly used by the [Secure World](#) in [TrustZone](#) configurations.
- **System Error (SError):** Asynchronous exceptions generated by the memory system, typically as a result of errors that cannot be attributed to a specific instruction, such as bus errors or uncorrectable memory faults. SErrors are generally considered unrecoverable.

Exceptions that happen in [EL0](#), are always trapped from higher [ELs](#), usually from [EL1](#), which makes EL0 not have a [Vector Table](#). Exceptions happening in EL1 and [EL2](#) can also be trapped into higher [ELs](#), however, as it is not relevant for this project development, the specifics will not be explained.

Another detail to consider is that when a hardware exception happens, the SPSEL_EL<n> [sysreg](#) is always automatically set to use SP_EL<n>.

Vector Tables

In order to tell the **PE** the address where the code that handles the exception is, the system register `VBAR_EL<n>` has to be used. `VBAR_EL<n>` is the responsible **sysreg** that holds the **Vector Table** base address, which is an area of **Normal Memory** containing instructions. When an exception happens, the program counter changes to the address set inside `VBAR_EL<n>` plus an offset from that base (each exception type has a defined offset from the base).

The Vector Table has a size of 2 KiB and its base address must also be aligned to 2 KiB.

The Vector Table not only identifies the exception type that was triggered, but it also allows identifying if the exception was taken from a lower, or the current EL, if it was taken from AArch64 or AArch32 mode and what stack pointer was being used when it comes from the current EL. The format of the **Vector Table** is shown in Table 2.1.

As only 128 bytes (32 instructions) are available for each table entry, usually a branch (b) instruction is used to jump to the address of the exception handler code of its type.

Table 2.1: Armv8-A exception **Vector Table** format.

Offset	Exception type	Source
0x000	Synchronous	Current ELt, (SP_EL0)
0x080	IRQ	Current ELt, (SP_EL0)
0x100	FIQ	Current ELt, (SP_EL0)
0x180	SError	Current ELt, (SP_EL0)
0x200	Synchronous	Current ELh, (SP_EL<x>)
0x280	IRQ	Current ELh, (SP_EL<x>)
0x300	FIQ	Current ELh, (SP_EL<x>)
0x380	SError	Current ELh, (SP_EL<x>)
0x400	Synchronous	Lower EL, AArch64
0x480	IRQ	Lower EL, AArch64
0x500	FIQ	Lower EL, AArch64
0x580	SError	Lower EL, AArch64
0x600	Synchronous	Lower EL, AArch32
0x680	IRQ	Lower EL, AArch32
0x700	FIQ	Lower EL, AArch32
0x780	SError	Lower EL, AArch32

Exception Masking

The architecture allows for specific exception disabling, which is commonly referred to as **exception masking**. Exception masking is a common operation and is performed by setting some of the **DAIF** system register's bits to one. Each letter on the DAIF word refers to the type

of exception to be masked.

- **D** stands for the Debug exceptions mask bit. They are a subtype of synchronous exceptions that allow hardware debuggers to work.
- **A** stands for the SErrors exceptions mask bit.
- **I** refers to the [Interrupt Request \(IRQ\)](#) mask bit.
- **F** stands for the [Fast Interrupt Request \(FIQ\)](#) mask bit.

2.2.7 Memory Management Unit (MMU)

The Armv8-A architecture's memory is controlled by the [Memory Management Unit \(MMU\)](#), which allows defining virtual address spaces which can be configured to map physical memory address ranges.

When the MMU is disabled, addresses are accessed directly as physical addresses. However, the architecture is thought to be used with the MMU always enabled (except for early initialization stages), as the caches, out of order execution and memory attributes are ignored while the MMU is disabled.

The MMU is local to each core of the processor, so each core must configure its own MMU.

[Armv8-A](#) defines a 64-bit virtual address space with the minimum addressable unit of a byte. Even though it uses 64-bit addresses, not all the bits are actually used. The architecture supports configurable address bit sizes, typically 48 bits, which allows for a virtual address space of 256 TiB.

This space is split into two regions: low address space and high address space. Both virtual address spaces require different access address formats and are divided by a large address gap:

- **Low address space:** This address space starts from 0x0000_0000_0000_0000 and ends at the maximum possible address for the configured bits minus one. For example, for 48-bit addresses, 0x0000_7FFF_FFFF_FFFF would be the maximum valid address. The upper bits from the sign bit must always be zero for low address space addresses. The sign bit is defined as the bit at the last valid position of the configured address width. For 48-bit addresses it would be the bit 47, for 36-bits, the bit 35, etc.

$$\text{max_low_addr} = 2^{T0SZ_bits} - 1$$

- **High address space:** This address space starts from the sign bit set to one, (bit 47 or 0x0000_8000_0000_0000 for 48 bits). However, high address space addresses require that all the addresses have the unused upper bits set as one, which is usually known as sign-extension. This sets the first valid address as the sign bit plus all the upper bits from that set as one (0xFFFF_8000_0000_0000 for 48 bits). The last valid address of

this space is 0xFFFF_FFFF_FFFF_FFFF.

$$\text{min_high_addr} = 2^{64} - 2^{T1SZ_bits}$$

Note that for example 0xFFFF_0000_6000_0000 would be an invalid address for a 48-bit configuration, as the upper bits from the sign bit do not follow the sign value. 0xFFFF_8000_6000_0000 however, could be a valid address that targets the high address space, and 0x0000_0000_6000_0000 would be another valid address that would target the low address space.

Addresses that follow those rules are known as [canonical addresses](#). Using 48-bit addresses, when trying to access what would intuitively be thought as the address 0x0000_8000_ABCD_EF00, should instead be accessed using the 0xFFFF_8000_ABCD_EF00 address. However, addresses with the sign bit value as zero, (e.g., 0x0000_0000_ABCD_EF00) should be accessed directly. The first example would target the high address space and the second example would target the low address space. Using non-canonical addresses will cause a hardware exception [translation fault](#).

The split between low and high address space is fundamental for kernel development, as it provides the natural boundary between kernel and user address spaces. Usually, low address space is reserved for the user space, and high address space is reserved for the kernel.

MMU Configuration

The MMU works by mapping virtual address ranges, to physical address ranges. The size of those ranges is defined by the configured *granularity*. The Armv8-A MMU allows for 4, 16 and 64 KiB granularity configurations, each one defining the range size. A mapped range of the granularity size is what's generally known as a page. A page is always aligned to its size. Low and high address spaces can have different granularity configurations, and each page can have its own configuration.

The MMU configuration involves two aspects: core local configuration and page configuration:

- **Core local MMU configuration:** done using [sysregs](#). The main involved sysregs are the following:
 - *MAIR_EL<n>*: Allows defining the memory attributes that will be used for the page configurations.
 - *TCR_EL<n>*: The Translation Control Register allows for configuration of multiple aspects, such as the low and high address bits (using *T0SZ* and *T1SZ* bit fields), memory shareability, granule size, etc.

- *SCTLR_EL<n>*: The System Control Register is in charge of enabling/disabling the MMU, caches (instruction and data caches individually), alignment checks and many other configurations.
- *TTBR<0, 1>_EL<n>*: *TTBR0_EL<n>* allows for setting the level zero translation table for low address space and *TTBR1_EL<n>* for high address space.
- **MMU page configuration**: page access behavior can be configured per page (and per descriptor block). It allows for configurations such as the read write and execute permissions memory attributes and allowed privilege levels. This point will be explained more in depth in the following section.

MMU Page Tables

The MMU translates virtual addresses to physical addresses using a hierarchical structure of page tables [7]. A page table is an array of descriptors, where each descriptor either points to the next table of the hierarchy or directly to a physical memory page. The operation of traversing through page tables to obtain the physical address of a virtual address, is generally known as a *translation table walk*.

The descriptors are divided into four categories:

- **Table descriptor**: They point to the physical address of the next page table.
- **Page descriptor**: It points to a physical page address.
- **Block descriptor**: It points to the physical page address of a range of contiguous pages.
- **Null descriptor**: It declares a virtual page or a contiguous range of them as unmapped.

The maximum depth of the hierarchy is defined by the selected granularity. For 4KiB granularity, with 48-bit addresses, the translation uses four levels: L0, L1, L2 and L3. The L0 tables can only contain null or table descriptors, and the last level tables can only contain null or page descriptors. Two L0 tables can be configured via the system registers *TTBR0_EL<n>* and *TTBR1_EL<n>*, defining the start page tables of the low and high address spaces. The virtual address bits are used as indices for each level of the table, until reaching either a block descriptor, or the L3 page descriptor. If a null descriptor is reached instead, a synchronous hardware data abort exception will be thrown.

As an example, a 48-bit address is split into 9 bit-width fields, that serve as the index to the different page tables. Each page table for this configuration has 2^9 (512) entries and the last 12 bits of the address are added to the translation as an offset. The bit 47 is used to decide what L0 page table the MMU should take, if set to zero it will take the low address space table, if set to one the high address space.

The Figure 2.1 shows an example of how a translation table walk is performed when using 4 KiB granularity and 48-bit addresses.

In [Armv8-A](#), MMU descriptors are 8-byte entries configured through different bit fields. They define the descriptor type, page access permissions, memory attributes, and other aspects. The specific bit fields of a descriptor are shown in Figure 2.2.

- **[1:0] Descriptor type:**
 - Value 0b00: Null descriptor
 - Value 0b01: Block descriptor
 - Value 0b11: Table descriptor (when not at last table level)
 - Value 0b11: Page descriptor (when at last table level)
- **[4:2] AttrIdx:** 3-bit index into MAIR_EL1, selecting the memory type and cacheability attributes for this mapping.
- **[5] NS:** Non-Secure. Indicates whether the output address targets Secure or Non-Secure physical memory. Only relevant when [TrustZone](#) security extensions are in use.
- **[7:6] AP:** Access Permissions. Controls read and write access for [EL0](#) and [EL1](#).
 - 0b00: Read/write, EL1 only.
 - 0b01: Read/write, EL0 and EL1.
 - 0b10: Read-only, EL1 only.
 - 0b11: Read-only, EL0 and EL1.
- **[9:8] SH:** Shareability domain. Determines whether the mapping is non-shareable, inner-shareable or outer-shareable, which affects cache coherency behavior across cores.
 - 0b00: Non-shareable.
 - 0b10: Outer-shareable.
 - 0b11: Inner-shareable.
- **[10] AF:** Access Flag. Must be set for the descriptor to be considered accessible. If the bit is set, any memory access to an address inside the region will throw an access flag synchronous exception.
- **[51:11] Output address:** The physical address of the mapped block, table, or page, depending on the descriptor type. Note how the field starts at bit 12 because pages are always aligned to the granularity size. For 4 KiB granularity, the lowest 12 bits of any valid physical address are always zero, so they can be repurposed for attribute storage. This is a common pattern in hardware and low level design.
- **[53] PXN:** Privileged Execute Never. When set, prevents instruction fetch from this mapping at [EL1](#). Typically set for data regions to enforce WX policies.
- **[54] UXN:** Unprivileged Execute Never. When set, prevents instruction fetch from this mapping at [EL0](#).

Figure 2.1: Translation table walk example for 48-bit addresses with 4 KiB granularity.

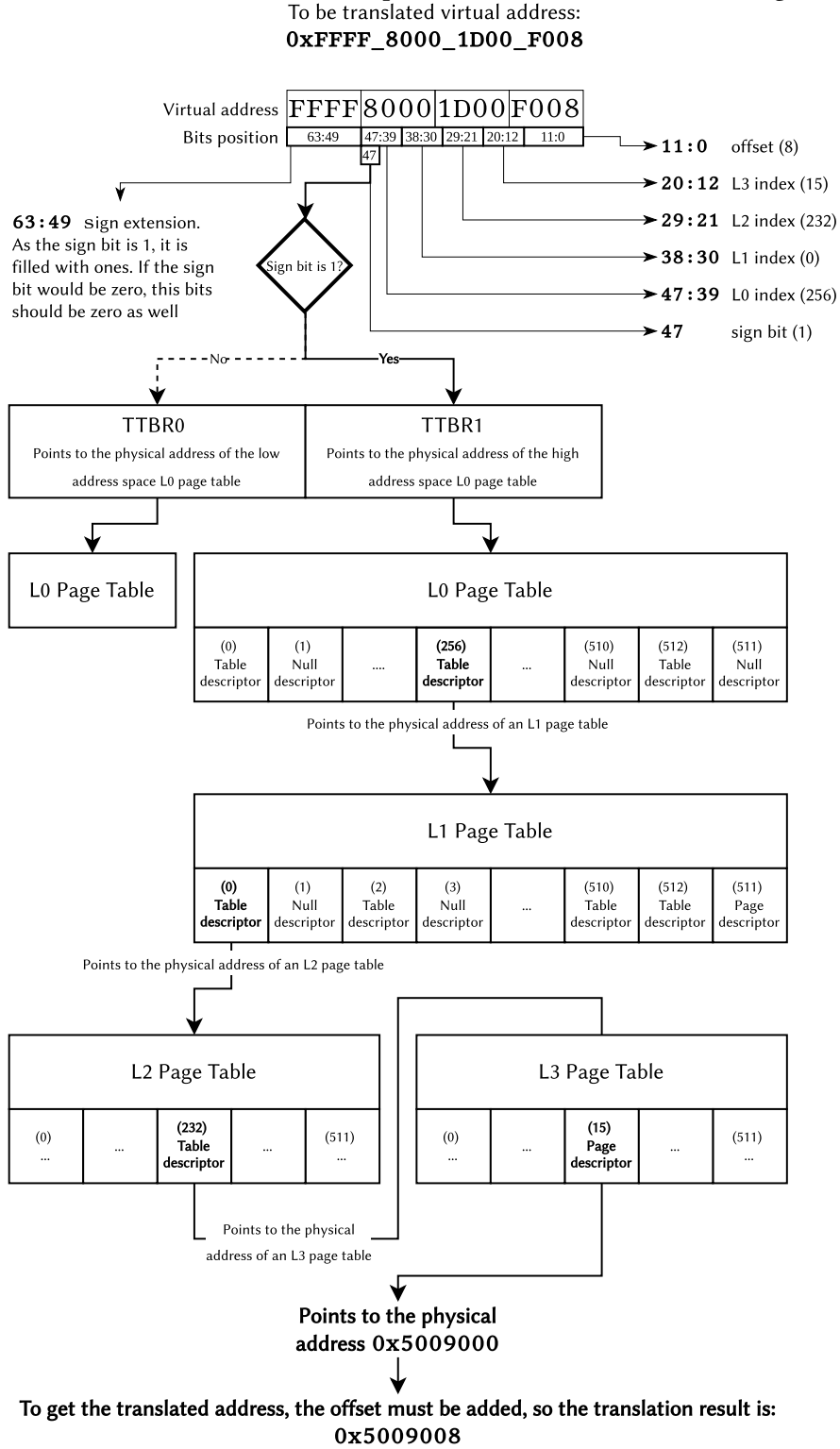
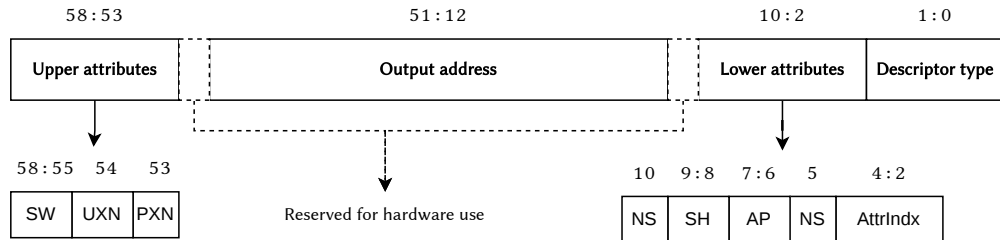


Figure 2.2: MMU page table descriptor bit fields.



- [58:55] **SW**: Software defined bits. These bits are ignored by the MMU and are available for use by the kernel to store per-page metadata.

2.2.8 Memory Model

Memory Access Alignment

For general memory accesses, alignment checking is controlled by the A bit in `SCTLR_EL<n>`. When enabled, the hardware requires that memory accesses are aligned to its access size. A 4-byte load must be 4-byte aligned, a 2-byte load 2-byte aligned, and so on. Violations cause an [alignment fault](#) exception. When disabled, unaligned accesses are generally permitted, though they could cause a performance penalty depending on the memory type and specific characteristics of the access. While the MMU is disabled alignment checks are always enforced.

The stack pointer also has alignment checks. The architecture requires it to always stay aligned to 16-bytes, and the exception caused by violating this rule can never be disabled.

Memory Types and Attributes

[Armv8-A](#) does not treat all memory equally. The architecture defines two categories of memory, each with different ordering and caching guarantees.

Normal Memory is the general purpose memory type used for code and data. It allows the processor to apply optimizations such as reordering, speculative access, and caching. However, this means that accesses to Normal Memory are *weakly ordered*. The processor does not guarantee that memory operations will be observed in program order by other observers such as other cores or [Direct Memory Access \(DMA\)](#) devices. Explicit barrier instructions (DMB, DSB, ISB) are required when ordering guarantees are needed.

Device Memory is expected to be used for [Memory-Mapped I/O \(MMIO\)](#) regions, such as peripheral registers. It disables caching, speculative access, and provides stronger ordering guarantees to ensure that reads and writes to hardware registers behave as expected. The Device Memory is further classified into subtypes that control gathering, reordering and early

write acknowledgement behaviors. For most peripheral access, Device-nGnRnE subtype is the safest and most restrictive option.

The memory types are not determined by the physical address alone, but assigned per mapping through the (MAIR_EL<n>) [sysreg](#), which defines up to eight memory attribute profiles. Each entry in the translation tables then references one of these profiles via a 3-bit index, allowing different pages to be assigned different memory types.

Memory Ordering and Barriers

As explained before, [Normal Memory](#) accesses are weakly ordered. This is a design choice that allows the processor and memory subsystem to apply performance optimizations, but it places responsibility on the programmer to insert barriers where ordering must be enforced.

Armv8-A provides three barrier instructions for this purpose. ISB (Instruction Synchronization Barrier) flushes the pipeline and ensures that following instructions are fetched after the barrier, which is necessary after modifying system registers if the changes must be applied after the MSR instruction. DMB (Data Memory Barrier) makes memory accesses before the barrier to be observed before those after it, without stalling the pipeline for non-memory instructions. DSB (Data Synchronization Barrier) stalls execution until all pending memory accesses are complete.

2.2.9 Common Extensions

The Armv8-A architecture allows CPU manufacturers to include optional hardware extensions that extend the base instruction set. While many extensions exist, the most common one found in general-purpose SoCs and mobile devices is the Advanced [Single Instruction, Multiple Data \(SIMD\)](#) and Floating-Point extension, also known as *NEON*.

NEON adds 32 128-bit vector registers (v0–v31) and a set of [SIMD](#) instructions that operate on them, allowing multiple data elements to be processed in a single instruction. The same registers are also used for scalar floating-point operations, which means that any code that uses floating-point arithmetic depends on this extension being available and enabled.

Even if the NEON extension is available, it has to be explicitly enabled using the CPACR_EL<n> register, or an exception will be thrown when any related instruction is executed.

Using NEON instructions (floating point or [SIMD](#)) in stages where the MMU is disabled is discouraged, specially when they come from high-level programming languages compilations like C, C++ or Rust. Alignment check exceptions are always enforced when the MMU is disabled, and compiled code sometimes ignores NEON alignment requirements that would be ignored when the MMU is enabled (if the checks are manually disabled).

2.3 Targeted Hardware

The development of a kernel does not only involve the architecture of the target processor, but the specific hardware where it runs also matters. Even if an implementation of a kernel can run on a specific processor, without the knowledge of how external hardware like peripherals, [DDR](#) memory or accelerators work, the kernel is useless as it can neither input nor output any information.

NXP i.MX 8M Plus

For the development of the proposed kernel, the selected target hardware is the *NXP i.MX 8M Plus* [8]. It is a [System On a Chip \(SoC\)](#) that integrates a quad-core Arm Cortex-A53 processor, with a [Neural Processing Unit \(NPU\)](#), 2D and 3D graphics accelerators.

It is a chip developed for industrial purposes, focused on machine learning and vision, advanced multimedia, and industrial automation with high reliability.

From a kernel development perspective, one of the most relevant characteristics of this [SoC](#) is the availability of detailed public documentation. NXP provides an extensive [Technical Reference Manual \(TRM\)](#) that covers hardware implementation details like the memory map, peripheral register definitions, boot sequence and hardware initialization details. The [TRM](#) by itself counts with around 7500 pages, but there is many other documentation provided, like debugging manuals, hardware pinouts, etc. This level of public documentation is not always available for embedded SoCs, and it is a significant factor in making this hardware a good choice for a project where hardware interaction is as relevant as in this case.

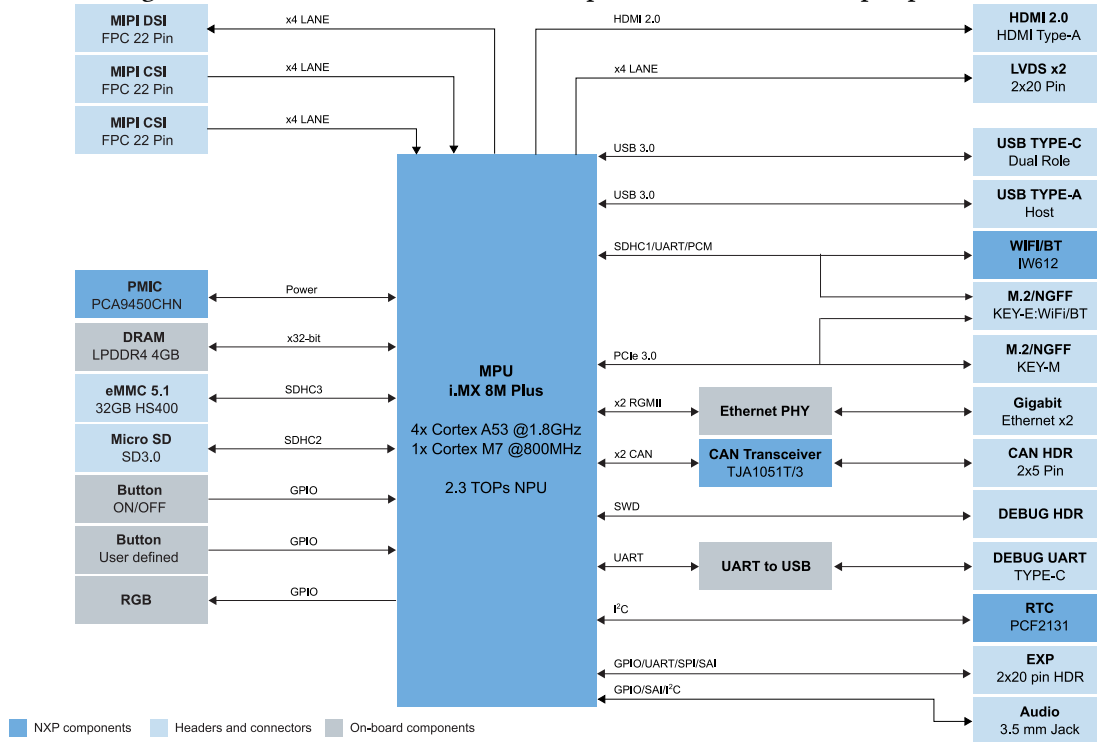
The standard software environment for this SoC is a Yocto based Linux distribution, as it is the standard installed operating system for this SoC, and many documentation assumes the hardware will run this operating system. However, the kernel developed in this project runs directly on the hardware without any underlying operating system layer.

The Specific SoC board chosen for the development of the project has been the “FRDM i.MX 8M Plus Development Board”. It is a board that mounts most of the supported peripherals for the i.MX 8M Plus, as it is built as a development board for developers to test, try and develop for the SoCs hardware. The board counts with 4 GiB of [LPDDR4](#) memory directly welded in. The figure [2.3](#) shows the mounted peripherals by this board.

Memory Map

In order to communicate with the peripherals and their registers, the NXP i.MX 8M Plus (and the Armv8-A architecture in general) uses [MMIO](#). MMIO means that communication with such registers do not happen using specific instructions, but directly by using the memory bus. The SoC manufacturers, NXP in this case, define specific physical address areas where peripheral

Figure 2.3: FRDM i.MX 8M Plus Development Board mounted peripherals



registers are accessed, and the bus redirects the operations that target those memory addresses to the peripheral hardware registers.

The memory map defines the physical address ranges assigned to each hardware component visible to the processor by the memory bus. The memory map of the i.MX 8M Plus is provided by its [TRM](#) [8], and allows knowing where the kernel should be located in memory and where the kernel's drivers should access the peripheral registers. Table 2.2 shows a simplified view of the regions defined for the NXP i.MX 8M Plus.

Table 2.2: Simplified Cortex-A53 memory map of the i.MX 8M Plus.

Start	End	Region	Size
0x0000_0000	0x0003_FFFF	Boot ROM	256 KiB
0x0090_0000	0x0098_FFFF	On-Chip RAM (OCRAM)	576 KiB
0x3000_0000	0x30FF_FFFF	Advanced IP Bus Switch (AIPS) (MMIO)	64 MiB
0x3880_0000	0x388F_FFFF	Generic Interrupt Controller (GIC)	1 MiB
0x4000_0000	0xFFFF_FFFF	DDR Memory	3 GiB
0x1_0000_0000	0x2_3FFF_FFFF	DDR Memory (extended)	5 GiB

The Boot ROM contains the firmware responsible for the initial hardware initialization and [bootloader](#) loading sequence. The [OCRAM](#) is a small on-chip SRAM available immediately

after reset, before external [DDR](#) memory is initialized, and is used for early boot stages. The [Advanced IP Bus Switch \(AIPS\)](#) regions are where the [MMIO](#) registers of most peripherals reside. The [GIC](#) register region is where the Generic Interrupt Controller is memory-mapped and configured. The [DDR](#) memory region is where the main memory resides, and is where the kernel and user processes are loaded and executed.

ARM Cortex-A53

The ARM Cortex-A53 [9] used by the i.MX 8M Plus, is a high efficiency processor that implements the Armv8-A architecture. It can be configured with one to four cores, each with a dedicated 8–64 KiB L1 instruction cache and 8–64 KiB L1 data cache, and a single shared L2 cache of up to 2 MiB.

It is the first Armv8-A processor that was aimed at providing power efficient 64-bit processing. The processor implements an in-order, 8-stage, dual-issue pipeline. In-order means that instructions are executed, and memory accesses are committed in program order. This simplifies the memory ordering model when compared to other Armv8-A out-of-order cores.

The core implements the NEON extension for [SIMD](#) and VFPv4 floating-point unit. Interrupt management is handled through a [GICv3 \(Generic Interrupt Controller Version 3\)](#), which manages interrupt distribution and prioritization among the cores.

Hardware Debugging

In order to be able to debug the developed code, the Armv8-A CPUs allow for direct hardware debugging via [Joint Test Action Group \(JTAG\)](#) or [Serial Wire Debug \(SWD\)](#). It allows communicating the target hardware, the NXP i.MX 8M Plus in this case, with a computer, creating the connection using a hardware debugger. The debugger used for this project is the J-Link EDU Mini, which is an educational simple debug probe that supports [Armv8-A](#), Armv8-M and RISC-V debugging.

Using a debugger [10] provides fundamental kernel development tools, like reading the different registers, reading and writing memory regions, and step-by-step debugging. Standard debug software programs like [GNU Debugger \(GDB\)](#) can be used in combination with a debugger connecting to the debugger through a GDB server, which allows the inspection and control of the kernel's execution state.

During the development of this project, the use of GDB has been a fundamental part of the development, and without it the development would have been impossible due to many bugs where step-by-step inspection was required.

For simpler debugging, where for example the knowledge of the internal state of an allocator is relevant information, direct [UART](#) based debugging has been used. In this kind of debugging, the kernel directly prints the relevant information. However, it is only useful for

the stages where the UART drivers are already initialized, and the debugged behaviors are not hard faults or sections where the PE hangs.

Kernel Design and Implementation

THIS chapter covers the design decisions and implementation details of the kernel developed in this project. The development is organized around the different subsystems implemented in the kernel. Each subsystem usually builds on top of the previous ones, so the sections follow the same order in which the subsystems were developed. The full source code of the kernel described in this chapter is available at https://github.com/unaiiborra/Armv8_kernel [11].

Before describing the implementation of each subsystem, the first section covers the key design decisions taken even before starting the implementation.

3.1 Design Choices

The decisions taken before the implementation shaped the architecture of the kernel from the start. These decisions allowed the implementation to follow some rules that helped the project maintain coherency during all the project's development.

Target Hardware

The selection of the target hardware was heavily influenced by previous experiences by the author of this report in bare-metal development. The *Raspberry Pi Zero 2 W* and *Raspberry Pi 5* were platforms where some small bare-metal projects were developed. The prior knowledge of basics of the Armv8-A architecture, influenced the decision of selecting this architecture as the one to develop the kernel in. RISC-V architecture was also taken into consideration, but the prior knowledge of Armv8-A ruled it out.

The prior experiences with the Raspberry Pi SoCs, also revealed significant limitations of those platforms for the development of a kernel. The documentation provided by Broadcom related to the Raspberry Pi peripherals and general hardware details is usually private, and

when public, is generally incomplete and often insufficient. Critical details are undocumented and sometimes only available through community reverse-engineering efforts.

This experience made documentation availability and quality a primary criterion for hardware selection. As described in Section 2.3, NXP generally provides public, extensive and detailed documentation, which ruled the decision to use NXP hardware. The specific decision of choosing the NXP i.MX 8M Plus as the base hardware for the project was due to the SoCs peripheral selection and multicore qualities.

Even though the kernel is developed specially for the NXP i.MX 8M Plus, the decision to develop the kernel in a hardware-agnostic way was taken. This means that the kernel by itself should be able to run in different hardware (for example a Raspberry Pi 5 and a NXP i.MX 8M Plus) with minimal changes in then kernel. The main factor that does not directly allow a kernel to run in two different chips that share the same architecture, are the driver implementations and the memory map. There are many existing solutions for unifying those aspects, allowing a kernel to not be fully dependent on the specific chip running it, which will be explored in later sections.

Monolithic or Microkernel Approach

A fundamental decision in kernel design is if a monolithic or a microkernel approach will be taken. As described in Section 2, monolithic kernels run all core services: memory management, scheduling, drivers and system calls in a single privileged address space, while microkernels keep the privileged core minimal and use as much as possible the user space.

The kernel developed in this project follows a monolithic approach. This decision was taken due to two factors:

- **Simplicity of implementation:** a monolithic kernel avoids the complexity of communicating microkernels between their components.
- **Performance:** as some [context switching](#) and communications are avoided, monolithic kernels perform better than microkernels.

The main disadvantage of this decision is that when a bug in a subsystem of the kernel happens, it can break the whole system, as all the components share the same address space and privilege level. However, the simplicity and performance advantages outweigh the disadvantages.

Programming Languages

The kernel is written mainly in C with Assembly where required by the architecture. C was chosen for its direct mapping to hardware, its maturity in bare-metal development, and the

availability of GCC-specific extensions [12], which are used throughout the project. The `gnu23` standard is used. Rust was considered but discarded due to its steeper learning curve at the low-level, it is however used for userspace programs via a custom `no_std` standard library.

3.2 Project Structure

Table 3.1 shows the main structure for the project.

Table 3.1: Top-level project directory structure.

Directory	Contents
<code>src/arm/</code>	Architecture-specific modules: MMU, exceptions, cache, SMCCC ...
<code>src/boot/</code>	Assembly boot sequence and early processor initialization
<code>src/drivers/</code>	Platform-specific drivers: GICv3, UART, timer and thermal sensor
<code>src/kernel/</code>	Kernel subsystems: memory management, scheduler, I/O, syscalls, VFS...
<code>src/lib/</code>	Shared utilities: data structures, locks, string formatting, performance monitor...
<code>include/</code>	Public headers, mirroring the <code>src/</code> structure
<code>user_examples/</code>	Userspace programs and benchmarks
<code>userspace/stl/</code>	Rust standard library for userspace programs
<code>scripts/</code>	GDB, OpenOCD and J-Link debugging scripts
<code>make/</code>	Build system configuration and compiler flags

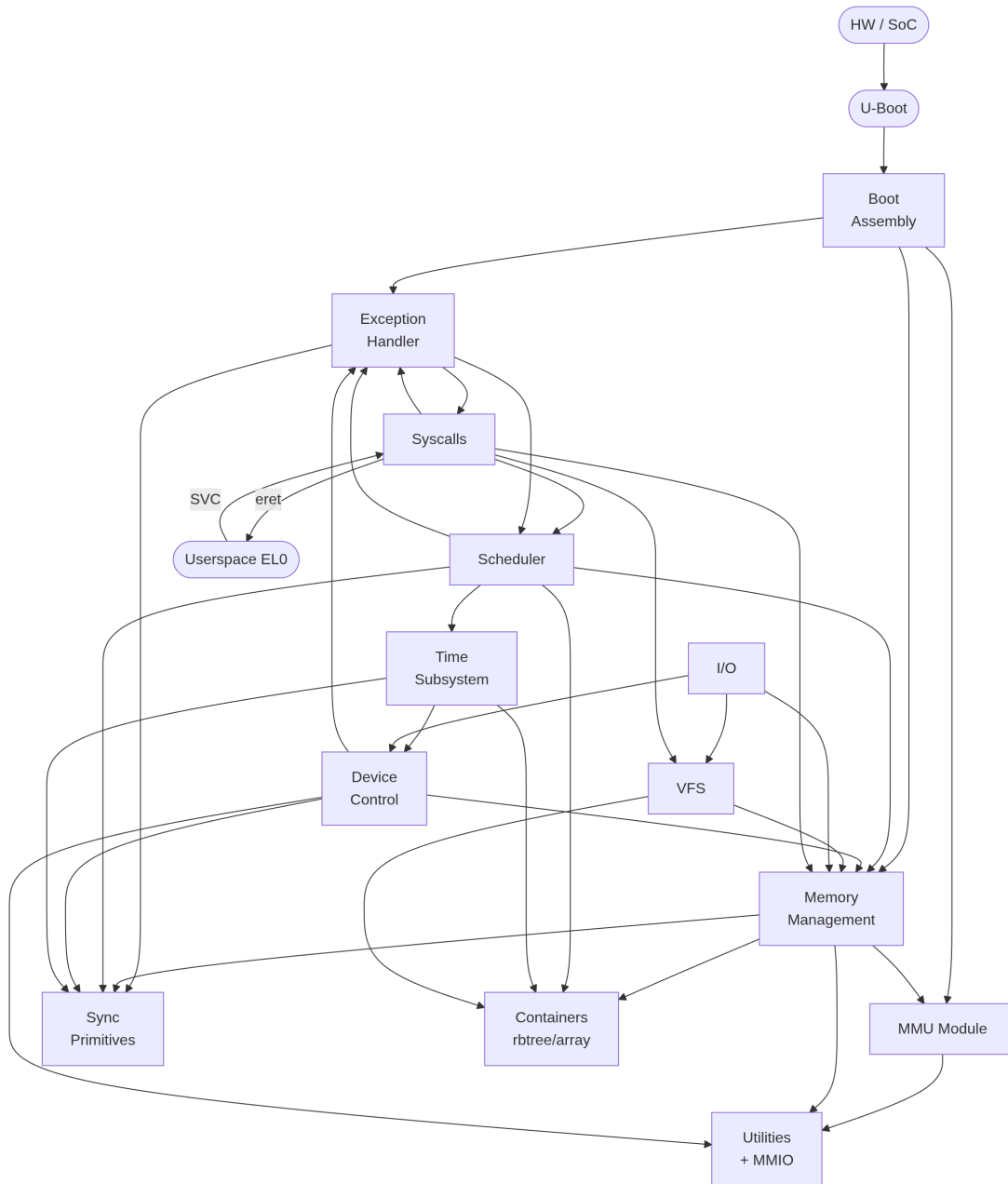
Figure 3.1 shows the dependency relationships between the main kernel subsystems, which will be described in the following sections of this chapter.

Figure A.1 shows the same dependencies in greater detail, breaking each subsystem into its individual modules and relation to the source folder structure.

3.3 Kernel Boot Stage

The kernel *boot* is the stage that begins when the [bootloader](#) loads the kernel binary into memory and jumps to its entry point address.

Figure 3.1: Project subsystem dependencies



Bootloader

The **bootloader** is the software which runs after the basic initialization done by the firmware. It is in charge of further initializing the hardware (including **DDR** memory, clocks and essential peripherals) and loading and transferring the execution to the operating system kernel.

The bootloader used in this project is *U-Boot* [13], an open-source bootloader very adopted in embedded Linux systems and supported by most **SoCs** and development boards. Choosing U-Boot was not an active design choice in this project as it is the bootloader that comes preinstalled on the hardware and is the standard bootloader supported by NXP for this SoC.

From the kernel's perspective, what matters is the state in which U-Boot leaves the hardware when the control is transferred. U-Boot leaves one core running at **EL2** in AArch64 state with the other cores deactivated. The bootloader, then jumps to the kernel entry address.

U-Boot leaves the MMU enabled in a **identity mapping** state, which means that the MMU is active and working but with a 1:1 virtual to physical mapping.

Kernel Boot

This is the first point where the kernel gains the control of the execution of the processor. The entry point must be defined in Assembly, generally defined with the `_start`, as not even the stack pointer is initialized in any known address. The boot process has been implemented the following way:

Mask DAIF exceptions

As the **Vector Table** is not configured yet, the first step is to disable all the **Debug**, **SError**, **IRQ**, **FIQ** (**exception mask bits**) () exceptions. If an exception was going to arrive, the processor would try to enter in the Vector Table address, which at this point holds an undefined value, so the result would probably end in an infinite hang. This step could actually be skipped if the kernel would only be loaded through U-boot, because it already **masks exceptions**, but as debugging and loading via GDB is also supported, this step is needed. When **GDB** loads the kernel, it forcefully stops whatever code U-Boot was executing and loads the kernel in the exact processor state it was in. Listing 3.1 shows how **DAIF** exceptions are atomically **masked**.

Listing 3.1: daifset instruction

```
// daifset is alias that generates an specific instruction
// that masks the DAIF sysreg. It can only be used with
// immediate values that represent the DAIF bits.
msr daifset, #0xF
```

Zero all the registers

Zeroing the registers is done in order to enforce more deterministic behavior, as their value is undefined at this stage [14]. This allows for better debugging, because all the changed values can be assured to have been modified by the kernel's code and not by an unknown starting value.

Clean the *block starting symbol (bss)* section

The *bss* is the section of memory that contains statically allocated variables. It is pre-defined at the linker, and it holds global variables that are not initialized from the program's binary (the kernel binary in this case). The C standard requires this section to be zeroed, so manual cleaning before jumping to C compiled code is required. Listing 3.2 shows the process of cleaning the bss section using SIMD instructions.

Listing 3.2: .bss initialization

```
_clean_bss:
    ldr    x0, __bss_start // provided by the linker-script
    ldr    x1, __bss_end   // provided by the linker-script

/* as values provided by the linker-script use high address
   space (they are signed and sign-extended), the physical
   address must be obtained. KERNEL_BASE is a macro that
   defines the smallest valid address for high address space
   (0xFFFF_8000_0000_0000). The bitwise and result is applied
   as a mask to clean the address sign. This is a process
   that will be further explained in the memory management section. */
    and    x0, x0, #~KERNEL_BASE // &= 0x7FFFFFFFFFFFFFFF
    and    x1, x1, #~KERNEL_BASE // &= 0x7FFFFFFFFFFFFFFF
    // x0 = physical address of bss_start
    // x1 = physical address of bss_end
    // store zero at v0-v3
    movi    v0.16b, #0
    movi    v1.16b, #0
    movi    v2.16b, #0
    movi    v3.16b, #0
1:  cmp     x0, x1
    b.ge    2f      // if (start_pointer >= __bss_end) break;

    // vectorial 64-byte store at x0 + post-add 64 to x0
    st1     {v0.16b, v1.16b, v2.16b, v3.16b}, [x0], #64
    b       1b      // goto 1;
2:  ret
```

Invalidate and clean the cache

While EL2 has the MMU enabled with [identity mapping](#), EL1 MMU has not been configured yet, so it is disabled. As explained in [Section 2.2.7](#), when the MMU is disabled, caches are ignored. U-Boot performs a cache flush automatically after loading a binary, so coherence is guaranteed when booting normally. However, when the kernel is loaded via a debugger, no such flush is performed. The binary is written to the cache levels but may not have been stored into memory. When transitioning to EL1, all loads happen directly from memory (as the MMU is disabled), which would still hold undefined values. This would result in constant instruction faults and an unrecoverable hang. Invalidating and flushing the caches to memory before the transition is required to support debugger based loading.

This is done by using the `_cache_flush_range` function, which uses instructions that invalidate the caches of specific cache lines, as shown in [Listing 3.3](#).

Listing 3.3: Cache flush function

```
ldr x0, =__text_start // provided by the linker
ldr x1, =__stacks_end // provided by the linker
and x0, x0, #~KERNEL_BASE // &= 0x7FFFFFFFFF
and x1, x1, #~KERNEL_BASE // &= 0x7FFFFFFFFF

// flush text, rodata, data, bss and stacks sections.
bl _cache_flush_range

...

.global _cache_flush_range
_cache_flush_range:
1: cmp x1, x0
   b.le 2f // if (region_end <= ptr) break;

   dc cvau, x0 // invalidate and flush to memory the d-cache
               // line (x0 has the pointer to the address)

   add x0, x0, #64 // add 64 (cache line size) to x0
   b 1b

2: dsb ish // data synchronization barrier only to the inner
           // shareable domain
   ic iallu // invalidate all the i-cache
   dsb ish
   isb // instruction synchronization barrier
       // (flush the processor's pipeline)
   ret
```

At this point, the kernel binary will be visible for the EL1, even if the MMU is disabled when transitioning, the registers hold deterministic values and the `bss` is clean. Transitioning to EL1 at this point is safe, the only thing still required is configuring the state we want the

processor to be in after the transition.

Setup SP_EL1

The stack region used for [EL1](#) is pre-defined at the linker-script, and will be used as the initialization stack of the kernel. Another detail to consider is that as the kernel is multicore, the stack regions for each core should be different. As the stack grows from higher to lower addresses, the stack pointer must be set at the highest address of the region. Listing 3.4 shows the stack initialization function used in this project.

Listing 3.4: Early stack initialization

```
.global _setup_stack_el1
_setup_stack_el1:
    mrs x0, MPIDR_EL1
    and x0, x0, #0xFF // core Affinity0, used as coreID

    ldr x1, =__EL1_STACK_SIZE
    ldr x2, =__stacks_el1_end
    mul x0, x0, x1      // x0 = coreID * stack_size
    sub x0, x2, x0      // x0 -= stack_bottom
    and x0, x0, #~KERNEL_BASE // &= 0x7FFFFFFFFFFFFFFF

    msr SP_EL1, x0      // set the SP_EL1 address
    ret
```

EL1 exception vector configuration

The exception [Vector Table](#) has not yet been configured for [EL1](#). If an exception was to happen when at [EL1](#), the processor would jump to the address stored at `VBAR_EL1`, which would be undefined. This would probably end in an infinite hang where the processor jumps to an invalid address that causes a new exception, leaving the processor in an irrecoverable loop.

In order to define the exception vector that the kernel will use at [EL1](#), its code must be defined in the kernel's binary. The main strategy to do so is to declare a 2048 (2^{11}) byte aligned code block, with each entry of the table aligned to 128 (2^7) bytes, as shown in Listing 3.5. Each vector entry has the instruction to jump to the specific handler for the type of the exception. The assembler will add padding to the remaining bytes of each entry. The structure of a [Vector Table](#) is explained in Section 2.2.6.

Listing 3.5: [Vector Table](#) implementation

```
.section .vectors, "ax"      // an specific subsection has been
                             // defined inside the text section
                             // to hold EL1 and EL2 vectors
.global _el1_vector_table
.align 11                   // align the table to  $2^{11}$  (2048)
```

```

_el1_vector_table:
// [Entries 0..3] Current EL1t, SP_EL0. The exceptions happening
// from EL1 will enter from these entries when SP_EL0 is the
// stack pointer being used as sp
.align 7 b _el1_cur_sp0_sync // branch to the _el1_cur_sp0_sync
                             // function. The b instruction will
                             // be set at the next '2 aligned
                             // address.
.align 7 b _el1_cur_sp0_irq // branch to _el1_cur_sp0_irq function
.align 7 b _el1_cur_sp0_fiq // ...
.align 7 b _el1_cur_sp0_serror
// [Entries 4..7] Current EL1h, SP_EL1. The exceptions from EL1
// will enter from these entries when SP_EL1 is the stack pointer
// being used as sp
.align 7 b _el1_cur_spx_sync
.align 7 b _el1_cur_spx_irq
.align 7 b _el1_cur_spx_fiq
.align 7 b _el1_cur_spx_serror
// [Entries 8..11] Lower EL, AArch64. All the exceptions
// from userspace (EL0) will enter from these entries
.align 7 b _el1_low_a64_sync
.align 7 b _el1_low_a64_irq
.align 7 b _el1_low_a64_fiq
.align 7 b _el1_low_a64_serror
// [Entries 12..15] Lower EL, AArch32. They are not actually
// used in this kernel implementation as AArch32 will not
// be implemented
.align 7 b _el1_low_a32_sync
.align 7 b _el1_low_a32_irq
.align 7 b _el1_low_a32_fiq
.align 7 b _el1_low_a32_serror

```

The address of the [Vector Table](#) must be then stored at the VBAR_EL1 system register.

Transition to EL1

The `_switch_to_el1` function performs the processor configuration and [Exception Level](#) transition to [EL1](#). It receives the EL1 entry address in `x0` and a context value in `x1`. The code has been defined inside a function and not directly at the kernel entry address code because it will be used for later multicore initialization.

The first configured [sysreg](#) is SCTLR_EL1. Its configuration is set so the MMU is left disabled, exception returns from EL1 are defined as context synchronizing events. CPACR_EL1 configuration enables [SIMD](#) and floating-point instructions at EL1 and [EL0](#). SP_EL1 and VBAR_EL1 are set as described in the previous sections.

For the [EL2 sysregs](#), CNTVOFF_EL2 is zeroed so that the virtual timer counter from EL1 matches the physical counter. HCR_EL2 is configured to allow AArch64 as the execution state for EL1, with all trap bits cleared, as no hypervisor functionality is implemented.

SPSR_EL2 defines the processor state to restore when the `eret` instruction is executed. DAIF exceptions are `masked` and EL1h is selected as the target mode. ELR_EL2, which defines the address `eret` jumps to, is set to the `kernel_entry` function, declared in C.

Listing 3.6 shows the implementation of the `_switch_to_el1` function.

Listing 3.6: `_switch_to_el1` function implementation

```
// x0: entry address for EL1 (kernel_entry is the parameter
// provided at kernel boot)
// x1: ctx
.global _switch_to_el1
_switch_to_el1:
    mov x8, x0 // save entry address in x8
    mov x9, x1 // save ctx in x9

// EL1 sysreg configurations
// SCTL_EL1 (System Control Register)
// [11] EOS: 1 = exception returns from EL1 are context
//           synchronizing events
// [0] M: 0 = MMU disabled (left at reset value)
    mov x0, #(1 << 11)
    msr SCTL_EL1, x0

// CPACR_EL1 (Architectural Feature Access Control Register)
// [21:20] FPEN: 0b11 = SIMD and floating-point instructions
//           do not trap at \gls{EL0} or EL1
    mov x0, #(3 << 20)
    msr CPACR_EL1, x0

    adr x0, _el1_vector_table
    msr VBAR_EL1, x0

    bl _setup_stack_el1

// EL2 sysreg configurations
// CNTVOFF_EL2 (Counter-Timer Virtual Offset Register)
// Set to zero so the virtual counter seen from EL1
// matches the physical counter
    mov x0, #0
    msr CNTVOFF_EL2, x0

// HCR_EL2 (Hypervisor Configuration Register)
// [31] RW: 1 = EL1 execution state is AArch64
// Remaining bits control traps to \gls{EL2}, cleared since no
// hypervisor functionality is implemented
    mov x0, #(1 << 31)
    msr HCR_EL2, x0

// SPSR_EL2 (Saved Program Status Register)
// [3:0] M: 0b0101 = EL1h (EL1 with SP_EL1)
// [4] M[4]: 0 = AArch64 execution state
// [6] F: 1 = FIQ masked
// [7] I: 1 = IRQ masked
```



```

// [8]   A: 1 = SError masked
// [9]   D: 1 = Debug exceptions masked
mov x0, #0x3c5
msr SPSR_EL2, x0

// ELR_EL2: branch target when eret instruction
// is executed
msr ELR_EL2, x8

mov x0, x9 // pass ctx as first argument to kernel_entry
isb
// jump to ELR_EL2 address and set the processor to the
// state configured at SPSR_EL2
eret

```

During this section the boot implementation for [EL1](#) transition has been explained. Other minor steps are also done at this stage, like setting up the [EL2 Vector Table](#) and [SP_EL2](#) initialization, but they have not been explained due to not being very relevant for the kernel. The [EL2 Vector Table](#) configuration is mainly done for error control and debugging purposes and is not required.

Listing 3.7 shows the full kernel boot entry code implementation.

Listing 3.7: Kernel boot entry code

```

#include <kernel/mm.h>

.section .text.boot
.global _start

_start:
    // disable DAIF exceptions
    msr daifset, #0xf

    msr spsel, #1 // select SP_EL2 as sp
    isb

    // setup \gls{EL2} stack
    mrs x0, MPIDR_EL1
    and x0, x0, #0xFF // core Aff0, used as coreID
    ldr x1, =__EL2_STACK_SIZE
    ldr x2, =__stacks_el2_end

    mul x0, x0, x1 // x0 = coreID * stack_size
    sub x0, x2, x0 // x0 = stack end - coreID * stack_size
    and x0, x0, #~KERNEL_BASE

    mov sp, x0
    mov x0, #0
    msr sp_el0, x0

    bl _init_gpr // zero the registers
    bl _el2_init_vector // set \gls{EL2} vector table

```

```

b1 _setup_stack_el1
b1 _clean_bss

ldr x0, __text_start
ldr x1, __heap_start
and x0, x0, #~KERNEL_BASE
and x1, x1, #~KERNEL_BASE
b1 _cache_flush_range

adr x0, kernel_entry // EL1 entry address
mov x1, #0           // ctx
b _switch_to_el1

```

3.4 Kernel Initialization

After the basic processor configuration and transition to [EL1](#), the kernel has just entered the `kernel_entry` function, and entered C code for the first time. However, the kernel is still in a raw state. The MMU is disabled and the different subsystems of the kernel are uninitialized.

At this stage, the initialization of the kernel is required to allow all the subsystems to work and allow user processes to run.

Listing 3.8: `kernel_entry` initialization calls

```

noreturn void kernel_entry()
{
    if (get_cpuid() == 0) {
        if (!mm_kernel_is_relocated())
            kernel_early_init();
        else
            kernel_init();
    }

    ...
}

```

As shown on Listing 3.8, the kernel first checks that only the core 0 can initialize the kernel, as when secondary cores are woken up later, they also enter through `kernel_entry`. `mm_kernel_is_relocated` checks if the current execution address has the sign bits set. This check is done because early initialization stage returns to the `kernel_entry` function from the start with the kernel relocated at the high address space.

The kernel initialization is divided into two substages:

Early Initialization

The early initialization stage is responsible for initializing the memory management subsystem and relocating the kernel to the high address space. It runs almost entirely with the MMU

disabled and without access to standard memory allocation functions.

The entry point of this stage is `mm_early_init`, which performs the following steps in order:

1. **Memory layout discovery** (`mm_info_init`): reads the physical memory layout available to the kernel and initializes memory management helper functions.
2. **Early allocator initialization** (`early_kalloc_init`): set up a simple allocator used during early initialization, before the full memory management subsystem is available.
3. **MMU early initialization** (`mm_mmu_early_init`): perform the minimum MMU configuration required to enable virtual memory.
4. **Identity mapping** (`early_identity_mapping`): map the kernel's physical address range to the same virtual addresses, allowing execution to continue after the MMU is enabled.
5. **Page and virtual allocator initialization**: initialize the physical page allocator and the virtual memory allocator.
6. **Kernel relocation** (`mm_reloc`): relocate the kernel to the high address space and resets the stack pointer. Control returns to `kernel_entry` with the kernel running at its final virtual address.

Early initialization functions until enabling [identity mapping](#) have been declared with the `safe_early` attribute, shown in Listing 3.9. It is a macro that forces no optimizations and does not use floating-point (nor [SIMD](#)) registers. This is done because the C compiler uses floating-point registers to improve memory operations like struct initialization. These operations sometimes don't follow the pre-MMU alignment check rules, and causes an [Alignment fault](#), which at this stage of the kernel is irrecoverable.

Listing 3.9: `safe_early` macro definition

```
#define safe_early [[gnu::target("general-regs-only"), gnu::cold, gnu::optimize("O0")]]
```

The details of the early memory management subsystem initialization are covered in depth in Section 3.7.

Standard Initialization

The standard initialization stage runs after the kernel has been relocated to the high address space. At this point the MMU is enabled, and basic memory allocation is available. This stage initializes the remaining kernel subsystems in the following order:

1. **Memory management** (`mm_init`): initialize the kernel heap allocator.
2. **Scheduler** (`scheduler_init`): set up the thread scheduler.

3. **Device and interrupt controllers** (`device_ctrl_init`, `irq_ctrl_init`): initialize the driver management subsystem and registers available devices and initializes interrupt routing.
4. **Driver initcalls**: run all registered driver initialization functions.
5. **Exception unmasking**: enable **IRQ**, **FIQ**, **System Error (SError)** and debug exceptions.
6. **GIC initialization**: configures the Generic Interrupt Controller for the current core.
7. **I/O subsystem** (`io_init`): initialize kernel printing and logging.
8. **Timer** (`time_ctrl_init`): initialize the system clock and timer subsystem.

Each of these subsystems is described in detail in the sections that follow.

3.5 Architecture Modules

Architectural Armv8-A-specific modules have been implemented isolated from the kernel subsystems, and provide an abstraction layer over the hardware. These modules are intended to be used by the different kernel subsystems to interact with the hardware, but not the other way around (the modules do not know the subsystems, they are kernel-agnostic). This provides isolation over these critical modules, avoiding bugs that might come from other kernel subsystems. As explained before, the kernel implementation is monolithic, so the modules are not totally independent of the kernel, as they are compiled and linked together. They just provide a common interface for the kernel subsystems to use.

3.5.1 MMU

In order to use virtual memory, the MMU [15] must be implemented. The MMU module implementation should abstract the hardware details related to the MMU, and provide easier control over its features. The designed MMU module focuses its API into two related levels: core-level and mapping-level.

MMU Design

The **core-level API** provides low level control over core-level MMU configurations. It allows defining a core-local control struct that holds the MMU state for a specific core. Each `mmu_core_handle` struct allows holding the core's configuration and two mappings. Listing 3.10 shows the main provided functions of the core-level API.

Listing 3.10: MMU core-local API main functions

```
bool mmu_core_handle_new(
    mmu_core_handle* out,
    mmu_mapping*     io_mapping,
```

```

mmu_mapping*    hi_mapping,
bool            hi_enable,
bool            lo_enable,
bool            d_cache,
bool            i_cache,
bool            align_trap);

bool mmu_core_set_lo_va_bits(mmu_core_handle* ch, uint8_t bits);
bool mmu_core_set_hi_va_bits(mmu_core_handle* ch, uint8_t bits);
bool mmu_core_set_lo_granularity(mmu_core_handle* ch, mmu_granularity g);
bool mmu_core_set_hi_granularity(mmu_core_handle* ch, mmu_granularity g);

```

The **mapping-level API** provides functions that allow managing the different mappings. Operations such as virtual-to-physical mapping and unmapping are supported in the API. A mapping represents a virtual address space (high or low) that is mapped to a physical address space.

Each `mmu_mapping` owns an independent L0 translation table by referencing the allocated table address via a pointer. Intermediate page tables (L1, L2, L3) are allocated when `mmu_map` requires it, and freed recursively when `mmu_unmap` removes a page table (or when freeing the whole mapping). The specific functions for table allocation and deallocation are defined per mapping to allow different allocators for different initialization stages and post-initialization stages.

Another relevant variable stored per mapping is the `physmap_offset`, which refers to the physical to virtual memory offset of the allocated page tables. Since page table entries store physical addresses, but the kernel accesses memory through virtual addresses after relocation, this offset allows the mapping functions to convert between the two without depending on a global state. This design means the same MMU module can be used both before and after kernel relocation. During early initialization, the offset is zero (**identity mapping**), and after relocation it is set to the high address space base (`0xFFFF_0000_0000_0000`). The allocator provided to each mapping must guarantee that allocated the memory virtual address follows this same offset. Listing 3.11 shows the most common functions provided by the MMU header related to mapping operations.

Listing 3.11: MMU mapping-level API main functions

```

// struct that represents a page configuration. Internally is packed as a 32-bit
// integer, however, the API struct fields have strict alignment to avoid pre-\gls*{MMU}
// enabling alignment faults.
typedef struct {
    _Alignas(4) uint8_t    attr_index;
    mmu_access_permission ap;
    uint8_t                shareability;
    _Alignas(4) bool       non_secure;
    _Alignas(4) bool       access_flag;
    _Alignas(4) bool       pxn;
    _Alignas(4) bool       uxn;
}

```

```

    _Alignas(4) uint8_t    sw;
} mmu_pg_cfg;

// creates a new mmu_mapping struct with the given parameters
static inline mmu_mapping mmu_mapping_new(
    mmu_tbl_rng      rng,
    mmu_granularity  g,
    size_t           va_addr_bits,
    uintptr_t        physmap_offset,
    mmu_allocator     allocator,      // void* (*mmu_allocator)(size_t bytes)
    mmu_allocator_free allocator_free // void (*mmu_allocator_free)(void* addr)
) {
    ...
}

// maps a virtual address range to a physical address range, size must be aligned
// to the page size
mmu_map_result mmu_map(
    const mmu_mapping* m,
    vuintptr_t        va,
    puintptr_t        pa,
    size_t            size,
    mmu_pg_cfg         cfg,
    mmu_op_info*       info);

// unmaps a virtual address range, setting NULL descriptors at the specified range,
// size must be aligned to the page size
mmu_unmap_result mmu_unmap(
    const mmu_mapping* m,
    vuintptr_t        va,
    size_t            size,
    mmu_op_info*       info);

// walks one page and returns the page information. treats the result as if it
// found a last level descriptor, even if found at a higher level (if va is not
// aligned to the real block it gives the corresponding pa to the provided va)
void mmu_walk_page(const mmu_mapping* m, vuintptr_t va, mmu_walk_result* result);

```

MMU Implementation

The main **Memory Management Unit (MMU)** API functions are `mmu_map` and `mmu_unmap`. They walk through the page tables using calculating the L0-L3 indices by splitting the virtual address, as described in Section 2.2.7.

`mmu_map` walks the page tables using these indices until reaching the last level descriptor and configuring the descriptor to point to the required physical address range. If a null descriptor is found while walking, a new page table is allocated, and the descriptor is updated to point to the new page table's physical address. If a block descriptor is found, if the block descriptor already points to the required address range, the iteration is skipped. If the block

did not point to the required address range, a new page table is allocated, whose descriptors are set to point to the block's previously configured addresses. Finally, the specific descriptor that was going to be updated is set to point to the required physical address.

`mmu_unmap` works similarly. It walks the page tables until reaching the appropriate level for the range being unmapped. If the unmapping range is aligned to a block boundary and covers a full block, a null descriptor is set directly at the block level (L1 or L2) without descending to L3, and the corresponding page table is freed recursively. Else, the walk descends to L3 and sets null descriptors.

These steps are done per page to be mapped or unmapped, by adding the page size to the virtual address parameter variable each iteration. This means that if mapping 4 consecutive pages, the tables are traversed 4 times, once for each page.

An optimization implemented in `mmu_map`, is to check if the requested virtual address range is aligned to a block boundary. If so, block descriptors are used instead of page tables, avoiding unnecessary memory allocations. Another optimization is planned for `mmu_unmap`, where for example a completely null L3 table is detected, it is freed, and a null descriptor is set in the L2 table.

Figure 3.2 shows a simplified flowchart of the implementation of `mmu_map`.

3.5.2 Exception Handling

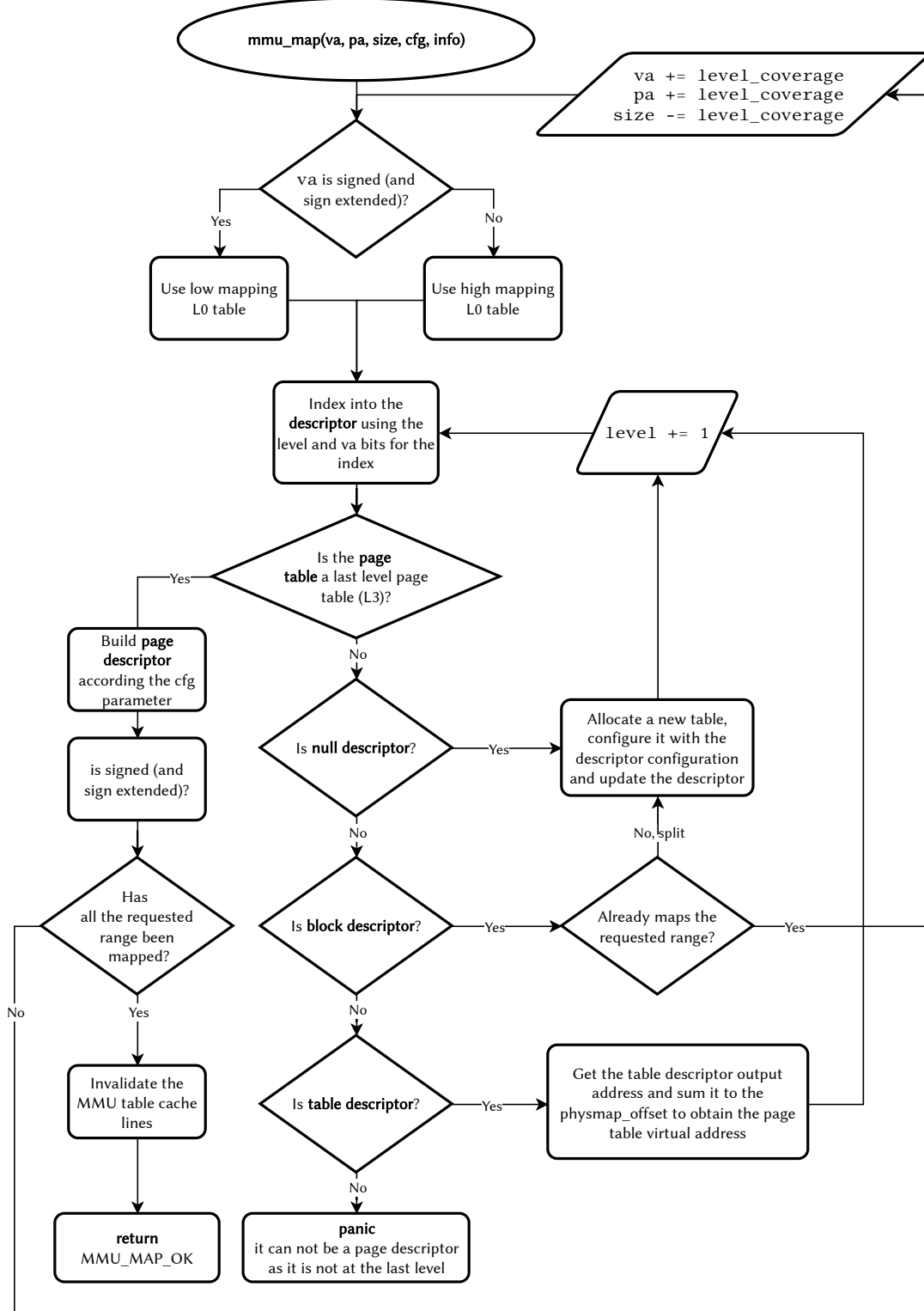
The exception handler is the kernel module in charge of managing and redirecting exceptions to the different handlers depending on the cause of the exception.

The exception handler is in charge of remembering the full state the processor was in when the exception occurred, so it can handle the cause of the exception, and restore that state if the cause can be handled. If a certain exception cannot be handled, the panic function will be called and the kernel will remain in an irrecoverable state.

When an exception happens, the `ELR_EL1` and `SPSR_EL1` are automatically written by the hardware. `ELR_EL1` holds the address of the instruction where the program counter was when the exception happened. `SPSR_EL1` holds the state of the processor when the exception happened, like the endianness, instruction flags or `DAIF` state. Both system registers, also are used when returning from an exception when the `eret` instruction is used, where `ELR_EL1` defines the address the program counter will start executing, and `SPSR_EL1` the state the processor will be set to. After an exception handler finishes, the `eret` instruction is always called.

In order to store the state the processor was in before the exception, the `arm_ctx_t` struct has been defined, shown in Listing 3.12. This struct is used to copy the state of the processor just when an exception happens.

Figure 3.2: mmu_map simplified flowchart



Listing 3.12: arm_ctx_t struct definition

```

#define XREG_COUNT 31
#define VREG_COUNT 32

typedef uint64_t    gpr_t;
typedef __uint128_t vreg128_t;
typedef uint64_t    sysreg_t;
typedef uint64_t    sp_t;

typedef struct {
    sysreg_t fpcr; // Floating-point Control Register
    sysreg_t fpsr; // Floating-point Status Register
    sysreg_t elr;  // Exception Link Register (return address after eret)
    sysreg_t spsr; // Saved Program Status Register (processor state to be configured after eret)
    sp_t sp_elx;   // SP_EL<X> (defined by the spsel value referenced in spsr)
    gpr_t      x[XREG_COUNT]; // x0-x30
    vreg128_t v[VREG_COUNT]; // v0-v31
} arm_ctx_t;

```

As explained in Section 2.2.6, when an exception happens, the program counter is set to the address of the **Vector Table**, plus an offset that varies on different conditions shown in Table 2.1 (IRQs, Sync, the stack pointer being used...). Each vector entry is configured to call the different handlers in charge of managing the cause of the exception.

Each of these handler calls is wrapped with the `ectx_store` and `ectx_load` assembly macros (exception context store/load). These macros, shown at Listing B.1 are used to wrap the branch with link instruction to the handler with instructions that are in charge of storing/restoring into the stack, the values defined at the `arm_ctx_t` struct. The macro `ectx_store` returns into `x0` the pointer to the struct stored in the stack. The macro `ectx_load` can also be configured to load an `arm_ctx_t` struct from a pointer stored in `x0`, instead of from the stack pointer, which is useful for **context switches** to EL0. Listing 3.5 shows how the kernel's **Vector Table** is configured and each call to its handler is wrapped with the context saving macros.

The different exception cause handlers implemented for this project are shown in Table 3.2. For the cases where no exception handler is implemented, a macro that panics and shows the exception case has been used as the implementation for such causes.

Irq Dispatcher

The `irq_dispatch` function handles interrupts exceptions thrown both from EL0 or EL1. It uses the registered device for interrupt handling to detect the interrupt ID that caused the exception, and calls the handler registered for such instruction in the interrupt table. The interrupt table is an array that holds callbacks to the different interrupt handlers registered, allowing $O(1)$ access to the different handlers. How the device for interrupt handling is registered and used is described in Section 3.5.2.

Table 3.2: AArch64 [Vector Table](#) entries and their handlers

Source	Type	Handler
Current EL, SP_EL0	Synchronous	<i>Not implemented^a</i>
	IRQ	<i>Not implemented^a</i>
	FIQ	<i>Not implemented^b</i>
	SError	<i>Not implemented^c</i>
Current EL, SP_ELx	Synchronous	<i>Not implemented^a</i>
	IRQ	irq_dispatch
	FIQ	<i>Not implemented^b</i>
	SError	<i>Not implemented^c</i>
Lower EL, AArch64	Synchronous	exception_handler_sync
	IRQ	irq_dispatch ^d
	FIQ	<i>Not implemented^b</i>
	SError	<i>Not implemented^c</i>
Lower EL, AArch32	Synchronous	<i>Not implemented^e</i>
	IRQ	<i>Not implemented^e</i>
	FIQ	<i>Not implemented^e</i>
	SError	<i>Not implemented^e</i>

^a SP_EL0 is not used in [EL1](#), triggering this entry indicates a kernel bug.

^b FIQ is not used, the GIC is configured to route all interrupts as IRQ.

^c SError would mean an error in the kernel/hardware, it is routed to panic.

^d IRQ from [EL0](#) additionally calls the scheduler functions before and after dispatch, enabling preemptive thread switching on timer interrupts.

^e AArch32 execution is not supported.

Sync Exception Handler

The `exception_handler_sync` function manages synchronous exceptions from [EL0](#). It reads the `ESR_EL1` system register to obtain the Exception Class (EC) field, which tells the cause of the exception, and dispatches the required handler. [Table 3.3](#) shows the exception classes handled by the kernel implemented in this project, which are wrapped with the functions required for scheduling.

3.6 Common Modules

The development of a kernel requires a set of base building blocks that are shared across its subsystems. Instead of reimplementing these utilities in each component, they have been centralized under the `src/lib/` path, allowing for code reuse and consistency throughout the project. This section covers the main shared modules developed for this purpose. These kinds of modules are usually provided by the programming languages standard libraries, however,

Table 3.3: Synchronous exception classes and their handlers

Exception Class	Handler
ESR_EC_SVC_AARCH64	System call from AArch64. Dispatches via <code>sysc64_dispatch</code> , explained at Section 3.13.
ESR_EC_DABT_LOWER_EL	Data abort from EL0. Dispatches via <code>page_fault_handler</code> , explained at Section 3.7.12.
ESR_EC_IABT_LOWER_EL	Instruction abort from EL0. The faulting task is terminated via <code>terminate_task</code> , which defers the killing of the task's threads.
All other causes	Not handled. A debug trace log is printed and execution continues or panics depending on the cause.

they are dependent on the operating-system they are intended to be running at. As a kernel runs in bare-metal, these kinds of standard library modules can not be used and must be implemented from scratch.

Common Data Structures

On the data structures side, a **dynamic array** provides a resizable, contiguous-memory allocations used for collections whose size is not known at compile time, while a **red-black tree** offers an efficient self-balancing binary search tree that guarantees $O(\log n)$ insertion, deletion and lookup operations.

Dynamic Array

A dynamic array is a contiguous array that grows and shrinks automatically as elements are inserted or removed. Unlike a fixed-size array, it manages its own backing allocations, resizing it when needed while preserving $O(1)$ complexity for push and pop operations and $O(1)$ random access by index.

The kernel provides a dynamic array implementation through the `kvec` type. The buffer that actually holds the data is always sized to a power of two in bytes, starting at a minimum of 64 bytes. When the buffer is full, `kvec_push` doubles its capacity and copies the existing elements into the new allocation. When popping, `kvec_pop` halves the buffer size when the number of elements falls to or below a quarter of the current capacity, keeping memory usage proportional to the number of elements. A minimum buffer size of 8 elements is defined to prevent from excessive reallocations on small vectors.

Because C does not implement generics, `kvec` stores the element size and alignment at runtime and uses raw `void*` pointers internally. The `kvec_new(T)` macro initialises the

struct with the correct `sizeof(T)` and `_Alignof(T)` values. Element access is provided through `kvec_get_copy`, which copies the element out to a caller-supplied buffer, and `kvec_get_mut`, which returns a direct pointer into the backing array for in-place mutation. The `kvec_dataT(T, kvec)` macro provides the raw buffer pointer as a typed array when direct iteration is preferred.

The internal allocations of the vector are performed by the standard kernel allocator, which will be later described in Section 3.7.11.

Red-Black Tree

A red-black tree is a self-balancing binary search tree in which every node carries a color attribute, either red or black, and follows a set of rules that ensure proper balancing. The root is always black, red nodes can only have black children, and every path from the root to a null leaf contains the same number of black nodes. These rules limit the height of the tree to $2 \log_2(n + 1)$, guaranteeing $O(\log n)$ complexity in the worst case for insertion, deletion, and lookup. When an insertion or deletion would violate the invariants, the tree is restored through a sequence of recolorings and rotations. The recoloring and rotation operations are not expensive, as they only affect nodes close to the node where the operation is performed.

The kernel provides a red-black tree implementation highly inspired after the implementation found in the Linux kernel [16, 17]. Each node stores its parent's pointer and color bit into a single packed field, using the fact that node pointers are naturally aligned to 8 bytes, so they always have their least-significant bit free. This avoids a separate field per node and improves cache locality.

The node layout, shown in Listing 3.13, includes a union for the key, supporting signed and unsigned 64-bit integer keys directly, as well as an arbitrary-byte array `T[]` for callers that provide a custom comparator callback. The API exposes three insertion and lookup variants: `rbt_insert_i64/rbt_find_i64`, `rbt_insert_u64/rbt_find_u64`, and a generic `rbt_insert/rbt_find` pair that accepts an `rbt_condition_t` callback. The callback variant allows callers to define custom return codes apart from `EQUALS`, `LESS`, and `GREATER`, providing greater information of why an operation was rejected when implemented at different subsystems. The `rb_header_t` struct is designed to be embedded as a header in caller-defined node structs, allowing each subsystem to append its own fields and data after the tree metadata. Nodes are intended to be allocated by the caller itself, allowing for different allocation methods.

Listing 3.13: Red black tree node header struct

```
// private struct representation for red black tree nodes
typedef struct rbtnode {
    struct rbtnode *left, *right;
    uint64_t        packed; // bits[63:1](struct rbtnode *parent), bit[0](color)
    union { // Allows for the use of different keys for different implementations of the rbtree
```

```
uint64_t key_unsigned;
int64_t key_signed;
uint8_t T[];
};
} rbtnode_t;

// at public rbtree.h header, used as an opaque struct used as the node header
typedef struct {
    uint64_t _[3];
} rb_header_t;
```

Common Utilities

On the utilities side, the shared modules cover several areas required for many subsystems of the kernel.

Memory utilities offer low-level functions for memory manipulation, including copying and zeroing memory regions, which are typically provided by the C standard library but must be implemented from scratch in a bare-metal context. These functions have been implemented in Assembly due to their relevance on performance, and use [SIMD](#) instructions when alignment and size allows it. Address manipulation helper functions and types for physical and virtual memory management have also been defined.

String and formatting utilities offer basic string operations and formatting functions, enabling string manipulation and kernel messaging without relying on any environment.

Common headers merge a set of compiler attributes (`stdattribute.h`), branch prediction hints (`branch.h`), bitfield helpers (`stdbitfield.h`), alignment macros (`align.h`), unit definitions for memory and time (`unit/`), and [MMIO](#) peripheral register types and macro definitions (`mmio/`), providing a consistent foundation for the rest of the code.

Performance monitoring and **metrics** modules expose an abstraction over the hardware cycle monitoring and a generic monotonic clock [sysregs](#) to measure and track the execution behaviour of the kernel at runtime. This module will be further explained at Section 4.1.

Finally, given their importance to the correct operation of a multicore kernel, **synchronization primitives** are covered in their own dedicated subsection below.

Synchronization Primitives

Synchronization primitives are mechanisms that prevent concurrent accesses to shared resources from producing inconsistent state. In a multicore kernel, multiple CPUs may execute kernel code simultaneously, and interrupt handlers may preempt ongoing operations at any point, so careful coordination is required whenever shared data structures are accessed.

The kernel implements three families of synchronisation primitives, each targeting a different parallel or concurrent scenario.

IRQ lock: the simplest lock is the `IRQ` lock, which disables all `maskable` interrupts on the current CPU by setting the `DAIF mask` register. `irqsave` saves the current interrupt flags and `masks interrupts` atomically, while `irqrestore` restores the saved flags. This is enough to protect data shared with an interrupt handler.

Spinlock: the spinlock protects data shared across multiple CPUs. It is implemented as an `atomic_int` flag with a WFE/SEV event loop. A CPU waiting to acquire the lock issues a `wfe` instruction, which suspends it until another CPU executes the `sev` instruction (or an interrupt wakes it up), causing an event on release, reducing bus contention compared to a busy-wait loop. Acquisition uses `atomic_compare_exchange_weak` with `memory_order_acquire`, and release uses `atomic_store` with `memory_order_release`, which provides the necessary memory ordering guarantees. Both `IRQ` and CPU locks can be combined via `spinlock_acquire_irqsave` and `spinlock_release_irqrestore` when a critical section must also be protected against interrupts on the local CPU.

CPU lock: the CPU lock (`cpulock_t`) is a recursive spinlock that tracks which CPU core holds it and allows that core to reenter the lock without deadlocking, maintaining the recursion count. This is useful for protecting data that may be accessed from nested paths on the same core, such as from both a normal execution context and an exception handler. The unique CPU identifier is read at runtime from the `MPIDR_EL1` system register. Like spinlock, it can be paired with `IRQ` saving for combined protection.

Scoped locking macros: all three locks are also available through scoped locking macros `irqlocked`, `spinlocked`, `cpulocked`, and their `_irqsave` variants. These macros wrap a `for` loop and use the GCC `cleanup` attribute to release the lock automatically when the scope is exited, regardless of the escape method (`break`, `return`, or fall off the end). This is very useful as it eliminates a class of lock-release omission bugs.

3.7 Memory Management Subsystem

The memory management subsystem is one of the most complex and the biggest implemented subsystem in this project. It is the subsystem in charge of managing the MMU, and allowing dynamic virtual and physical region allocations. It also helps manage the userspace memory by controlling memory assignments to processes. Most other subsystems use it, and is one of the pillars that sustain the kernel.

It has been designed in a modular way, with different submodules independent between them, and higher level modules that use them together:

- **Memory management MMU module**
- **Kernel allocators**
 - **early_kmalloc**: used for initialization of the more complex allocators.
 - **page_allocator**: physical page allocator.
 - **vmalloc**: virtual page allocator.
 - **raw_kmalloc**: raw page allocator (physical + virtual).
 - **cache_malloc**: manages smaller allocations.
 - **kmalloc**: allocates different sized regions.
 - **uregion**: userspace region allocator and handler.
- **Page fault handler**: handles [EL1](#) and [EL0](#) page faults.
- **ELF loader**: loads [Executable and Linkable Format \(ELF\)](#) formatted files into memory.

3.7.1 Memory Layout

Before describing the individual modules of the memory management subsystem, understanding how the kernel's address space is organized is useful. The layout is defined by the linker script and determines where each section of the kernel's binary and pre-reserved memory regions live in physical and virtual memory.

Following the standard Linux, and most kernel's address configuration, 48-bit virtual addresses will be used for the implementation of the project. This configuration allows for two big 256 TiB virtual memory spaces that can be configured with the smaller granularity available (4 KiB), to avoid internal [fragmentation](#).

Virtual Address Space

As described in [Section 2.2.7](#), the address space is split in two regions separated by a big gap. The addresses located at the gap are considered non-[canonical](#), and the valid addresses are considered [canonical](#).

The kernel is located at the high address space, starting at `KERNEL_BASE` (`0xFFFF_8000_0000_0000`), and user processes are located at the low address space starting at `0x0`. This separation is made by the MMU using `TTBR0_EL1` for user space and `TTBR1_EL1` for kernel space. Because of this design, switching between user processes only requires changing `TTBR0_EL1`, without affecting the kernel mappings.

Kernel Binary Layout

The kernel's binary is loaded by U-Boot into physical [DDR](#) memory starting at `KERNEL_PA_BASE`, which is the DDR base address offset by a protected region reserved

by [Trusted Firmware-A \[18\]](#) (TF-A) (`__TFA_PROTECTED_SIZE`)¹. The linker script maps this physical region to the high virtual address space using the `AT` directive, which defines separate load addresses (physical) and virtual addresses for each section.

The abbreviations “PA” and “VA” have been used in many variable names to define a “physical address” and a “virtual address” respectively.

`__TFA_PROTECTED_SIZE` is a 2 MiB sized area located at the [DDR](#) start address. In the Linux memory layout for the i.MX 8M Plus, that region is reserved for [DMA](#) devices. TF-A (Trusted Firmware-A), has that region marked as an unsafe for core wake up entry points, and will return an error code.

This was a found unexpected behavior and the cause was discovered by looking at the TF-A repository. There, the required condition to wake up cores was found: the physical address must be between `0x4020_0000` and [DDR](#) end address, even if [DDR](#) starts at `0x4000_0000`. Looking at the Linux `.dtsi` files for the i.MX 8M, that region appears as reserved for [DMA](#) devices, confirming why TF-A reserves that zone. Operations like reading, writing and executing are allowed at that region by the firmware, so it is used be for heap pages.

A solution for allowing the use of this region for core wake up, could be to modify the condition at the source code of TF-A and compiling and using the modified firmware. However, this option was discarded to allow the use of the standard TF-A binary for this [SoC](#). This is why the kernel binary load address has been defined as `0x4020_0000` and not as [DDR](#) start `0x4000_0000`.

There is another memory region protected by the [Secure World](#). [OP-TEE](#), lives between the addresses `0x5600_0000` and `0x5800_0000` (32 MiB). Any read or write operation that ends in that physical address range will cause an exception, as it is a secure-world protected area. This region is not excluded at the linker script, as it lives far into free [DDR](#) zone and is not directly related to the kernel’s binary. It is avoided by allocators using other mechanisms. This region was discovered while stress-testing the `page_allocator`, and the source of the exception was found when comparing the fault addresses with the Linux i.MX 8M `.dtsi` file.

The kernel binary is organized into the following sections, each aligned to 4 KiB page boundaries:

1. **.text**: executable code, including the boot entry point (`.text.boot`) and the exception [Vector Table](#) (`.vectors`). The kernel’s entry point (`_start`) is located at the starting position of this section.
2. **.rodata**: read-only data, constants, and the `.kernel_initcall` section, which holds an array of function pointers to driver initialization functions called during standard initialization.

¹TF-A is a reference implementation of [Secure World](#) software for Armv7-A, Armv8-A and Armv9-A processors [18].

3. **.data**: initialized global and static variables.
4. **.bss**: uninitialized global and static variables. This section is not stored in the binary and must be zeroed by the boot code before entering C, as required by the C standard.
5. **.stacks**: per-core stacks for [EL2](#) and [EL1](#), sized as `NUM_CORES * STACK_SIZE` for each level. Defined at the kernel elf file, but not stored in the binary.

After *.stacks*, is where the heap section starts. Everything from `__heap_start` onwards is considered available memory for the kernel's allocators. The heap end is set to `0xFFFF_FFFF_FFFF_FFFF`, meaning the allocators manage the entire remaining virtual address space up to the end of the high address space.

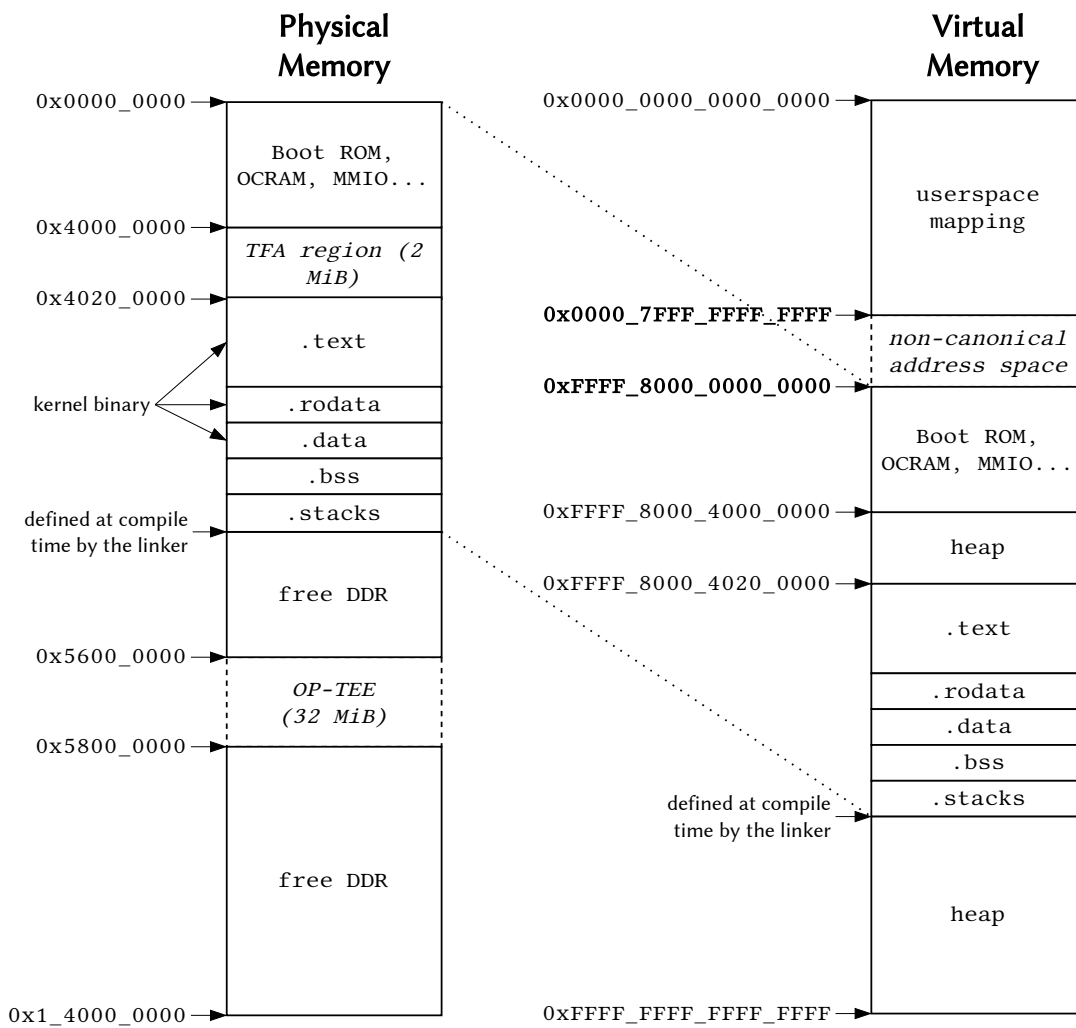
Figure 3.3 shows the physical and virtual memory layout of the kernel's binary after being loaded by U-Boot in physical and virtual space (after relocation). As seen in this Figure, the virtual layout matches the physical layout with the `KERNEL_BASE (0xFFFF_8000_0000_0000)` mask applied, except for the heap region, which ends at the maximum valid virtual address. This decision differs from other kernels like Linux and the reasons behind it will be explained in Section 3.7.7.

3.7.2 Early Allocator

The early allocator (`early_kalloc`) is designed to solve three problems that appear at early initialization:

- **Circular dependency**: initializing the main allocators requires dynamic allocations for their internal data structures, but no allocator exists yet. The early allocator allows this to happen.
- **Physical memory layout**: the kernel needs to know the physical map layout. Knowing which regions are free, reserved, permanent or [Device Memory](#), is required to initialize the main physical allocator. The early allocator builds this map from the linker and `MEM_REGIONS`.
- **Hardware-agnostic abstraction**: the memory layout is defined with a simple region descriptor interface (`MEM_REGIONS`), which separates the hardware-specific memory map from the allocator logic. In the current implementation this is hardcoded for the i.MX 8M Plus, but allows the configuration for devices with different memory layouts with minimal changes.

Figure 3.3: Kernel memory layout



Data structure

The early allocator maintains a linked list of `ek_node` structs (*early kalloc node*), each one describing a contiguous physical memory regions with the following metadata: start address, page count, a free flag, a debug tag, a permanent flag, and whether the region is [Device Memory](#). Deallocations are later handled by the standard allocators, not by the early allocator itself.

Nodes are stored backwards from the end of free [DDR](#) memory, while allocations are done from the start of free [DDR](#) regions. This ensures that the node array and the allocated regions do not collide, even without a separate allocator for the nodes themselves.

Early allocator initialization

The initialization of the allocator requires a pre-defined `MEM_REGIONS` pointer. It consists of a simple array that defines the [MMIO](#), [DDR](#) and `RESERVED` regions, which are then translated into `ek_node` nodes. The initialization of the allocator defines the physical memory layout and is done in three steps:

1. **Memory region mapping:** only the [DDR](#) and [MMIO](#) regions are pushed to the list, representing the available physical address space.
2. **Reserved region marking:** `RESERVED` regions like the [OP-TEE](#) region are removed out from the available regions by splitting the affected nodes and marking them as non-free and permanent. `RESERVED` regions define the not allowed physical regions for the kernel, and can overlap other regions when defined at `MEM_REGIONS`.
3. **Kernel binary reservation:** each kernel binary section (`.text`, `.rodata`, `.data`, `.bss` and `.stacks`) is reserved to prevent the allocator from reusing memory already used by the kernel. The specific positions of the regions are provided by the linker script. They are marked as non-free and permanent.

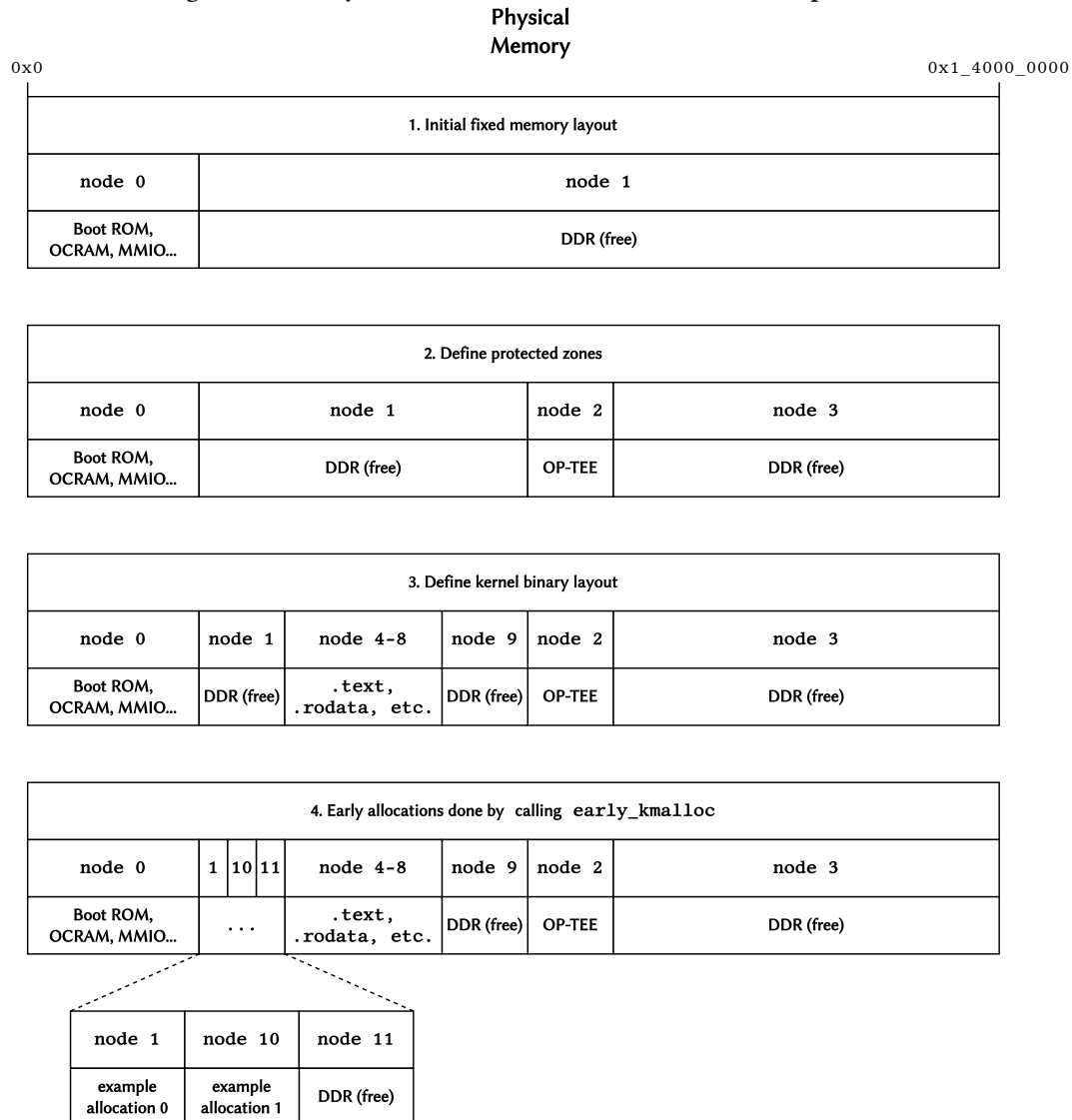
Dynamic early allocations

After the initialization of the allocator, the memory map is known to the allocator. This means that it can now allocate memory from the free regions for the other subsystems that require dynamic allocations at early initialization stage. Figure 3.4 shows the steps involved in the initialization and allocation process and the internal state at each step.

Early allocator end

When the early initialization stage is finishing, `early_kalloc_get_memregs` is called, which transforms all the internal data structures into a contiguous array of memory region descriptors. This array is provided to the physical and virtual memory allocators, in order to initialize their

Figure 3.4: Early allocator initialization and allocation process



own state. After this point, the early allocator nodes are freed (discarded), and any call to `early_kalloc` will panic.

3.7.3 Memory Management MMU submodule

The `mm_mmu` submodule is a private module for the `mm` subsystem. It is in charge of managing and providing variables for the [Memory Management Unit \(MMU\)](#) module described in [Section 3.5.1](#). It manages the MMU module allocators and provides the `mmu_core_handle` variable of each core, and standard kernel mappings pointers.

Kernel MMU configuration

This small submodule is where the MMU configuration for the kernel is defined. The kernel uses 48-bit addresses and 4 KiB granularity for both address spaces, with two global mappings:

- **MM_MMU_KERNEL_MAPPING**: the high address space mapping (TTBR1_EL1) used for all of kernel's virtual memory. All kernel allocations, driver [MMIO](#) mappings and kernel stack are mapped here.
- **MM_MMU_UNMAPPED_LO**: a low address space mapping (TTBR0_EL1) that is always empty. It is used as the default low mapping before a user process is assigned to a core. Its allocator is deliberately configured to panic on any allocation attempt, as the low mapping should never require new page tables while in this state, any such attempt would indicate a bug in the kernel.

The standard MMU allocator (`MM_STD_MMU_ALLOCATOR`) used for kernel mapping page table allocation, uses the `reserve_malloc` allocator described in [Section 3.7.8](#).

Each core has its own `mmu_core_handle`, stored in a cache-line aligned struct to avoid false sharing between cores. The handle holds the active low and high mappings for that core and is used when switching between user processes, as only the low mapping needs to be updated on a [context switch](#). [Listing 3.14](#) shows the main functions exposed in the `mm_mmu.h` header related to core MMU operations.

Listing 3.14: `mm_mmu` main functions and variables

```
/* --- PUBLIC (only for the mm subsystem) --- */

mmu_mapping* const    MM_MMU_KERNEL_MAPPING;    // Kernel high mapping
mmu_mapping* const    MM_MMU_UNMAPPED_LO;      // Unmapped low mapping
const mmu_allocator    MM_STD_MMU_ALLOCATOR;    // Standard MMU allocator
const mmu_allocator_free MM_STD_MMU_ALLOCATOR_FREE; // Standard MMU deallocator

// sets up a already configured for the kernel mapping
mmu_mapping    mm_mmu_mapping_new(mmu_tbl_rng rng);
```

```
// provides the pointer to the mmu_core_handle of the given coreid
mmu_core_handle* mm_mmu_core_handler_get(uint32_t coreid);
// provides the pointer to the mmu_core_handle of the current core
mmu_core_handle* mm_mmu_core_handler_get_self();
```

3.7.4 Identity Mapping

When the MMU uses [identity mapping](#), virtual addresses map directly to the same physical addresses in a one-to-one relationship. For example, the virtual address 0x1000 maps to the physical address 0x1000. [Identity mapping](#) is important during early initialization because it allows enabling the MMU, and with it, the caches and memory protections, without changing the data where the processor is currently executing at.

Enabling the MMU also disables the alignment checks that are enforced when the MMU is off. This is relevant because C compiled code, even at `-O0`, can generate unaligned memory accesses. The compiler may use [SIMD](#) instructions for some operations when it determines they are more efficient than scalar ones, even if the memory access is not naturally aligned. Packed structs and byte sized accesses can also trigger [alignment faults](#). For this reason, identity mapping is set up as early as possible in the initialization stages, just after the early allocator is initialized.

The `early_identity_mapping` function maps the physical address space to both the low address space (identity mapping) and the high address space. High addresses are obtained by applying `KERNEL_BASE` as a bitwise OR to the physical address, setting the sign bit and sign-extension bits. This gives the kernel virtual address corresponding to the physical address:

$$kva = pa \mid \text{KERNEL_BASE}$$

For example, the physical address 0x0000_0000_0000_1000 maps to the kernel virtual address 0xFFFF_8000_0000_1000. This relationship between physical and kernel virtual addresses is called [direct-map](#) and is used throughout the kernel for all the memory regions except for the heap, and is described in more detail in [Section 3.7.7](#).

Configuring the high address space mapping at this stage is very useful. Before this point, all pointers to global variables must be converted to their physical address using the `as_kpa` (as kernel physical address) macro before dereferencing, because the linker script places all the symbols at high virtual addresses. Once the high address space is mapped, kernel virtual addresses become directly accessible and `as_kpa` is no longer needed for global variable access through compile-time defined pointers.

Setting Up Identity Mapping

Identity mapping is set up using the MMU module (Section 3.5.1) and the `mm_mmu` submodule (Section 3.7.3). The process iterates over `MEM_REGIONS` in two passes:

1. **First pass, region mapping:** `DDR` and `MMIO` regions are mapped to both the low and high address space mappings. `DDR` regions are mapped as `Normal Memory` with read, write and execute permissions. `MMIO` regions are mapped as `Device Memory` (`Device-nGnRnE`) with read and write permissions and execute-never set.
2. **Second pass, reserved region unmapping:** `RESERVED` regions are unmapped from both address spaces, since reserved regions can overlap `DDR` or `MMIO` regions defined in the first pass.

Finally, both configured MMU mappings are applied to the `mmu_core_handle` struct and the MMU is enabled with the MMU module functions, that internally set the `SCTLR_EL1[M]` bit. Listing 3.15 shows how this process is done. Enabling the MMU is safe at this point because the all the memory sections stay in place. After the MMU enabling instruction, when the processor fetches the next instruction (`pc + 4` bytes), the fetched instruction is the same as when executing with the MMU disabled due to the identity mapping. Invalidating the data and instruction caches is not required in this case because it was done just before transitioning to EL1, at the kernel boot stage.

Listing 3.16 shows the page configurations used for each memory type. Figure 3.5 shows an example of the resulting MMU mapping state after both passes. The region between `0x0` and `0x4000_0000`, that contains the Boot ROM, `OCRAM`, `MMIO` and many others that are not `DDR`, have been simplified in the Figure as one contiguous `DEVICE` region, but in the actual implementation, only the actual `MMIO` region is defined, and the others are marked as reserved (unmapped).

Listing 3.15: `early_identity_mapping` MMU core configuration.

```
extern mmu_mapping* const MM_MMU_KERNEL_MAPPING;
extern mmu_mapping* const MM_MMU_IDENTITY_LO_MAPPING;

safe_early void early_identity_mapping() {
    mmu_core_handle* core0_handle = mm_mmu_core_handler_get_self();

    // Initialize the low address space (identity) mapping.
    // physmap_offset is 0 because the kernel is not reallocated yet.
    *as_kpa(MM_MMU_IDENTITY_LO_MAPPING) = mmu_mapping_new(
        MMU_LO,                // low address space
        MMU_GRANULARITY_4KB,
        48,                    // 48-bit virtual address space
        0x0,                   // physmap_offset = 0 (pre-relocation)
        (void*)as_kpa((uintptr_t)im_alloc), // backed by early_kalloc
```

```

        (void*)as_kpa((uintptr_t)im_free)); // panics on free (not supported at this early stage)

// Initialize the high address space (kernel) mapping.
*as_kpa(MM_MMU_KERNEL_MAPPING) = mmu_mapping_new(
    MMU_HI, // high address space
    ... // same as MM_MMU_IDENTITY_LO_MAPPING
);

... // map MEM_REGIONS into both address spaces

bool result = mmu_core_handle_new(
    core0_handle,
    as_kpa(MM_MMU_IDENTITY_LO_MAPPING), // TTBR0_EL1
    as_kpa(MM_MMU_KERNEL_MAPPING), // TTBR1_EL1
    true, // enable high mapping
    true, // enable low mapping
    true, // enable d-cache
    true, // enable i-cache
    false // disable alignment trap
);
ASSERT(result);

// Activate the MMU configuration of the core0_handle variable at the current core.
mmu_activate_result cres = mmu_core_activate(core0_handle);
ASSERT(cres == MMU_ACTIVATE_OK);
}

```

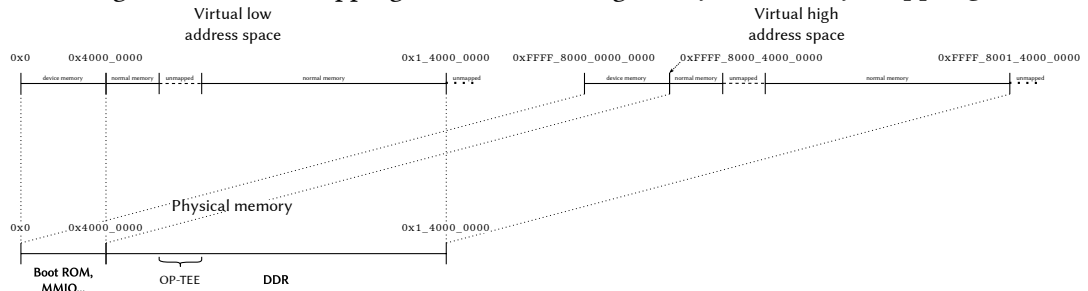
Listing 3.16: Page configurations used for identity mapping of MMIO and DDR regions.

```

// Page configuration for MMIO (Device memory) regions.
// Device-nGnRnE: no gathering, no reordering, no early write acknowledgement.
// Execute-never for both \gls{EL0} and EL1, because peripheral memory should never
// be executed.
const mmu_pg_cfg DEVICE_CFG = {
    .attr_index      = 1, // 1 - Device-nGnRnE in MAIR_EL1
    .access_permissions = MMU_AP_EL0_NONE_EL1_RW, // EL1 read/write, \gls{EL0} no access
    .shareability     = MMU_SH_NON_SHAREABLE, // non-shareable
    .non_secure       = true, // a kernel is non-secure world
    .access_flag      = true, // disable access flag faults
    .pxn              = true, // EL1 execute-never
    .uxn              = true, // \gls{EL0} execute-never
    .sw               = 0 // software defined, not used
};

// Page configuration for normal DDR memory regions.
// Inner-shareable to ensure cache coherency across cores.
// EL1 execute allowed (kernel code), \gls{EL0} execute-never.
const mmu_pg_cfg MEM_CFG = {
    .attr_index      = 0, // 0 - normal memory in MAIR_EL1
    .access_permissions = MMU_AP_EL0_NONE_EL1_RW, // EL1 read/write, \gls{EL0} no access
    .shareability     = MMU_SH_INNER_SHAREABLE, // inner-shareable coherency
    .non_secure       = true,
    .access_flag      = true,

```


Figure 3.5: MMU mappings state after calling `early_identity_mapping`

```

.pxn          = false,           // EL1 execute allowed
.uxn          = true,            // \gls{EL0} execute-never
.sw           = 0
};

bool result = mmu_core_handle_new(
    core0_handle,
    &identity_lo_mapping,
    as_kpa(MM_MMU_KERNEL_MAPPING),
    true,
    true,
    true,
    true,
    false);
ASSERT(result);

mmu_activate_result cres = mmu_core_activate(core0_handle);

```

3.7.5 Kernel Relocation

The kernel relocation happens after initializing the physical and virtual allocators that will be explained in the following sections. It allows the kernel to locate itself in the high virtual memory address space, allowing for easy userspace process MMU mapping switchings and providing greater isolation.

Before relocating, the MMU is in the [identity mapping](#) state, explained in the previous [Section 3.7.4](#). As the identity mapping function left the high virtual space [mapped in a direct](#) relationship to the physical memory, just by applying the `KERNEL_BASE` mask, high address space is accessible.

The kernel relocation is performed in the following steps:

1. Jump to a known virtual high address space. In the case of the project, a branch instruction is performed to the `_reloc_cfg_end` function, defined in Assembly. This jump can not be done by just calling the function from C, as it would be compiled in

a branch with link instruction relative to the program counter, and not as an absolute address jump.

2. Unmap the low address space mapping. Before this point the program counter is executing at high address space, but the low address space is still mapped. As low address space is used for user processes, it is unmapped.
3. Unmap unused regions. The early allocator's reserved memory regions are read, and any non-reserved region is unmapped from high virtual address space. This allows detecting bugs in the code and prevents the kernel from writing unused physical memory.
4. Allocate and set the main stack. Until this point, the stack being used was running at low address space, and using a small region pre-reserved at link-time, thought for initialization. As the allocators (that will be described in later sections) are now functional, the kernel can allocate a larger region for its own stack which lives in high address space. Allocating a new stack region means that all the previous data will no longer be used.
5. Relocate `VBAR_EL1`. The `sysreg` `VBAR_EL1` holds the address of the exception vector. Previously the physical/identity address of the `Vector Table` was set, but as now the low address space has been unmapped, that address is no longer accessible. The new virtual address of the vector set is set as `prev_table_address | KERNEL_BASE`.
6. Enter `kernel_entry`. A jump to the kernel's entry function from high virtual space is performed, with a new stack, everything running at high address space, the MMU, and virtual and physical allocators configured, the kernel's early initialization stage is now completed.

3.7.6 Physical Memory Management

Virtual memory is an abundant resource to the kernel, using the common 48-bit configuration for virtual space, leaves 256 TiB available for the kernel to map as it requires. Physical memory, however, is a scarcer resource and the limiting factor that determines the maximum amount of data the kernel can access with low latency.

In order to manage dynamic kernel allocations, the kernel must implement a system capable of tracking used and unused physical memory, and of quickly providing unused pages to the different subsystems that request them. The allocator that serves this purpose is the *page allocator*.

The Page Allocator

The page allocator, is the core of other allocators, making it one of the areas for the kernel where performance has the greatest impact. While the implementation has been fully developed

from scratch, the design choices and algorithms used have been heavily influenced by the Linux page allocator. Like Linux, the design is based on the *buddy allocator*.

A *buddy allocator* is a data-structure that allows for the allocation and deallocation of power of two N sized regions. It works by splitting the available memory area (physical memory in this case) in blocks known as *buddies*, which range in size from order 0 to a maximum order defined by the implementation. The page allocator always attempts to maintain the largest possible order for all free buddies, minimizing [fragmentation](#) costs low.

The main difference from the Linux implementation is that while Linux only uses the page allocator for the [DDR](#) memory, while this implementation of the page allocator represents all the physical memory. This design choice was taken to avoid the need of other modules that keep the used and unused memory like [MMIO](#) regions, but in future versions of the kernel, this might be implemented and the design of the page allocator changed to only represent [DDR](#) memory.

Data Structures

The allocator maintains two primary data structures. The first is a *page frame array*, a flat array of `page_node` structs, one per physical page, indexed by a [Page Frame Number \(PFN\)](#). Each node stores the block's current order, a free flag, and a `mm_page_data` field that holds metadata such as a tag, whether the region is permanent, and whether it maps [Device Memory](#). Some fields used are not strictly necessary for the kernel implementation but are useful for debugging purposes. As a node's index is always multiple of its order's power of 2, indexing operations are very performant, due to bit-wise operations being very useful due to those conditions.

The second structure is an array of $\text{MAX_ORDER} + 1$ free lists, one per order. Each list is implemented as a double linked list. The links of the list are not pointers, but [PFNs](#) (indexes of the array), that point to the next or previous free buddy of the same order. The linked list takes advantage of the *page frame array* to store the linked list directly in the actual free slots, avoiding the need for node allocations.

Allocation

When a block of order o is requested, the allocator first looks up into the free list of order o . If a block is available, it is removed from the list and returned immediately. Otherwise the allocator scans up through the orders $i \in [o + 1, \text{MAX_ORDER}]$ until it finds the smallest free block of order $> o$. That block is then split until order o .

In order to split a buddy, it is removed from its list, and at each step from order k down to $o + 1$, the right buddy is pushed onto the corresponding free list. The left block is kept

and its order set to o . The result is that one block of the requested order is returned, and all intermediate buddies are saved as available for future allocations.

Figure 3.6 shows a simple example of how the state of the allocator would change with an Order 4 buddy when different allocations deallocations are applied.

Deallocation and Coalescing

Freeing a block pushes it back onto its order's free list and sets up a *merge* pass. The allocator calculates the buddy of the freed block using an XOR on the PFN. Given a block at index i of order o , its buddy is at index $i \oplus 2^o$. If the buddy is also free and at the same order, both blocks are removed from their list, the left one is set to order $o + 1$, and the process repeats. This continues until either the buddy is not free, has a different order, or the maximum order is reached. In this way the allocator quickly rebuilds large contiguous blocks as soon as they become available, keeping fragmentation low.

Initialization and Reservation

During the early initialization stage, the allocator gets the total page count from the memory map and sets `MAX_ORDER` as $\lfloor \log_2 N \rfloor$. The *page frame array* is allocated using the early allocator and all page nodes are zeroed and all free-list heads set to `NULL_IDX`. Physical memory is then inserted into the free lists by repeatedly extracting the largest power-of-two block that fits in the remaining range, ensuring coverage even when the total page count is not itself a power of two.

In a second pass, `page_allocator_update_memregs` function walks the early memory region descriptors produced by the `early_allocator` and reserves every region that must be reserved by the allocator (kernel image, device trees, firmware-reserved areas, early allocations, and so on). This sets up the page allocator as if it had been the one that performed the allocations and reservations made by the `early_allocator`. Reservation works by finding the free block that currently covers the target range, splitting it down to the exact order needed, and marking the resulting block as allocated with the required metadata. Figure 3.7 shows how the page allocator is initialized in a simplified way.

Regions tagged as *permanent* are protected at runtime and any attempt to free them triggers a kernel panic.

Advantages and Disadvantages

Using a buddy-based allocator for the page allocator, has some very relevant advantages, mainly related to performance, however, it also has some disadvantages that should be commented:

Figure 3.6: Order 4 page allocator example

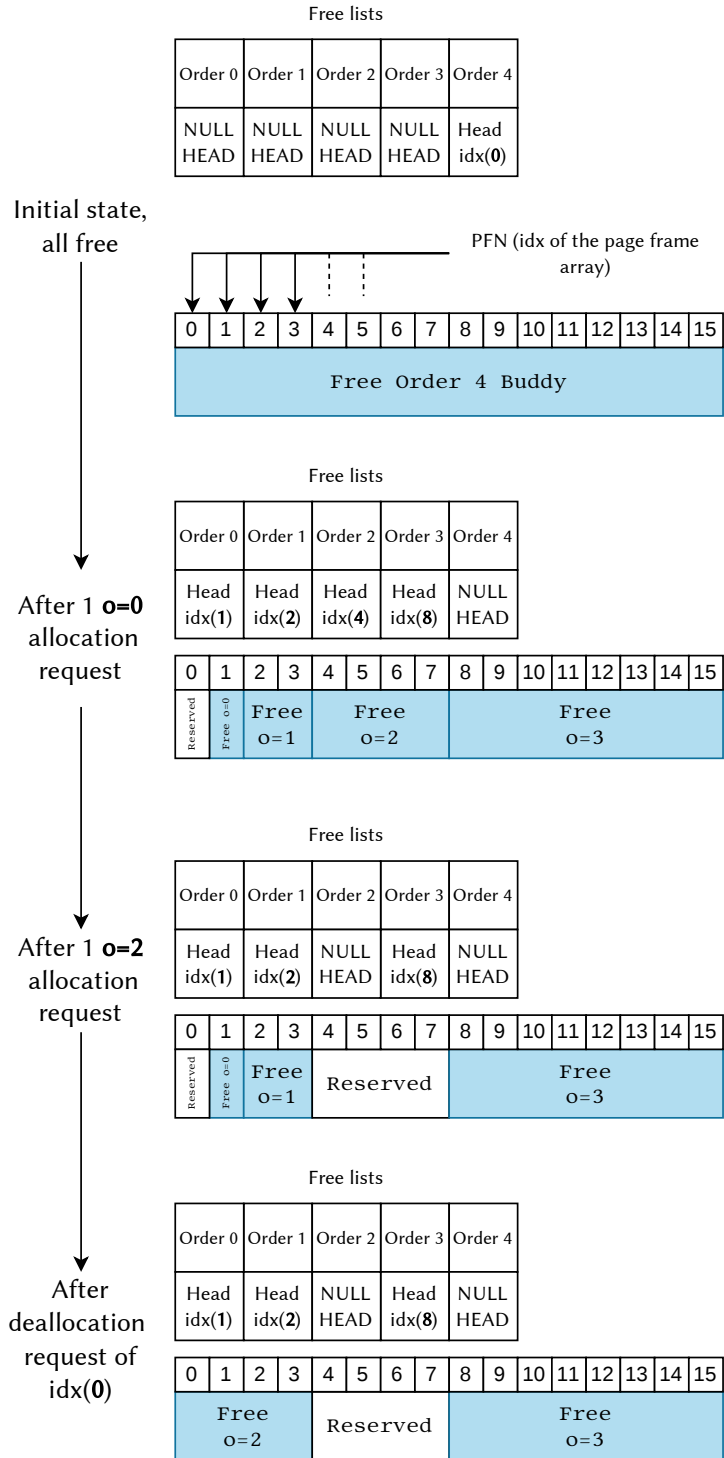


Figure 3.7: Page Allocator initialization state

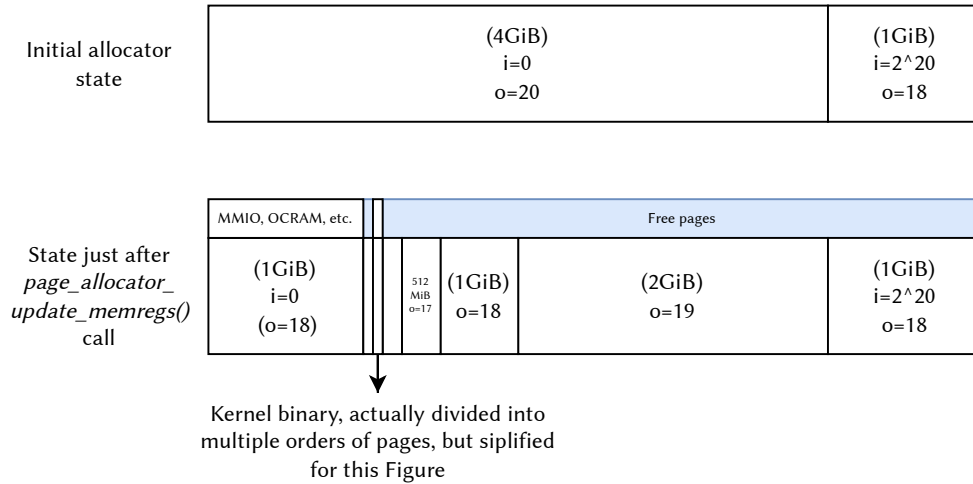


Table 3.4: Advantages and disadvantages of the buddy page allocator

Advantages	Disadvantages
Low external fragmentation: blocks are reconstructed as soon as they are freed if possible, minimizing external fragmentation. Alignment to the size: any allocation is ensured to be aligned to its size. For example, a 2-page allocation (order 1) is guaranteed to be aligned to 8 KiB. $O(\log n)$ allocation and deallocation: at most MAX_ORDER levels are traversed in either operation, however, a block of the requested order is often directly available or close to the order, making the average case close to $O(1)$. No auxiliary allocations: the free list nodes are embedded directly in the page frame array, avoiding any extra memory allocations for the list nodes. The lists head node links do not live in the page frame array, but the size is insignificant due to being an array of MAX_ORDER 32-bit integers. Efficient bitwise operations: buddy identification using XOR and power-of-two indices keep the array path very cheap.	External fragmentation: Even though the external fragmentation is low, it is not fully prevented. Previous allocations may leave small-order buddies scattered across the address space, preventing their parent blocks from being coalesced into larger contiguous regions even when enough free memory exists in aggregate. Fixed granularity: only power-of-two block sizes are supported. Arbitrary sizes require a higher-level allocator. Merge overhead: frequent free operations trigger repeated merge passes even when blocks will soon be reallocated. Fixed memory overhead: the page frame array requires one page_node per physical page regardless of use, consuming 32–64 bytes per page (e.g. 32–64 MiB for 4 GiB of RAM (~0.7%–~1.4% of all the system’s memory)). Linux uses 64-byte entries, however this implementation uses 32-byte ones given its simpler scope.

3.7.7 Virtual Memory Management

Virtual memory is the memory created by the MMU. Correct use of this memory is crucial for a kernel's operations and allows high level allocators to link physical with virtual memory dynamically. In order to implement dynamic allocations, the kernel must implement a system capable of tracking reserved and free virtual memory for virtual allocations and deallocations.

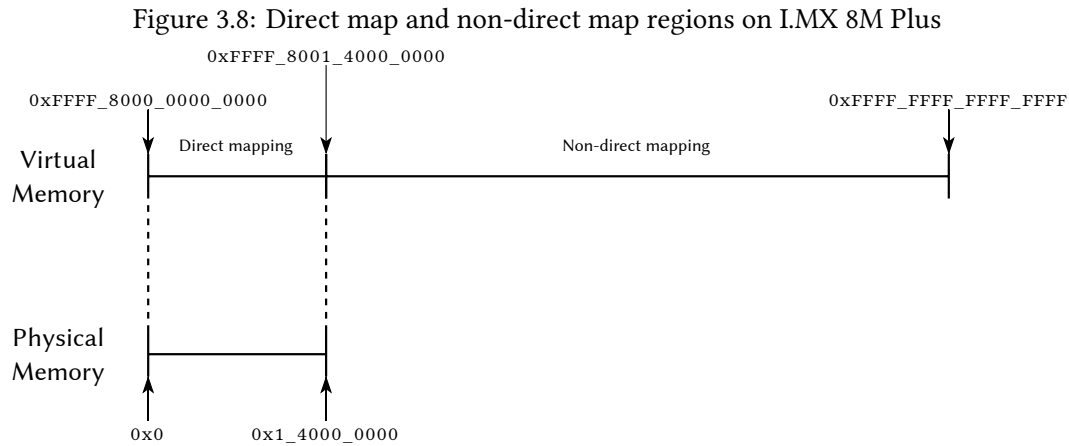
Kernel's Virtual Memory Regions

Not all the kernel's virtual memory regions are used for the same purposes. As seen in previous sections, the kernel's binary lives in high address space. Most of the seen memory sections until this point on the report are mapped with a direct relationship to the physical addresses, where applying the `KERNEL_BASE` (`0xFFFF_8000_0000_0000`) bitwise-or mask results in the used virtual addresses.

This virtual memory region is known as **direct-map**, and allows obtaining the physical address it points to easily. It is very useful for virtual memory sections that require the knowledge of where their physical memory lives. For example, the MMU page tables are always direct mapped, because knowing their physical memory is a requirement for specifying the `physmap_offset`. In this case, just setting the `physmap_offset` as the `KERNEL_BASE` value will result in a fast virtual-to-physical conversions that does not require any data structure who stores what virtual pages point to what physical pages. The kernel binary sections and **MMIO** regions are direct mapped, as well. Although they do not require the knowledge of their physical pages as much as the MMU page tables, it is useful for pointer conversions at early initialization stages and debugging purposes.

This region, however, is unfit for dynamic virtual contiguous range allocations. As seen in Section 3.7.6, the page allocator provides physical pages in contiguous blocks whose size is always a power of two. This makes creating virtual contiguous ranges that do not fit a power of two waste resources, as the direct mapped region relates directly to the physical memory. If for example a contiguous virtual direct mapped region of 17 pages were to be created, an allocation of 32 contiguous physical pages would be required, leaving 15 physical pages unused.

The remaining high address space that does not directly overlap with the physical memory when applying the `KERNEL_BASE` mask is not unused. It serves the purpose of dynamic allocations, where knowing the physical memory the regions points to is not required by the systems requesting them. This region avoids the limitations imposed by the direct mapped region and allows creating contiguous virtual ranges that map to different physical pages that don't have to be contiguous. Using the same example as before, to create a contiguous virtual range of 17 pages, mapping the 17 pages to a block of 16 physical pages (2^4) and a block of one



page (2^0) results in no internal [fragmentation](#), and no wasted resources. This works with any requested range, as any number can be divided into powers of two. For example 45 would be divided into $32 + 8 + 4 + 1$, and so on with any number. This is a concept that will be further explained in Section 3.7.9.

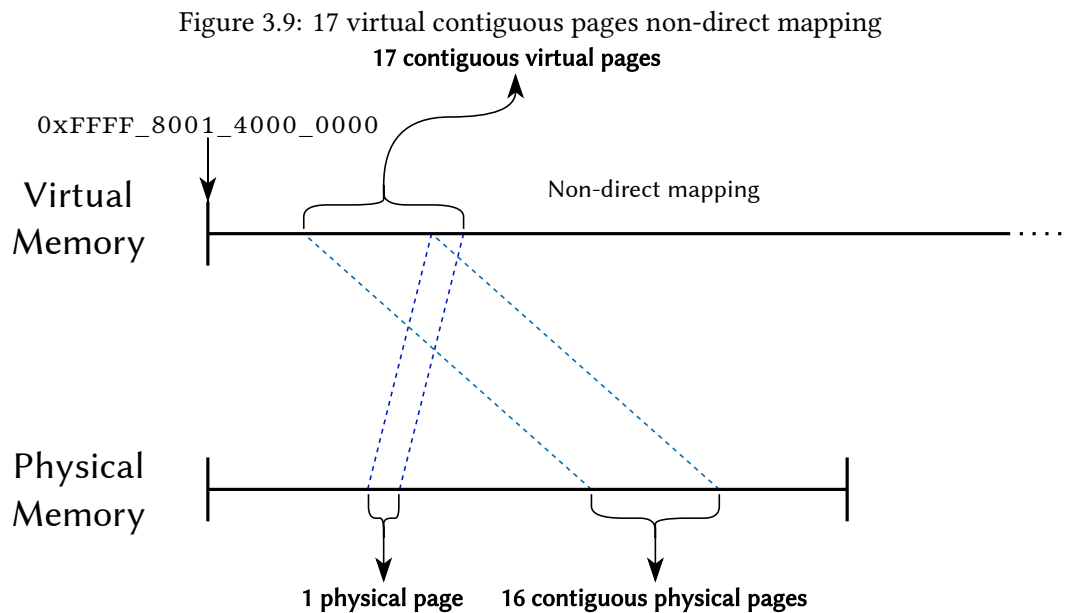
Figure 3.8 shows the ranges direct mapped and non-direct mapped on an IMX 8M Plus. Figure 3.9 shows an example of how a 17 contiguous virtual page range would be built from non-contiguous physical ranges. As it can be seen, the location of the physical pages used to build the virtual region are not required to be contiguous nor related to the virtual addresses at all.

Virtual Memory Allocator

The *virtual memory allocator* used in this project has been named `vma11oc`. It is in charge of tracking all the available 128 TiB of high memory space used by the kernel. It serves two main purposes: tracking reserved and free kernel virtual memory and saving useful metadata of each allocation area.

The implementation of this allocator uses binary search trees to keep track of used and unused memory. The `vma11oc` implementation has not been inspired by the Linux implementation, and has been developed from scratch. It uses the red-black tree module that will be explained in Section 3.6. A red-black tree is a self-balancing binary tree that ensures maximum $O(\log n)$ search, insertion and removals.

The `vma11oc` implementation allows for allocations and deallocations of [direct-mapped](#) and non-direct mapped virtual ranges, as the higher level allocators require them to be taken into account as explained before. `Vma11oc` does not actually map any virtual memory nor interact with the hardware, it only keeps track of what virtual memory ranges have been used and mapped, and what regions are unused and unmapped. In order to apply the actual virtual



mapping to the hardware, it must be used alongside the MMU module, previously explained in Section 3.5.1. Using only the MMU module for tracking purposes would require a manual walk over the page tables when wanting to allocate and deallocate any region, which would result in inefficient operations, and storing metadata would be extremely limited.

A question that may arise from the use of the red-black tree, is how the nodes of the tree are allocated. Vmalloc cannot directly allocate its own nodes, as it creates a circular dependency. In order to break this dependency, the reserve allocator is used, which will be later explained in Section 3.7.8.

Another question would be how the nodes are allocated without using a full physical page for each node. Until this point in the kernel, no mechanism for smaller allocations has been developed. Vmalloc nodes have a size of 48 or 80 bytes, depending on if they represent a reserved or free region and using one page for each node would result in a significant waste of physical memory. To avoid this case, the pages requested to the reserve allocator are managed by an internal module that divides them into node-sized and aligned portions.

Vmalloc containers is the internal module used by vmalloc to manage the tree node allocations. It is a simplified cache allocator built on top of the reserve allocator. A cache allocator works by using a doubly linked list of large arenas/caches, which contain smaller same-sized entries ready to be provided as allocations. An arena is aligned to its own size. To keep track of what entries are free or reserved, a bitmap is stored inside the arena. A pointer to an arena that has at least a free entry is always maintained to provide fast allocations. For

deallocations, finding the address of the arena is very easy, just by aligning down the pointer of the allocation to the arena alignment. This submodule uses page sized arenas, as the reserve allocator can only provide individual pages.

The “vmalloc containers” submodule implements two cache allocators, one for reserved and one for free vmalloc nodes. How a cache allocator works will be explained with more detail in Section 3.7.10.

The vmalloc implementation works by using binary search trees, as introduced before. It maintains four trees: a pair for reserved virtual memory regions, and a pair for free regions. Each pair consists of one tree for **direct-mapped** regions and another for non direct-mapped regions.

When a **direct-mapped allocation** is requested, the caller provides the physical address to be reserved via the `kmap_pa` field of the `vmalloc_cfg` structure. Vmalloc then looks up the free **direct-mapped** tree for the region containing that address, removes it, splits it if necessary, inserts the remainder back into the free tree, and finally registers the reserved region in the reserved tree. For a **non direct-mapped allocation**, no physical address is provided. Instead, vmalloc searches the free non direct-mapped tree for any region large enough to hold the request, performing the same procedure. In both cases, the allocator stores metadata alongside each reserved region, including a tag for identification and debugging purposes, mapping flags, and a list of the physical allocations backing the virtual range, which allows higher-level modules to later query the physical layout of any given allocation via `vmalloc_get_pa_info`.

To avoid tree walks when freeing or querying a region, vmalloc supports an optional `vmalloc_token`. It is a lightweight handle that holds a direct reference to the internal tree node of an allocation. This avoids the $O(\log n)$ search and provides $O(1)$ access for deallocation and metadata retrieval.

3.7.8 Reserve Allocator

The reserve allocator is used by the systems that might break the allocation process by introducing a circular dependency. For example, for allocating an MMU page table, the page allocator must be used to reserve a free page, then vmalloc for registering the **direct-map** region it overlaps, and finally map with the MMU the page itself to be able to write in it. In this example, there are two points where a circular dependency might or might not appear, depending on the state of the used modules:

1. When using vmalloc, a page for the internal nodes via the “vmalloc container” submodule could be required if all the other arenas are full, causing a circular dependency.

2. Mapping a page might require another page to map that new page, and that other page another one and so on. As there are four levels of page tables (L0–L3), up to four pages could be required. Usually the maximum required pages is one because the used L0–L1 page tables have already been allocated by the early allocator, but the L2–L3 tables might be required.

To break this dependency, the reserve allocator maintains a fixed pool of 32 pre-allocated, [direct-mapped](#) pages that are available without calling any of the subsystems that could trigger a circular dependency. The pool is initialized at early initialization time using the early allocator, before the rest of the memory subsystem is active. When a subsystem requires a page where circular dependency could occur, the pages are drawn from this pool. Once the operation that could cause the dependency has completed, the caller is responsible for the refilling of the pool via `reserve_malloc_fill`, which allocates new pages using the standard kernel page heap allocator and refills any empty slots. The standard allocator will be explained in Section 3.7.9.

The refill process itself may consume additional reserve pages if the allocations it performs, internally trigger dependencies. This is handled by the fill loop, which iterates until the pool is completely full instead of using a single pass. With a pool of 32 pages, the probability of emptying the reserve during refilling is considered negligible in practice, as the maximum nesting depth of dependent allocations is bounded and below that capacity.

An alternative design would have been to use the page allocator directly, treat each page as [direct-mapped](#), and defer the `vmalloc` and MMU registration until after the dependent operation is completed. This approach was discarded due to the complexity of correctly deferring and updating all the involved steps, making the reserve pool a simpler more fit solution considering the scope of this project.

3.7.9 Raw Kernel Memory Allocator

Until this point on the report, all the explained allocators are not by themselves capable of managing what would be considered an standard page allocation. They register the state and provide free region pointers, but do not actually interact with the hardware to actually provide usable memory. If for example `vmalloc` was to be used alone, when trying to access the received pointer, a [translation fault](#) is assured to happen, as `vmalloc` is not capable nor responsible of updating the MMU to actually map the region. The higher level allocator that merges these allocators and provides usable contiguous virtual mapped page ranges is the `raw_kmalloc` allocator. It provides contiguous virtual pages that have been mapped to other physical pages, and can be configured to use [direct-map](#) and non direct-map regions.

The implementation follows a different path depending on whether a direct-mapped or non direct-mapped allocation is requested.

Direct-mapped allocation requires the requested size to be a power of two of the page size, as the `direct-mapped` region relates to physical memory and the page allocator only provides power-of-two blocks. As seen in Listing 3.17, the page allocator is called to obtain a contiguous physical range of the requested size. The resulting physical address is passed to `vmalloc` with the direct-mapping configuration, which registers the corresponding virtual region. Finally, the MMU module maps the virtual range to the physical range.

Listing 3.17: `raw_kmalloc` direct-mapped allocation core logic

```
ASSERT(is_pow2(pages), "only pow2 n pages can be kmapped"); /* panics if the condition is not met */
puintptr_t pa = page_malloc(log2_floor(pages), ...);
vuintptr_t va = vmalloc(pages, tag, kmap_cfg(pa), NULL);
mmu_map(MM_MMU_KERNEL_MAPPING, va, pa, pages * PAGE_SIZE, mmu_cfg);
```

Non direct-mapped allocation does not have a power-of-two restriction on the total size. `vmalloc` is first called to reserve a contiguous virtual region of the requested size in the non `direct-mapped` space. Since this virtual range does not need to correlate to a contiguous physical range, it is decomposed into power-of-two sub-ranges multiple of the page size. As seen on Listing 3.18, each sub-range is requested as an independent physical allocation from the page allocator, and the MMU module maps it individually into the reserved virtual region. The metadata of each physical allocation is stored into the `vmalloc` node using `vmalloc_push_pa`, simplifying later deallocation without additional MMU walks.

Listing 3.18: `raw_kmalloc` non-direct mapped path summarized

```
vuintptr_t start = vmalloc(pages, tag, vmalloc_cfg_from_raw_kmalloc_cfg(cfg, 0), &vtoken);
vuintptr_t va = start;
size_t rem = pages;

while (rem > 0) {
    size_t o = log2_floor(rem); /* 63 - __builtin_clzll(rem) */
    size_t subpage_count = power_of2(o); /* subrange page count (1UL << o) */
    size_t order_bytes = subpage_count * PAGE_SIZE;

    puintptr_t pa = page_malloc(o, ...); /* get physical pages */
    vmalloc_push_pa(vtoken, o, pa, va); /* store PA in vnode */
    mmu_map(MM_MMU_KERNEL_MAPPING, va, pa, order_bytes, *mmu_cfg, NULL); /* Map the pages */

    va += order_bytes;
    rem -= subpage_count;
}

return (void*)start;
```

Both paths then merge together once the allocation has completed, and `reserve_malloc_fill` is called unless explicitly disabled in the `raw_kmalloc_cfg` configuration struct. The caller can also request that the resulting allocation be zero-initialised, in that case, `memzero64` is

applied to the full allocated range after the lock has been released. `memzero64` is a function implemented in Assembly that takes advantage of `SIMD` instructions to zero a 16 byte aligned, multiple of 64 sized range.

When `raw_kfree` is called, if the provided pointer is inside the `direct mapped` or non-direct mapped range is checked. The path taken is different for each case, but the logic behind is similar. The corresponding virtual address and physical addresses are deallocated from `vmalloc` and the page allocator, and the entire range is unmapped at once.

Neither `vmalloc` nor the page allocator implement locks themselves, as they only track virtual and physical state respectively, and both states are directly related. A physical allocation always implies a corresponding virtual registration, and freeing one without the other would leave the system in an inconsistent state. The lock is placed at the `raw_kmalloc` level, where both operations are combined. The lock is a combined `IRQ` and CPU spinlock, making the allocator safe for use in `Symmetric Multiprocessing (SMP)` environments and from interrupt context.

Additionally, the lock is exposed publicly via `raw_kmalloc_lock` and `raw_kmalloc_unlock`, so that other low-level subsystems that need to call `vmalloc` or the page allocator directly can acquire it, maintaining global coherence of the memory state. This should only be used in special cases, and most page sized allocations for the kernel should use `raw_kmalloc` directly. Special cases like reserving physical pages for userspace require it, because `vmalloc` only tracks virtual allocations of the kernel space, and not user space, but the page allocator must still remain `SMP` safe.

3.7.10 Cache Allocator

Until this point on the report, all the explained allocators only support sizes that are multiples of the page size (4 KiB). However, most common kernel allocations require smaller sizes. An allocator that supports smaller sized allocations is required for this purpose.

The cache allocator has been introduced previously in Section 3.7.7, where the `vmalloc` containers submodule used a simplified cache allocator to store the different nodes required for the binary trees of `vmalloc`. The allocator used for the kernel's small allocations uses the same logic, but instead of obtaining pages from the reserve allocator for the arenas, uses `raw_kmalloc`.

Supported sizes The allocator supports nine fixed size classes, from 8 to 2048 bytes in powers of two: 8, 16, 32, 64, 128, 256, 512, 1024 and 2048 bytes. Each size class with its own list of arenas and its own state. Fixing the size classes avoids `internal fragmentation` within arenas. The internal fragmentation of the allocations themselves are relatively small as caches are defined with powers of two, so the maximum wasted space per allocation is just under

50% (e.g., a 33-byte allocation consumes a 64-byte arena).

Arena structure Each arena/cache is a contiguous region of pages obtained from `raw_kmalloc`, whose size depends on the entry size class. The arena layout consists of four parts: a buffer of fixed-size entries, a bitfield array tracking which entries are reserved, two pointers forming a doubly linked list of arenas for that size class and a 64-bit integer that indicates the cache size. All arenas of each size class are kept in linked lists, one for each size class. The allocator state tracks the first and last arenas, as well as a `first_free_cache` pointer pointing to the arena and position of the first known available slot. All the arenas are allocated as [direct-mapped](#), to ensure alignment to the size of the arena, which allows obtaining the pointer to the arena from any cache entry pointer, avoiding any per-allocation metadata.

Slot reservation Allocation consists of finding a free slot within an arena. The `find_empty_slot` function scans the bitfield array of an arena starting from a given position, checking each 64-bit bitfield word for any unset bit. Once a free bit is found, it is set and the corresponding entry pointer in the buffer is returned.

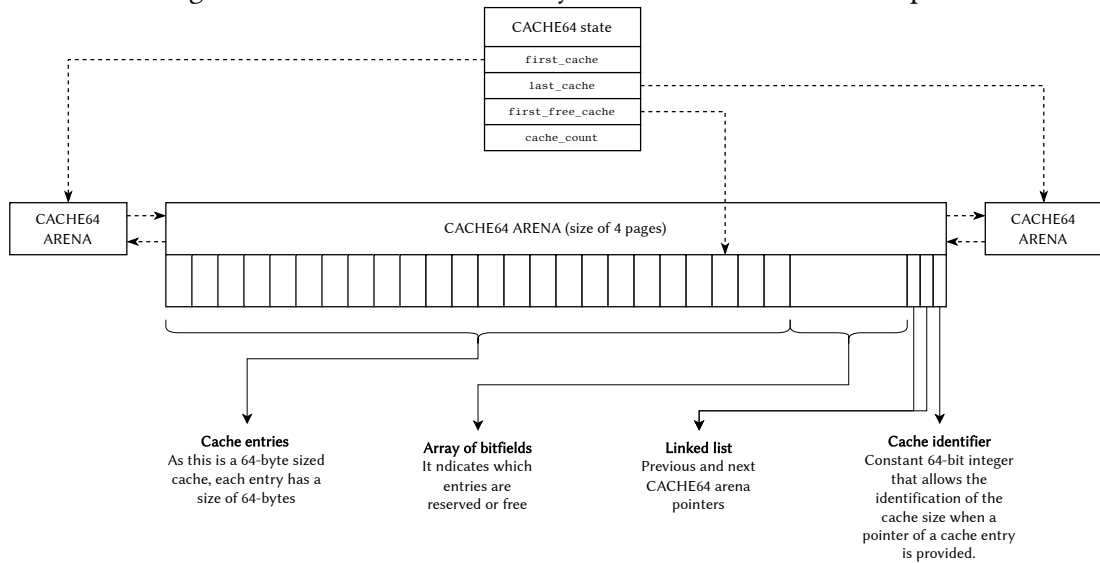
Arena allocation and `first_free_cache` The `first_free_cache` field in the allocator state stores a pointer to the arena and the bitfield position of the first known free slot, avoiding a full list traversal on every allocation. When an allocation is made, the pointer is updated to the next free slot in the same arena if one exists. If the arena becomes full, the list is traversed to find the next arena with a free slot. If no arena has a free slot, a new arena is allocated from `raw_kmalloc` and appended to the list.

Deallocation and arena release On `cache_free`, the entry's bitfield bit is cleared. If the arena still has other reserved entries, the `first_free_cache` pointer is updated if it had no valid target and the function returns. If after clearing the bit the arena is completely empty, it is unlinked from the doubly linked list and its pages are returned to `raw_kfree`.

Size class recovery from pointer Since arenas are allocated in the [direct-mapped](#) region, their alignment to the arena page size is guaranteed by the page allocator. This allows to recover the size class of any allocation without per-allocation metadata. The pointer is aligned down to each possible arena size, and the `cache_id` field is read from the fixed offset at the end of that region. The field encodes a magic value and the size class of the arena, allowing to validate the result and reject false positives.

Figure 3.10 shows the data structure of each arena.

Figure 3.10: Cache allocator 64-byte arena data structure example



3.7.11 Kernel Memory Allocator

The allocators described in the previous two sections target different allocation granularities. The cache allocator handles sub-page allocations in fixed size classes up to 2048 bytes, while `raw_kmalloc` uses only page-sized multiples. In practice, however, kernel subsystems require a unified interface that abstracts away this granularities, selecting the best allocator based on the requested size.

The kernel memory allocator is this layer. On allocation, it checks the requested size and dispatches either the cache allocator or `raw_kmalloc`. Requests that fall within a supported cache size class are passed to the cache allocator, while larger requests are passed directly to `raw_kmalloc`.

The free path requires no additional metadata to determine which allocator owns the pointer. Since the cache allocator uses the `direct-mapped` region and the `raw_kmalloc` allocations made through this interface use the non-direct-mapped region, a single range check against the direct-mapped region is enough to decide the owner.

As shown in Listing 3.19, the kernel memory allocator has been named `kmalloc`, and it is the main allocator used by the kernel in most allocations. As `kmalloc` does not zero-initialize the provided memory allocations, `kzalloc` has also been implemented, which zero-initializes the allocations made by `kmalloc`.

Listing 3.19: `kmalloc` implementation

```
void* kmalloc(size_t bytes) {
    if (bytes > MAX_CACHE_SIZE)
        return raw_kmalloc(
```

```

        div_ceil(bytes, PAGE_SIZE),
        "kmalloc page",
        RAW_KMALLOC_DYNAMIC_CFG,
        NULL);

    size_t cache_class_size = max(MIN_CACHE_SIZE, next_pow2(bytes));
    return cache_malloc(cache_class_size);
}

void kfree(void* ptr) {
    if (mm_va_is_in_directmap_range(ptr))
        cache_free(ptr);
    else
        raw_kfree(ptr);
}

void* kcalloc(size_t bytes) {
    void* ptr = kmalloc(bytes);
    memzero(ptr, bytes); // implemented in Assembly, uses SIMD instructions
                        // to zero the provided region

    return ptr;
}

```

3.7.12 Userspace Memory Management

A kernel is in charge of creating the illusion that the entire low address space is available to each user process, while enforcing isolation from other processes. User program developers should not need to consider concepts such as physical memory or the MMU. The kernel manages these architectural details and abstracts them for userspace programs.

In terms of memory management, a program is represented in memory as a process, which includes the memory layout of an executable binary plus all its active memory mappings. Each process has its own virtual low address space, with a dedicated MMU mapping that is changed on every [context switch](#), enforcing that one process cannot access the memory of another.

The kernel exposes memory management to userspace through two main mechanisms. The first is the initial memory layout set up when a process is created from an executable, where the kernel parses the binary, and maps its code and data segments into the process address space. The second is when a running process requests additional memory mappings at runtime, either anonymous memory for heap use or file-backed mappings through system calls. Due to time constraints and the limited scope of this project, a proper file system has not been implemented, so file-backed mappings have not been developed. In Linux, the kernel is also in charge of initializing the userspace stack, however, for this project, the decision that the user should allocate its own stack has been made to allow for lower level control from programs.

The following sections describe how the kernel implements these mechanisms.

User Regions

In order to represent and store metadata of a process, the `task_t` struct has been defined. This struct holds information such as the registers state, threads running at the process and the userspace memory regions. A userspace memory region is a range of virtual memory with associated permissions. More information about the task struct will be further explained in Section 3.8.

Permissions determine what can be done inside a memory region by defining if it can be read, written, and or executed. These permissions are not mutually exclusive, and they are enforced by configuring the MMU in such way that the permissions are assured by hardware.

The state of a region determines whether physical memory has been assigned to it. A region can be in three states: uncommitted, meaning no physical pages have been assigned yet, partially committed, meaning some pages have been assigned but not all, and fully committed, meaning all pages in the region have a physical address assigned. This distinction allows the kernel to implement lazy allocation. A region can be reserved in the virtual address space without using physical memory, and physical pages are only assigned when explicitly committed.

In order to change the state of a region the reservation and commit operations are defined. Reserving a region registers a virtual address range with its associated permissions in the process, without assigning any physical memory. Committing a region, or a sub-range of it, allocates physical pages and updates both the MMU mapping of the process and the internal metadata of the region. A special case is the static reservation, `uregion_reserve_static`, which combines reservation and full commit in a single operation and is used for regions whose physical memory has to be available immediately, like for example sections loaded from an executable.

Internal Data Structures

The regions of a process are stored as a red-black tree that holds `uregion_node_t` nodes, ordered from highest to lowest virtual address and without any possible overlap. Each node contains a `uregion_t` struct that holds the start address, page count, permission flags, a bitfield that tracks which pages have been committed, and a red-black tree of `physdata_node_t` nodes that holds the information about the physical pages mapped to the committed pages of the region. The bitfield uses a single 64-bit integer inline for regions of up to 64 pages, and

switches to a heap-allocated array for larger regions, in which case the bitfield is interpreted as a pointer to the array.

Reservation and Commit

When a region or sub-range is committed, the kernel calls `raw_kmalloc` to allocate **direct-mapped** regions, following the same power-of-two decomposition used by `raw_kmalloc` when dynamic allocations are performed. For each block, the physical address is inserted into the `physdata` red-black tree of the region, the MMU of the process is updated to map the user virtual address to the physical address with the permissions of the region, and the corresponding bits in the committed bitfield are set.

Kernel Access to Userspace Memory

Since userspace and kernel space use different MMU mappings, the kernel should not access userspace pointers directly. The architecture does not block this type of accesses (when the task's mapping is set), however, it is not good practice to access low address space from the kernel. Userspace data modification should happen from the kernel mapping, by accessing the **direct-mapped** physical regions registered to the process regions. Figure 3.11 shows an example of how a process has different unordered non-contiguous physical pages mapped to its virtual contiguous regions, and how the kernel can interact with them using the direct-map region.

Operations such as `umemcpy`, `umemzero`, and `ustrncpy` are used to translate userspace virtual addresses to physical addresses via the `physdata` red-black tree, and then access the memory through the kernel **direct-mapped** regions. This allows the kernel to safely read and write userspace memory even when the user process MMU mapping is not set.

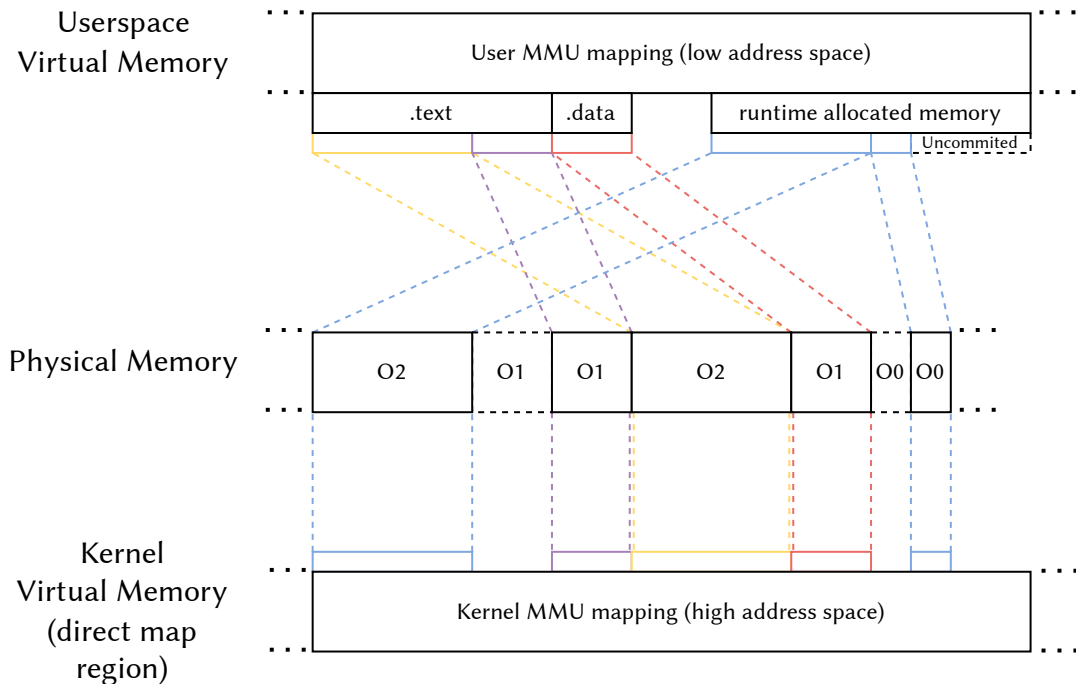
Access Checking and Commit on Demand

Before the kernel performs any operation that reads or writes userspace memory, it must verify that the target range is reserved with the correct permissions and that all pages in the range are committed. This is done by `uregions_check_access`, which walks the region tree over the requested range, and verifies permissions and commit state for each region it crosses. If commit on demand is set to true and a page is reserved but not committed, it is committed in a second pass after the full range has been validated.

Free Space Search

When a process requests a new anonymous memory mapping without specifying an address, the kernel must find a free virtual address range with enough size. `uregion_find_free`

Figure 3.11: Userspace physical memory mapping example



traverses the region tree from the highest address downward, calculating the gap between consecutive regions and returning the top of the first gap that fits the requested size. This top-down allocation strategy keeps low userspace addresses available for position-dependent code and follows the same convention used by most kernels for anonymous mappings.

Region Destruction

When a region is freed, `region_destroy` iterates over the “physdata” red-black tree, and for each node unmaps the user MMU mapping, frees the physical pages via `raw_kfree` using the kernel `direct-mapped` address, and frees the node itself. The committed bitfield array is also freed if the region used the heap-allocated array.

Page Fault Handler

The page fault handler is the memory management submodule that manages the response to data aborts happening while at `EL0`. When a userspace program accesses an unmapped memory address, the MMU throws a synchronous exception that is caught at `EL1` and is decoded as a “data-abort”, by reading the `esr_el1` `sysreg`. By reading the `sysreg` `far_el1`, the address where the exception happened is then obtained, and passed to the `uregions_check_access` function to get the state of the accessed page. From here, two paths can happen:

1. The accessed address resides inside a page within a reserved but not committed region. As the parameter `commit_on_demand` is set to true when calling `uregions_check_access`, the fault address page is committed and zero initialized. The thread's context is restored and the instruction that caused the data-abort is retried after returning to `EL0`, where now the address is accessible, thus not causing a new exception. Concepts involved in the process like [context switching](#) are further explained in [Section 3.8](#).
2. The address does not reside inside a page within a reserved region. In this case, kernels like Linux would defer a signal to notify the process that a fault has happened. Due to the smaller scope of this project, the task is directly terminated.

3.7.13 Binary loading

Executing a user process requires previously loading such process into memory. The sections of a binary must be copied into userspace memory, such as the text, data or rodata sections. Other section like `bss`, don't require any information to be copied, but require their section to be reserved. In order to perform those operations, the kernel must know the sections location and data, for which the [ELF](#) format is commonly used. An [ELF](#) format binary describes the sections to be loaded into memory, their virtual address, and their data [19].

An [ELF](#) parser has been developed, which reads [ELF](#) binaries and reserves the required regions using `uregion_reserve_static` and `umemcpy`.

As a file system has not been implemented due to the scope of the project, userspace binary files are embedded directly into the rodata section of the kernel, and accessed directly by obtaining their address from the linker. The folder `/embedded` inside the project has been created to hold those binaries. The build system automatically embeds them by creating assembly files which use the `.incbin <file path>` directive to embed the binaries into the rodata section with the file name as the global label (`shell.elf` file $\rightarrow$ `shell_elf` label).

From the kernel's side, obtaining the address of the file is done by defining the binary name as functions, process simplified using macros. The pointer and size of the binary is then passed to the `elf_load` function, which loads the [ELF](#)'s regions into a given task struct.

3.8 Scheduler

The scheduler is a fundamental subsystem that builds the core of any operating system. It is responsible for deciding what thread is executed by the CPU at a given time, managing concurrency, parallelism and a fair CPU time distribution amongst the threads.

The following sections describe how the scheduler of this project has been implemented.

3.8.1 Processes

As previously introduced in Section 3.7, a process contains the memory regions that represent a running program. Other than its memory layout, a process also holds the execution context of the program, like its open file descriptors, privileges, and the one or more threads that execute on it. File descriptors for this kernel are scoped to just stdin, stdout and stderr. However, in a kernel with a full virtual file system implementation, holds the process-local IDs that refer to the different nodes that describe a file of the virtual file system.

The data structure defined to represent a process² is the `task_t` struct, shown in Listing 3.20.

Listing 3.20: Task struct definition

```
typedef enum {
    TASK_NEW,    // just created, might not even have threads assigned
    TASK_ALIVE,  // has at least 1 thread alive
    TASK_DYING,  // the process called exit or aborted, defers the killing of threads
    TASK_DEAD,   // task has no threads alive
} task_state_e;

typedef struct task {
    uint64_t      task_uid;
    const char*   name;
    _Atomic task_state_e state;
    atomic_int     exit_code;
    atomic_bool    reschedule_required;

    spinlock_t     memory_lock;
    mmu_mapping     mapping;
    rbtree_t       regions;
    fd_table_t     files;

    spinlock_t     threads_lock;
    uint32_t       threads_per_cpu[NUM_CPUS];
    uint32_t       thread_ref_count;
    thread_ref_node_t* threads;
} task_t;
```

Memory isolation is obtained through the `mapping` and `regions` fields, which define the process's virtual address space as explained in Section 3.7. The `state` field is used to indicate the task's state to multiple cores while scheduling. For example, the `TASK_DYING` state, indicates the scheduler that if any thread that executes at this task is found, it should be killed. This field is not protected by the lock, as it is an atomic variable. The `threads` field, is used for when the task must be terminated, keeping an atomic reference count of the owned threads. The use for `threads_per_cpu` is later explained in Section 3.8.9.

²The terms *process* and *task* are used interchangeably throughout this report.

3.8.2 Threads

A thread is a unit of execution scheduled by the kernel. Threads share the address space of their parent task, and a single task can have multiple threads running concurrently.

Listing 3.21: Thread struct definition

```
typedef enum {
    THREAD_NEW,           // Just created, not scheduled
    THREAD_READY,         // Scheduled, not running
    THREAD_RUNNING,       // Scheduled and running
    THREAD_SLEEPING,      // Waiting until a condition is met
    THREAD_DEAD,          // Must be unscheduled
} thread_state;

typedef struct {
    uint64_t      th_uid;
    task_t*       owner;
    arm_ctx_t     ctx;
    uint64_t      last_access_time_us; // reserved for sleep, not implemented yet
    cpuid_t       sched_cpu;
    _Atomic thread_state state;
} thread_t;
```

As Listing 3.21 shows, the thread struct defines the state required to represent and schedule a thread. Each thread is identified by a unique ID (`th_uid`) and holds a reference to its parent task (`owner`). The hardware execution context is stored in `ctx`, which contains the CPU registers saved and restored on each [context switch](#), plus the PSTATE related system registers (explained in Sections 2.2.3 and 3.5.2). The scheduler uses `last_access_time_us` and `sched_cpu` to track when the thread last ran and on which core is assigned. Finally, `state` determines the thread's current lifecycle state, further explained in Section 3.8.8.

3.8.3 Context Switching

A [context switch](#) is described as the process of transitioning from one context to another. This usually happens when transitioning to a different an exception level, such as when an exception arrives and the processor enters the [Vector Table](#) entry, or when transitioning to a lower [Exception Level](#). As described in Section 3.5.2, every exception entry and exit saves and restores the full processor state into an `arm_ctx_t` struct on the kernel stack. The scheduler uses this mechanism to implement [context switching](#). By modifying the stack's `arm_ctx_t` values, it can modify the execution context that will be executed when retuning to [EL0](#), thus redirection execution to a different thread by restoring its previous state.

Each thread owns a copy of its `arm_ctx_t` in its `thread_t` struct (Section 3.8.2). When a [context switch](#) is required, the `scheduler_ectx_store` function copies the current exception context into the thread's `ctx` field, and `scheduler_ectx_load` writes the incoming thread's

saved `ctx` back into the stack's exception context. The `eret` instruction executed when the exception handler finishes, restores the incoming thread's registers and transitions to [EL0](#) at the address stored in its `elr` field.

In addition to restoring the register state, a [context switch](#) must also update the low address space MMU mapping to the incoming thread's owner task, ensuring memory isolation as described in [Section 3.7](#).

A design choice inspired by the Linux kernel is the use of `SP_EL0` as a fast pointer to the currently running `thread_t`. Since this register is not used as a stack pointer while at [EL1](#) in this project, the kernel uses it to store the address of the current thread struct while at [EL1](#). This allows the `get_current_thread()` inline function to be implemented with a single system register read, avoiding any memory dereference. The `SP_EL0` pointer is updated by `scheduler_ectx_store`, so `get_current_thread()` is only valid while code is executing between a `scheduler_ectx_store` and a `scheduler_ectx_load`. Calling it outside this window will result in undefined behaviour in release builds or a panic in debug builds.

[Listing B.3](#) shows the implementation of `scheduler_ectx_store` and `scheduler_ectx_load`.

3.8.4 Runqueue

The runqueue shown in [Listing 3.22](#) is the data structure that holds the threads available for scheduling on a given CPU core. Each CPU core has its own runqueue, stored in the runqueue array indexed by CPU IDs. This design avoids the need for a single global lock and reduces lock contention.

Listing 3.22: `runqueue_t` struct definition

```
typedef struct thread_node {
    struct thread_node *prev, *next;
    thread_t           th;
} thread_node_t;

typedef struct {
    _Alignas(CACHE_LINE) cpulock_t lock;
    thread_node_t* list;
    thread_t*      current_thread;
    atomic_ulong preemptive_duration_microsec;
    timer_event_t preemptive_event;
    bool schedule_required;
} runqueue_t;

static runqueue_t runqueue[NUM_CPUS];
static spinlock_t runqueue_thread_counts_lock;
_Alignas(CACHE_LINE) static uint32_t runqueue_thread_counts[NUM_CPUS]; // used for load balancing
static atomic_ulong thread_uid_counter;
static arm_ctx_t pre_sched_mode_ctx[NUM_CPUS];
```

The `list` field is a circular doubly-linked list of `thread_node` structs, where each node stores a `thread_t` struct directly. The circular structure is used for round-robin scheduling, as advancing to the next thread will always result in scheduling a new thread just by dereferencing the pointer. Recovering the container node from a `thread_t` pointer can be done using a simple pointer arithmetic offset operation.

The `schedule_required` field indicates that the next call to `scheduler_ectx_load` must select a new thread, instead of continuing with the current one. This separates the decision to reschedule from the moment it is applied. For example, a preemptive timer interrupt running at [EL1](#) can set this flag, but the actual thread rescheduling and switch only happens when the kernel returns to [EL0](#) through the exception return path.

The first field is aligned to a cache line to prevent false sharing between runqueues, as each runqueue is mostly accessed by their assigned [PE](#).

The `preemptive_duration_microsec` and `preemptive_event` fields are described in [Section 3.8.6](#).

3.8.5 Scheduling Algorithm

The scheduler implements a round-robin algorithm over the per-core runqueue list. The scheduling logic is implemented in the `runqueue_schedule()` function, which is called by `scheduler_ectx_load()` whenever `schedule_required` field is set to true or the current thread is in a no longer runnable state. The `runqueue_schedule()` function implementation is shown at [Listing B.4](#).

The function has two phases. First it handles the state of the outgoing thread, then it selects the next thread to run.

Outgoing Thread

The current thread is expected to be in the `THREAD_RUNNING` state. The scheduler attempts to transition it back to `THREAD_READY` via an atomic compare-and-exchange. If the thread was instead in the `THREAD_DEAD` state, for example because another core called `unschedule_thread()` on it, it is removed from the runqueue and deferred the freeing of the node for the end of the function. The `THREAD_SLEEPING` state is reserved for a future mechanism not implemented at the time of writing this report, and currently panics.

Thread Selection

Starting from the node after the outgoing thread, the scheduler walks the circular list looking for the next `THREAD_READY` thread. For each thread, it checks the state of the owning task before attempting to schedule the thread:

- If the owner is not `TASK_ALIVE`, the thread is unqueued and deferred for freeing, and the search continues.
- If the thread is `THREAD_READY`, it is atomically transitioned to `THREAD_RUNNING` and selected as the current thread.
- If the thread is `THREAD_NEW`, it is skipped, new threads cannot be scheduled until explicitly promoted to `THREAD_READY` via `thread_promote_to_ready()`.
- If the thread is `THREAD_DEAD`, it is unqueued and deferred for freeing, and the search continues.

If the list becomes empty during the search, the scheduler returns `NULL`, which causes `scheduler_ectx_load()` to call `scheduler_loop_cpu_exit()` and return the CPU to its pre-scheduler state. The Linux kernel, would panic if this was the case, as the first task in the Linux kernel should never exit. Allowing exiting from the scheduler loop has been implemented because it allows for better debugging and testing scenarios, however removing this feature is planned for when the kernel is in a more mature state.

Deferred Freeing

Threads are not freed immediately when removed from the runqueue. Instead, removed node pointers are stored into a local dynamic array (`to_free`) and freed after the new thread has been selected and `set_current_thread()` has been called. The `deferT` macro makes `free_threads()` to be called automatically when `runqueue_schedule()` returns, regardless of the exit path taken, as it uses the GCC `cleanup` attribute. This allows reading the internal data of the unscheduled nodes without use after free bugs while at the `runqueue_schedule()` function.

3.8.6 Preemptive Scheduling

When a thread is being executed in `EL0`, if the thread does not trigger any exception, the CPU will stay forever in `EL0`, blocking the return to the kernel, avoiding the rescheduling of new threads and denying control to the kernel. Even if the threads did eventually return control to `EL1`, by for example triggering a `yield` syscall, fair CPU time among threads would not be ensured. A thread might hold the core for 2 seconds before yielding, while another thread could occupy the core for just microseconds before yielding. The preemptive mechanism of the scheduler is designed to avoid these cases, and attempts to create a fairer CPU time distribution among threads.

The preemptive action works by setting up a hardware timer interrupt event just before returning from `runqueue_schedule()`. The `preemptive_duration_microsec`

and `preemptive_event` fields are used as the event duration and the event handle respectively, introduced at Section 3.8.4. The `preemptive_duration_microsec` field defaults to $3500\ \mu s$ (3.5 ms), and is declared as an `atomic_ulong` to allow it to be updated at runtime via `scheduler_set_preemptive_duration()` by any core. It is defined per-runqueue rather than globally to allow different preemptive timings per PE in future versions of the kernel. The `preemptive_event` field allows for cancelling the event in cases where the threads yields control to the kernel intentionally, before the event has fired.

This mechanism creates the illusion of concurrency, where even on a single-core processor, the fast switching between tasks makes it seem as though multiple threads are executing simultaneously.

The `runqueue_schedule()` function is only called from the `scheduler_ectx_load()` function, where `interrupts are masked` before. This ensures that the timer event will be triggered after returning to EL0, where SPSR_EL1 is configured with `interrupts unmasked`, thus never being able to be held at EL0 forever. Even if the timer event was set to $0\ \mu s$, the event would not be triggered before the transition to EL0 because the interrupts would be `masked` from the time of setting up the event until transitioning to EL0.

How the timers work is explained in Section 3.10.

3.8.7 Scheduler Loop

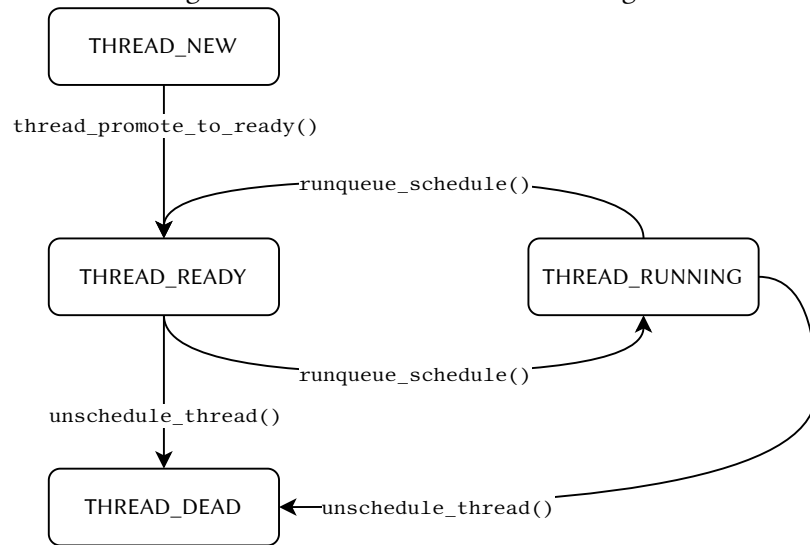
When a CPU core is ready to start executing threads, just after initializing the kernel, `scheduler_loop_cpu_enter()` is called. It scans the runqueue for the first `THREAD_READY` thread, sets up the preemptive timer, and transfers control to EL0 via the assembly function `_scheduler_loop_cpu_enter`, which loads the thread's `arm_ctx_t` and executes `eret`. Each CPU stores its pre-scheduler kernel context in `pre_sched_mode_ctx`, so that if the runqueue ever becomes empty, `scheduler_loop_cpu_exit()` can restore that context and return the core to its idle state via `_scheduler_loop_cpu_exit`.

The scheduler loop refers to the concept that once the kernel has entered the scheduler, the core will always stay in a loop between EL0 and EL1, where the execution of the `_scheduler_loop_cpu_enter` never would return. This is actually intended in most kernels, however, as mentioned before at Section 3.8.5, for easier testing scenarios, exiting the loop has been allowed.

3.8.8 Thread Lifecycle

Figure 3.12 shows the possible state transitions of a thread during its lifetime. A thread is created in the `THREAD_NEW` state and must be explicitly promoted to `THREAD_READY` via `thread_promote_to_ready()` before the scheduler will allow it for execution. These two steps allows the caller to configure the thread before exposing it to the scheduler.

Figure 3.12: Thread state transition diagram



Once ready, the scheduler transitions the thread to **THREAD_RUNNING** via an atomic compare-and-exchange in `runqueue_schedule()`. On the next scheduling event, the running thread is transitioned back to **THREAD_READY**, allowing other threads to run. A thread can be killed at any point by any core via `unschedule_thread()`, which atomically sets its state to **THREAD_DEAD**. The actual removal from the runqueue is deferred until `runqueue_schedule()` finds the dead thread's node during its next traversal of the circular list, which occurs at most after a full loop.

This design is inspired by the Linux kernel, and reduces cross-core lock contention by ensuring that during normal scheduling, list modifications are always performed by the core that owns the runqueue. Other cores can interact with other core runqueues threads, for example, killing them via `unschedule_thread()`, just through atomic operations, requiring no lock at all. The only exception is load balancing, described in Section 3.8.9, where the external core must acquire both its own and the other runqueue lock to transfer threads between lists.

The `threads` field of `task_t` keeps track of the amount of scheduled threads with that task as their owner. It is increased when a thread is inserted to a runqueue, and decreased when removed. When the counter reaches zero, the task is freed. This also ensures that no thread will access their owner task struct after it has been freed, because it is only freed after all its threads are fully removed from the runqueues.

3.8.9 Parallelism

The kernel boots on a single core (core 0), which performs all initialization tasks before activating the remaining cores. Secondary cores are woken up using the CPU_ON function of the [Power State Coordination Interface \[20\]](#) (PSCI), which is the ARM standard for power management and core lifecycle control. PSCI functions are called through the [SMC Calling Convention \[21\]](#) (SMCCC), which starts a Secure Monitor Call (smc instruction) to the firmware running at EL3, setting up as parameters, the target core, its entry point physical address, and a context value. This is a similar operation to what a userspace program calls a system call, but happens from the kernel to the firmware, instead of happening from the user to the kernel.

Once a secondary core is activated, it executes its own initialization sequence, configuring its local interrupt controller interface, MMU state, kernel stack, and kernel relocation. The initialization process of secondary cores is similar to the main core initialization, but avoids reinitialization of shared mechanisms as vmalloc or the page allocator.

Finally calls `scheduler_loop_cpu_enter()`, entering the same scheduling loop as the primary core. From this point, each core operates independently on its own runqueue.

Load Balancing

Until this point in the report, if a thread is spawned in a given runqueue, all its lifecycle will happen at that runqueue. This, however, would mean that the amount of work divided across runqueues can be unbalanced. In order to balance the threads assigned to each runqueue, a load balancing mechanism has been designed.

The algorithm must not only consider the total number of threads divided across runqueues, but also the distribution of threads per task. When a task spawns a new thread, the natural expectation by the user is that the new thread will provide additional computing power by running on a different core. Even if all threads were perfectly balanced globally, this expectation could not be met if all threads of a task are, for example, assigned to the same runqueue, benefiting from concurrency but not from parallelism.

Thread creation. `schedule_thread()` selects the target runqueue where the new thread will be inserted by applying the following criteria. The primary criterion is the number of threads already assigned to each core by the same task (`threads_per_cpu`), selecting the core with the least amount. When two or more cores are matched on this criterion, the secondary criterion is the global runqueue size (`runqueue_thread_counts[NUM_CPUS]`), selecting the globally least loaded runqueue among them. This guarantees that each new thread is placed in the position that ensures per-task balance, and advances global balance as much as possible.

Thread destruction. `unqueue_thread()` restores per-task balance after the removal of the killed thread. After removing the thread, the algorithm computes $t_{\max}$, the maximum count of that same task across all other runqueue, again using the global runqueue size as a tiebreaker when task counts are equal. It also defines $t_{\min}$ as the number of threads the task has in the runqueue it just removed the threads. If $t_{\max} - t_{\min} \leq 1$, the per-task distribution already satisfies the balance invariant and no action is taken. Otherwise, one thread from the $t_{\max}$ runqueue is transferred to the $t_{\min}$ runqueue, restoring balance in a single move. A single migration is always enough, because one deletion moves the per-task balance by at most one, so one move restores the invariant.

The balance invariant maintained throughout is: for every task with N threads across M cores, each core holds either $\lfloor N/M \rfloor$ or $\lfloor N/M \rfloor + 1$ threads of that task. This is a hard invariant, enforced on every insertion and deletion. Global balance is a property that emerges from the tiebreaker logic in `schedule_thread` but is not independently enforced.

Independent algorithm tests performed to check the global balance using random thread insertions and deletions result in an average global imbalance of approximately 3% for 100 threads. This imbalance is inversely proportional to the total number of scheduled threads. It increases when the thread count is low and decreases as N count gets higher, which is the expected behavior given that the remainder of $N \bmod M$ becomes negligible relative to N as N increases.

3.9 Device Control Subsystem

A driver is the abstraction layer that translates higher level commands into hardware specific instructions, usually through `MMIO` memory operations.

Even though the kernel explained in this project is designed for the IMX 8M Plus `SoC`, the kernel has been designed to allow future versions of the kernel to be able to run into any Armv8-A platform. However, this would seem impossible at a first glance, as different `SoCs` provide different hardware peripherals, with different registers, functionalities and hardware configurations. The kernel should require the knowledge of the specifics of each peripheral, and adapt every subsystem that uses those peripherals for their internal specifications.

In order to avoid this problem, a `Hardware Abstraction Level (HAL)` is defined, which allows the kernel to enforce a common interface for similar purposed peripherals through the *device control subsystem*. For example, even though two different `SoCs` may implement completely different `UART` peripherals, with the HAL layer, the kernel just has to call a common interface function to initialize, write or read into those peripherals. This HAL is implemented in the kernel through a device registration system. Each driver is registered under a device class (e.g. `DEVICE_CLASS_SERIAL`, `DEVICE_CLASS_IRQ_CTRL...`), exposing a

struct of function pointers that implement the interface for that class. The rest of the kernel interacts exclusively with these interfaces³, remaining agnostic of the hardware. When the kernel is executed in a different platform, only the platform-specific drivers need to be replaced, the kernel subsystems that use them require no changes. Listing B.5 shows the public API exposed through the `device.h` header for when the kernel requires communication with the registered peripherals.

3.9.1 Device Classes

At the time of writing this report, the following device classes have been defined for the kernel, each one representing different operations grouped by common peripheral types:

- `DEVICE_CLASS_GENERIC` is reserved for custom device types that do not relate to any of the other classes. No generic driver has been implemented yet, and it is intended for future extensions of the kernel.
- `DEVICE_CLASS_IRQ_CTRL` is used by interrupt controllers, providing operations to detect the interrupt source that triggered an IRQ and to signal its completion (acknowledge the irq). It is used by `irq_dispatch` (Section 3.5.2) to identify and acknowledge incoming interrupts.
- `DEVICE_CLASS_SERIAL` defines byte-level read and write operations, used by serial peripherals such as `UARTs` to provide the kernel with a communication interface.
- `DEVICE_CLASS_TIMER` allows scheduling timer events, which are interrupt-driven notifications that fire after a configured time duration.
- `DEVICE_CLASS_CLOCKSOURCE` defines operations to read the current cycle count and frequency of a hardware clock, used by the kernel to track global time and measure time intervals between two points.
- `DEVICE_CLASS_THERMAL_SENSOR` provides operations to read temperature measurements from on-chip thermal sensors, used to monitor and respond to over warning temperature conditions.

The version of the kernel at the time of writing this report, only allows determining what devices the kernel implements at compile-time. Other kernels like Linux, rely on the “Device Tree Blob” to allow for runtime driver selection. It is a data structure provided by the `bootloader` that tells the kernel the memory map, available devices, their physical address base (for `MMIO` register operations), and other relevant information about them.

³See `/include/kernel/devices/driver_ops` at the repository for the defined class operations.

3.9.2 Implemented Drivers Overview

Driver registration is performed via `KERNEL_INITCALL`, a macro that schedules initialization functions to be called during the kernel initialization stage. As explained before, driver initialization should ideally be driven by a Device Tree Blob (DTB), however, DTB parsing is out of the scope for this project, and driver registration is performed manually as a simpler alternative.

The following drivers have been implemented for the i.MX 8M Plus SoC⁴:

UART

The [Universal Asynchronous Receiver-Transmitter \(UART\)](#) driver targets the `UART2` peripheral of the i.MX 8M Plus. It provides the serial interface used for kernel debug output and `stdin`, `stdio`, and `stderr` files. It is registered under `DEVICE_CLASS_SERIAL` and allows for interrupt driven I/O.

GICv3

The [Generic Interrupt Controller \(GIC\)](#) version 3 (GICv3) driver implements the `DEVICE_CLASS_IRQ` interface, providing interrupt routing and acknowledgement for the kernel. It is responsible for configuring interrupt priorities, trigger modes, and CPU affinity for each interrupt source, and is used by `irq_dispatch` to identify and dispatch incoming interrupts as described in [Section 3.5.2](#).

ARM Generic Timer

The ARM Generic Timer is a standard Armv8-A peripheral present on all standard Armv8-A implementations. It registers two devices: a clocksource under `DEVICE_CLASS_CLOCKSOURCE`, which provides a monotonic counter for timekeeping, and a timer under `DEVICE_CLASS_TIMER`. The timer fires as a Private Peripheral Interrupt (PPI 27), local to each CPU core.

Thermal Monitoring Unit

The i.MX 8M Plus Thermal Monitoring Unit (TMU) driver registers under `DEVICE_CLASS_THERMAL_SENSOR` and monitors the on-chip temperature sensors. Even though it might seem like a critical driver for the hardware's safety, the firmware at the [Secure World](#), already implements a separate driver for this device, and fully shuts down the SoC if reached the

⁴The following drivers are briefly described, since each driver only relates to its specific peripheral's hardware, making the implementation details neither generalizable nor worth elaborating here. Full source is available in `src/drivers/` directory of the project repository.

critical temperature. This driver has been implemented primarily for different demo showing purposes.

3.10 Time Subsystem

The time subsystem provides an abstraction layer over the `DEVICE_CLASS_CLOCKSOURCE` and `DEVICE_CLASS_TIMER` device classes, and defines the `clock_t` interface that merges timekeeping and timer event scheduling.

A `clock_t` is created from a clocksource device and optionally, a timer device. The clocksource provides a monotonic hardware counter from which the current time from hardware wake up, can be read via `clock_now()`, returning a `timepoint_t` in nanoseconds. The timer device, if provided, allows the scheduling of timer events via `timer_create_event()` and `timer_create_event_delta()`.

The main purpose for this abstraction is that hardware timers usually only support a single scheduled event at a time. The time subsystem avoids this limitation by maintaining a sorted linked list of pending `timer_event_t` events per clock, where only the earliest event is programmed into the hardware timer. When that event fires, its callback is called and the hardware timer is reprogrammed for the next event. This allows any number of virtual timer events to be set up on top of a single hardware timer. The kernel additionally exposes a global pre-initialized high-resolution clock, `HRTIMER`, linked to the the ARM Generic Timer drivers, which is used by the preemptive scheduler as described in Section 3.8.6, and serves as the default timer for the kernel.

3.11 VFS Subsystem

The **Virtual File System (VFS)** provides a minimal abstraction layer that allows the kernel to interact with different devices through a file descriptor interface. Each task maintains a file descriptor table (`fd_table_t`), implemented as a red-black tree keyed by file descriptor number. Each entry relates a file descriptor with a `file_operations_t` struct, which is a set of function pointers implementing `read`, `write`, `open`, `close`, and `mmap`, and a device specific context pointer passed to each operation. The main idea is that those operations can implement different behaviors for different files, and userspace programs can interact with those behavior via file operation system calls.

The VFS implementation at the time of the writing of this report is minimal and upgrades are planned for future updates. `vfs_open()` is not yet implemented, as it would require a full filesystem. Instead, file descriptors are created at compile time directly to devices via `vfs_bind()`, which is used internally to attach `stdin`, `stdout`, and `stderr` to the serial device at

task creation time, as described in Section 3.12.

3.12 I/O Subsystem

The I/O subsystem provides interrupt-driven input and output for the kernel, built on top of the `VFS` and serial device abstractions. It is made of three layers: the `term` buffer, the `vfs_serial` binding, and the kernel's "stdio" interface.

Term

The `term` module implements an interrupt-driven I/O buffer. Writing to a `term_handle` first attempts to send data directly through the assigned output function, usually backed by a `putc` call to the serial driver. If the hardware is not ready to accept data (hardware FIFO is full), the remaining bytes are stored in a dynamically-growing linked list of page-sized buffers. When the hardware signals that it is ready to transmit new data via a TX interrupt event, `term_notify_ready()` is called, which drains as much of the pending buffer as the hardware will accept. This design avoids polling or busy-waiting for the peripherals, and allows the kernel to continue execution while output is transmitted asynchronously.

The same structure is used in reverse for input, where incoming bytes from the RX interrupt are pushed into the term buffer, and `term_remove_head()` is called by the `VFS` read operation to consume them. This, for example, allows userspace programs to read the input obtained from the main serial device.

VFS Serial

`vfs_serial_bind_stdio()` is called at each task creation time to bind file descriptors 0, 1 and 2 (`stdin`, `stdout` and `stderr`) to the primary serial device (number 2 `UART` for the IMX). The `stdout` and `stderr` files share the same `term_handle` and output buffer, while `stdin` uses a separate one maintained by the RX interrupt handler. Each file descriptor exposes the specific `file_operations_t`. The `stdin` file provides only `read`, while `stdout` and `stderr` provide only `write`. Thanks to those file operations, userspace programs can read and write to the main serial device of the kernel via `read` and `write` syscalls to such files.

Kernel stdio

The kernel stdio interface provides `print()` and `printf()` for debug and general kernel output. Unlike the interrupt-driven path, these functions write directly and synchronously to the primary serial device with interrupts `masked`, to ensure messages arrive at the point of

sending, with no delay. For printf, a minimal heap based formatting module has been made, inspired by the C standard library printf function.

3.13 System Calls

System calls are the mechanism used by userspace programs to communicate and request services from the kernel. They create the connection between [EL0](#) and [EL1](#), allowing unprivileged code to perform controlled privileged actions such as memory management, I/O, and process control, in a controlled and safe way, enforced by the kernel.

In AArch64 state, a system call is performed by executing the `svc` instruction, which triggers a synchronous exception with exception class `ESR_EC_SVC_AARCH64`. As described in [Section 3.5.2](#), this exception is caught by `exception_handler_sync` from the exception handler, which dispatches to the syscall subsystem.

System Call Dispatcher

The syscall subsystem entry function is called `syscall_dispatch()`, shown in [Listing 3.23](#). It is called within the window created by calling `scheduler_ctx_store()` and `scheduler_ctx_load()` to ensure that the `get_current_thread()` inline function (that accesses `SP_EL0`) is valid, and that the process state is stored into the task's context struct. The `syscall_dispatch()` function reads the current thread `x8` value, that holds the syscall code number set by the user before executing the `svc` instruction. That integer code is used to index into the syscall table, which is a simple array containing the different syscall handlers assigned to each syscall code. The registers `x0-x5` define the system call parameters, and are passed to the syscall handlers functions.

When a system call is triggered, a result code is expected at the register `x0`, which is set to the task's context struct after the syscall code handler returns.

Listing 3.23: `syscall_dispatch()` function implementation

```
static const syscall_handler SYSC64_TABLE[SYSC_COUNT] = {
    [SYSC_EXIT]    = syscall64_exit,
    [SYSC_READ]    = syscall64_read,
    [SYSC_WRITE]   = syscall64_write,
    [SYSC_SPAWN]   = syscall64_spawn,
    [SYSC_KILL]    = syscall64_kill,
    [SYSC_YIELD]   = syscall64_yield,
    [SYSC_MMAP]    = syscall64_mmap,
    [SYSC_MUNMAP]  = syscall64_munmap,
};

static inline void sysc64_set_result(gpr_t* gpr, int64_t result) {
    gpr[0] = result;
}
```

```

}

void sysc64_dispatch() {
    gpr_t* gpr = get_current_thread()->ctx.x;
    size_t syscall_id = gpr[8];

    if (syscall_id >= SYSC_COUNT) {
        sysc64_set_result(gpr, SYSC64_RES_UNKNOWN);
        return;
    }

    int64_t result = SYSC64_TABLE[syscall_id](
        gpr[0],
        gpr[1],
        gpr[2],
        gpr[3],
        gpr[4],
        gpr[5]);

    sysc64_set_result(gpr, result);
}

```

Table 3.5 summarizes the syscalls implemented. Most implemented syscalls are heavily inspired or directly emulate the behavior expected from Linux syscalls, although limited to the scope of the project.

Table 3.5: Implemented system calls

Syscall	Description
read	Reads from a file descriptor into a user-space buffer.
write	Writes from a user-space buffer into a file descriptor.
mmap	Maps anonymous memory into the process address space.
munmap	Unmaps a previously mapped memory region.
exit	Terminates the calling process.
kill	Terminates a thread within the calling process by thread UID .
yield	Voluntarily yields the CPU to the scheduler.
spawn	Creates a new thread within the calling process.

read and write

The `read` and `write` syscalls call to the [VFS](#) subsystem described in Section 3.11, with the provided arguments. All the memory operations are made by copying via `umemcpy()` to validate that the userspace buffer resides within a mapped region with the correct permissions before copying, and accessing userspace memory safely. The permissions `UREGION_F_WRITE` for `read` and `UREGION_F_READ` for `write` are checked.

mmap

The `mmap` syscall reserves a region of the process virtual address space via `uregion_reserve()`, as described in Section 3.7.12. Only anonymous mappings (`MAP_ANONYMOUS`) are supported, as file-backed mappings require a full filesystem. The protection flags `PROT_READ`, `PROT_WRITE`, and `PROT_EXEC` are supported. If the requested address is not available and `MAP_FIXED` is not set, the kernel selects a free region automatically via `uregion_find_free()`. Memory mappings obtained from `mmap` are reserved, but not committed until the process attempts to access the different pages of the mapping.

munmap

The `munmap` syscall calls the `uregion_free()` function to free a previously mapped region with `mmap`. As explained in Section 3.7.12, partial frees are not supported yet, so the caller must provide the parameters (size and map address) that exactly matches the mapped area.

exit

The `exit` syscall calls `terminate_task()` on the current thread's owner task, which marks the task as `TASK_DYING`. The scheduler then lazily removes them from the runqueue as described in Section 3.8.5.

kill

The `kill` syscall terminates a specific thread within the calling process by thread `UID`. It searches the owner task's thread list for the matching `UID` and calls `unschedule_thread()`, which atomically marks it as `THREAD_DEAD`. The actual removal from the runqueue is deferred to the scheduler as described in Section 3.8.8.

yield

The `yield` syscall, calls `schedule()` on the current CPU, which sets `schedule_required` as true and cancels any pending preemptive timer event. On return from the syscall, `scheduler_ectx_load()` detects the flag and selects a new thread, giving up the remaining CPU time voluntarily.

spawn

The `spawn` syscall is kernel-specific with no direct Linux equivalent. It creates a new thread in the calling process at a given entry point address, making it similar to `pthread_create` but implemented entirely in the kernel, without a threading library. Before creating the thread,

the kernel validates that the entry point address is mapped in the process's virtual address space and that the region is marked as executable. When spawned, the new thread's **UID** is returned to the caller, and the thread is immediately promoted to `THREAD_READY`. The new thread receives its own **UID** in `x0` and the caller-provided argument in `x1`.

Chapter 4

Results

THIS chapter presents measurements of the kernel's performance-critical paths. It first demonstrates successful userspace execution, then benchmarks context switch latency, system call overhead, and memory allocator performance across all allocation paths.

4.1 Userspace Execution

A custom Standard Library has been made in Rust for creating userspace code without the requirement of writing code in assembly each time a system call has to be made. This library also includes the standard initialization code to allocate the userspace stack, which is written in Assembly and is linked automatically by the Library's provided Makefile. The implementation can be seen at the `/userspace/stl` folder and a project template can be copied from `/userspace/template`. Rust has been used as it allows for defining a custom global allocator, that handles all the allocations. This means that all of Rust's `std::alloc` structures (`Vec`, `HashMap`...) are available by default when the implemented library is used. C headers to link against the Rust STL implementation are also provided.

As a simple hello world program has been made as an example of running userspace code. Listing 4.1 shows how its loaded from the kernel, and Listing 4.2 shows how it is implemented. The output obtained from `UART` is shown at Listing 4.3.

Listing 4.1: Process loading from the kernel

```
noreturn void kernel_entry()
{
    if (get_cpuid() == 0) {
        if (!mm_kernel_is_relocated())
            kernel_early_init();
        else
            kernel_init();
    }

    task_t* hello = task_new("hello world");
```

```

uintptr_t entry;
elf_load(
    hello,
    EMBEDDED_BINARY(hello_elf),
    EMBEDDED_BINARY_SIZE(hello_elf),
    &entry);

schedule_ready_thread(hello, entry);

scheduler_loop_cpu_enter();

loop asm volatile("wfi");
}

```

Listing 4.2: Hello world example

```

#![no_std]
extern crate stl;

#[unsafe(no_mangle)]
extern "C" fn main() -> i32 /* The STL automatically calls exit with the provided code */ {
    for i in 1..=5 {
        stl::printf!("Hello from userspace! {}\n\r", i); // rust manages the formatting and the STL
        handles the write syscalls
    }

    /* Other common STL provided functions
    thread_create(entry, arg)
    thread_kill(thid)
    syscall_mmap(addr, length, prot, flags, fd, offset)
    syscall_munmap(addr, length) */

    0
}

```

Listing 4.3: Hello world userspace logs

```

Hello from userspace! 1
Hello from userspace! 2
Hello from userspace! 3
Hello from userspace! 4
Hello from userspace! 5

```

4.2 Experiment Setup

All measurements presented in this chapter were collected on the target hardware described in Section 2.3: a FRDM i.MX 8M Plus development board, equipped with a quad-core ARM Cortex-A53 processor running at 1.2 GHz and 4 GiB of LPDDR4 memory. All benchmarks were executed directly on real hardware, without any emulation layer, which ensures that the

results reflect true hardware behavior including cache effects, memory latency, and pipeline characteristics that emulators typically mask.

All benchmarks are compiled at `-O2` optimization level. Debug assertion macros are removed in all experiments, ensuring that no debug instrumentation interferes the measured paths.

The compiler used for all benchmarks is the `arm-none-eabi-gcc 15.2.1` (Arm GNU Toolchain 15.2.Rel1).

Performance Monitoring Module

In order to obtain performance metrics, an independent module has been created which exposes relevant functions for this purpose, whose main API is shown at Listing 4.4. For the implementation, independent hardware counters are used for all measurements. CPU cycles are read from the *Performance Monitors Cycle Count* `sysreg pmcctr_el0`, which increments once per CPU clock cycle. Wall-clock time is read from `cntpct_el0`, the system counter, which increments at a fixed frequency of 24 MHz, even when the CPU is in idle state. Cycle counts are converted to nanoseconds using the known CPU frequency of 1.2 GHz, and system counter ticks are converted using the `cntfrq_el0 sysreg`, which reports the counter frequency at runtime. The modules functions have been developed under careful considerations to their own overhead, which is analyzed at Section 4.2. To see the full implementation see `/src/performance_monitor.c` at the repository.

Listing 4.4: Performance monitor module main API functions

```
// inspired by MPI_Win_fence use
void performance_monitor_fence();
// reads last fence metrics
monitor_data_t performance_monitor_read_fence(uint64_t cpu_freq);
monitor_data_t performance_monitor_print(uint64_t cpu_freq, const char* tag);
void performance_monitor_print_metrics(
    const kvec(monitor_data_t)* monitor_data,
    uint64_t cpu_freq,
    const char* tag);

void performance_monitor_fence_notify_exception();
```

Two clocks are used because the CPU cycles source stops when `wfe/wfi` instructions are called, which set the core in low power modes until a hardware event or an `IRQ` arrives respectively. Comparing both clocks is done to obtain the total `PE` use percentage, as the monotonic clock keeps counting even when the cycles clock is stopped due to `wfe/wfi` instructions. As all the synchronization primitives used for this project use the `wfe` instruction, this gives the information about how much time a `PE` was waiting for another to release the lock.

The measurements include the following metrics: minimum, maximum, mean, standard deviation, active time percentage, and the p50, p95, and p99 percentiles. The exception counter is also recorded per sample, allowing to determine how many exceptions arrived while the different measurements were running. Unless noted, all the experiments are locked by an IRQ lock, to avoid any exception interfering with the results.

Scheduler benchmarks are executed with all four cores active and the preemptive scheduler running, while the rest are tested with just one core, as they are not parallel mechanisms.

Experiment Precision

The precision of the main measurement API was done by placing two consecutive `performance_monitor_fence()` calls with no work between ($N = 20,000$), shown in Table 4.1. This function is the main function for measurements. Listing 4.5, shows the code executed to measure the API overhead and precision.

Listing 4.5: Performance monitor module main API functions

```
for (size_t i = 0; i < 20000 /* N */; i++) {
    performance_monitor_fence(); {
        /* in normal experiments, the code to be monitored is placed here */
    } performance_monitor_fence();
    // code outside the monitored fence is not taken into consideration, so the
    // calls to performance_monitor_read_fence and the kvec_push are not measured.
    monitor_data_t in = performance_monitor_read_fence(IMX8MP_CPU_FREQ);
    kvec_push(&fence_data, &in);
}
```

Table 4.1: Performance monitor fence overhead (N = 20,000 samples). The wall-clock resolution is limited by the 24 MHz system counter (41.7 ns per tick), which explains the zero values at low percentiles.

Metric	Wall clock	CPU time
Minimum	0 ns	27 ns
Maximum	250 ns	174 ns
Mean	14 ns	27 ns
Std dev	39 ns	1 ns
IQR	0 ns	0 ns
p50	0 ns	27 ns
p90	125 ns	27 ns
p95	125 ns	27 ns
p99	125 ns	27 ns
Cycles (min / avg / max)	33 / 33 / 209	
Exceptions	0 / 20,000 samples	

For sub-microsecond measurements, the wall-clock, at 24 MHz (~ 41.7 ns per tick) does not provide enough resolution, so CPU cycle count are the primary metric throughout this test.

The average fence cost is 33 CPU-clock cycles (27 ns), with a standard deviation of 1 ns. The maximum of 209 cycles (174 ns) seems to correspond to an instruction cache miss on the first function call plus another data miss, as the internals of `performance_monitor_fence()` require read and write operations to some global variables (40-bytes total, aligned to the cache line). No synchronous exceptions happened across all samples, as expected due to the use of an IRQ lock.

In order to validate that the test was working correctly, an inline assembly nop instruction was added for another execution. The nop instruction does nothing and takes 1 CPU, cycle. As expected, the results changed to an average of 34 cycles for each measurement.

This analysis shows that the overhead from the measuring module itself is negligible, and that provides an extremely precise resolution, scoped to sub-nanoseconds.

4.3 Context Switch Latency

Listing B.6 shows the userspace code executed in this experiment. It is written in Rust, it performs no memory allocations, and spawns 200 threads. All the threads attempt to acquire a global lock at the same time in order to print a formatted message (`LOCK.locked(|_| printf!("Thread {} spawned!", arg));`). This code is inefficient in purpose to allow multiple preemptive

actions to be taken by the kernel, where the context switching can be analyzed. IRQs have been disabled between the different measurements to ensure that IRQ driven I/O does not interfere with the results.

Scheduler Context Switch Results

This first experiment, shown in 4.2 targets only the scheduler function in charge of the context switching `scheduler_ectx_load()` and `scheduler_ectx_store()`. As seen in Section 3.8, `scheduler_ectx_store()` calls the `runqueue_schedule()` function internally, so scheduling is also considered.

Table 4.2: Preemptive context switch latency breakdown (N = 18,318 samples, IRQs masked during measurement). *Store*: `scheduler_ectx_store()`. *Load*: `runqueue_schedule()` + `scheduler_ectx_load()`.

Metric	Store phase		Load phase	
	Wall	CPU	Wall	CPU
Minimum	0.250 μ s	0.268 μ s	1.750 μ s	1.757 μ s
Maximum	2.625 μ s	2.575 μ s	9.500 μ s	9.535 μ s
Mean	0.274 μ s	0.274 μ s	2.390 μ s	2.390 μ s
Std dev	0.079 μ s	0.063 μ s	0.468 μ s	0.466 μ s
IQR	0.000 μ s	0.000 μ s	0.375 μ s	0.343 μ s
p50	0.250 μ s	0.270 μ s	2.250 μ s	2.285 μ s
p90	0.375 μ s	0.275 μ s	2.750 μ s	2.716 μ s
p95	0.375 μ s	0.279 μ s	3.250 μ s	3.244 μ s
p99	0.375 μ s	0.296 μ s	4.250 μ s	4.296 μ s
CPU active (min/mean/max)	71% / 102% / 147%		94% / 99% / 106%	
Cycles (min/mean/max)	322 / 329 / 3090		2109 / 2868 / 11443	
Exceptions	0 / 18,318 samples		0 / 18,318 samples	

The store phase has a median cost of 329 cycles (0.270 μ s), corresponding to taking the `runqueue`'s lock, and setting up the fast current thread pointer by writing the new value into `SP_EL0`. It is a small function, thus why it is so performant.

The load phase dominates at a median of 2,868 cycles (2.285 μ s), covering the scheduling decision in `runqueue_schedule()` (circular list traversal, atomic compare-and-exchange, and preemptive timer reconfiguration). No mapping switch is performed, due to only one task being scheduled.

The combined preemptive context switch costs approximately 3,197 cycles (2.66 μ s) at the median.

The CPU utilization of the load phase averages 99%, showing that `runqueue_schedule()` completes without spinning on the `runqueue cpulock` in the common case. With 200 active threads distributed across four cores, lock contention is low and the WFE wait path is rarely taken. The p99 latency increase to 4.296 μ s seems to be attributable to cache effects rather than lock contention. Values exceeding 100% are measurement errors caused by the wall-clock resolution limit, as the execution is extremely fast when compared with wall-clock resolution.

Context Switch Load and Store

As directly measuring the `ectx_store` and `ectx_load` macros (shown at Section 3.5.2 and Listing B.1) is not possible, due to not being able to use registers before executing them, due to their context saving nature, an independent wrapper function has been made, shown at Listing 4.6 and Listing 4.7. This wrapper encapsulates both macros, so the pure performance of the macros can be measured.

Listing 4.6: Assembly context switch experiment code

```
.globl ectx_metrics // executed inside a loop.
ectx_metrics:
    str lr, [sp, #-16]!
    bl performance_monitor_fence
    ectx_store
    bl push_store_fence_end // c wrapper that pushes into the vector (with no
                           // overhead compared to performance_monitor_fence)

    bl performance_monitor_fence
    ectx_load sp
    bl push_load_fence_end
    ldr lr, [sp], #16
    ret
```

Listing 4.7: C context switch experiment code

```
irqlocked() for (size_t i = 0; i < 50000; i++) {
    ectx_metrics();
}
```

Table 4.3: `ectx_store` and `ectx_load` macro overhead in isolation (N = 50,000, IRQs masked).

Metric	ectx_store		ectx_load	
	Wall	CPU	Wall	CPU
Minimum	0.000 ns	111 ns	125 ns	144 ns
Maximum	2375 ns	2304 ns	2000 ns	2016 ns
Mean	109 ns	111 ns	142 ns	144 ns
Std dev	43 ns	9 ns	44 ns	9 ns
IQR	0 ns	0 ns	0 ns	0 ns
p50	125 ns	111 ns	125 ns	144 ns
p99	125 ns	111 ns	250 ns	250 ns
Cycles (min/mean/max)	134 / 134 / 2765		173 / 173 / 2420	

The `ectx_store` and `ectx_load` macros cost 134 and 173 cycles (111 ns and 144 ns) respectively, combining for 307 cycles (255 ns). Both use [SIMD](#) instructions to save and restore the full register context struct. The zero IQR and 9 ns standard deviation confirm near-deterministic execution where isolated executions up to 2,765 cycles correspond to cache misses on the first invocations.

The total preemptive context switch costs 3,197 cycles (2.66 μ s) at the median, plus 307 cycles that are spent saving and restoring the register file. The scheduler overhead is therefore dominated by the scheduling decision, which scales well under the tested experiment. The MMU switch has not been measured due to being applied at the last step of the scheduler, however, its impact is known, due to each mapping operation requiring invalidating all cache entries related to MMU page table entries (done by specific instructions that target this requirement). The performance impact of [MMU](#) cache misses caused by the mapping switches is very likely higher than that of the context switch and scheduling themselves, as the execution only takes around 3 μ s.

4.4 Syscall Latency

Listing [B.7](#) shows the userspace program created in Rust for this benchmark. Listing [B.8](#) shows the `kernel_entry` configuration for this benchmark. The full logs are shown at Listing [B.9](#) and summarized in Table [4.4](#).

The benchmark attempts to represent best-case latency for each syscall. [IRQs](#) are masked, a single thread is running, and allocator caches are warm from previous iterations, and the cache allocator fast path is available. The intent is to measure the hot paths of each syscall with the expectation that all relevant data structures are already in the L1 data cache, isolating

the cost of the kernel logic from memory latency effects. Each measurement starts from the syscall dispatcher entry to its return, excluding the exception vector path and context switch overhead, analyzed in previous sections. All measurements were taken in the same execution of the program.

Table 4.4: System call latency (N = 20,000 per syscall unless noted, IRQs masked, -O2).

Syscall	N	Min	Mean	p50	p99	Max	Cycles (mean)
yield	20,000	370 ns	370 ns	370 ns	380 ns	1,219 ns	445
kill	20,000	162 ns	214 ns	215 ns	253 ns	2,180 ns	258
spawn	20,000	882 ns	1,074 ns	1,010 ns	2,990 ns	3,877 ns	1,289
read	20,000	992 ns	1,151 ns	1,150 ns	1,214 ns	4,542 ns	1,382
write	20,000	1,466 ns	2,123 ns	1,644 ns	3,803 ns	24,213 ns	2,548
mmap	20,001	465 ns	594 ns	586 ns	748 ns	2,854 ns	714
munmap	20,000	3,724 ns	4,225 ns	4,228 ns	4,597 ns	7,908 ns	5,070
exit	1	2,527 ns	2,527 ns	2,527 ns	2,527 ns	2,527 ns	3,033
commit	20,001	12,140 ns	12,742 ns	12,705 ns	13,405 ns	33,138 ns	15,290

yield is one of the cheapest syscalls, at 445 cycles (370 ns), as it only marks `schedule_required` and cancels the preemptive timer event, with no heavy memory or scheduling work involved.

kill is cheaper than **yield**, at 258 cycles, as it reduces to almost a single atomic compare-and-exchange on the target thread's state, with the actual runqueue removal deferred to the scheduler.

spawn (1,289 cycles) covers thread struct allocation, context initialization, and runqueue insertion with load balancing. As the thread node structs can be allocated with the cache allocator, involved `vmalloc` and `page_malloc` allocations are avoided most of the times, thus why the results are that fast.

read (1,382 cycles) represents the path when no data is ready (0 result), so no memory copies are performed.

write (2,548 cycles mean) visible in the gap between p50 (1,644 ns) and p75 (3,294 ns) shows a non-linear distribution. The lower gap corresponds to writes done directly into the `UART` hardware FIFO, while the upper ones reflect cases where the term buffer is full and data must be queued for deferred output via the `IRQ` driven I/O.

mmap (714 cycles) is cheaper than `yield` in cycles because it only inserts a node into the virtual memory red-black tree without touching physical memory (and the node path is hot for this benchmark due to `munmap` being called just after each iteration). Physical pages are allocated lazily on first access, measured separately as `commit_on_demand`.

munmap (5,070 cycles) is the most expensive non-memory System Call, as it must traverse the red-black tree, unmap the MMU entries, free the physical pages, and invalidate the MMU page tables cache lines.

exit (3,033 cycles) is only executed one time, just after exiting the benchmarking task. The performance represents the cost of the cold path of exit. Other independent benchmarks, where multiple tasks were spawned and exited after some time-consuming operations shows similar results, as the scheduler performs multiple preemptive switches and the hot paths are cleared from the cache. However, other benchmarks where the task is immediately created and deleted, the mean result is reduced significantly to around 552 cycles. This shows the impact of the cache in the performance of the CPU.

commit_on_demand (15,290 cycles / 12.7 μ s) represents the true cost of a memory allocation. Involves physical page selection from the buddy allocator, MMU page table update, and MMU page tables cache lines invalidation.

4.5 Allocator Latency

Listing B.10 shows the full logs from the benchmark. Listing B.11 shows the executed benchmark code, summarized in Table 4.5. Each benchmark runs 200 allocations followed by 200 frees in a loop, repeated 200 times ($N = 40,000$ total). This structure ensures that iterations operate on a partially fragmented allocator, producing more realistic latency distributions than a single pass would. As in previous benchmarks, IRQs are masked.

Page allocator. `page_malloc` is extremely fast at 126 cycles (78 ns median), reflecting the $O(1)$ free-list lookup when a block of the requested order is directly available. The maximum of 2,270 ns represents the first iteration of each run, where a big block must be split until reaching the requested order 0. `page_free` is more expensive at 971 cycles (781 ns) due to the buddy merging pass, which may traverse up to `MAX_ORDER` to merge freed blocks into larger ones.

Virtual memory allocator. `vmalloc` costs 2,366 cycles (1.97 μ s), dominated by the red-black tree insertion and node allocation from the `vmalloc` container submodule. `vfree` is

Table 4.5: Memory allocator latency (N = 40,000 per operation, IRQs masked, -O2). Cycles represent the mean. Page sizes: 1p = 4 KiB, 2p = 8 KiB, 3p = 12 KiB, 5p = 20 KiB.

Operation	Min	Mean	p50	p95	p99	Max	Cycles
<i>Page allocator</i>							
page_malloc	58 ns	105 ns	78 ns	225 ns	279 ns	2,270 ns	126
page_free	535 ns	809 ns	781 ns	1,056 ns	1,202 ns	3,104 ns	971
<i>Virtual memory allocator</i>							
vmalloc	521 ns	1,971 ns	1,970 ns	3,201 ns	3,315 ns	5,434 ns	2,366
vfree	138 ns	165 ns	163 ns	195 ns	220 ns	2,460 ns	199
<i>Cache allocator</i>							
cache_malloc(8)	160 ns	183 ns	160 ns	160 ns	172 ns	8,150 ns	220
cache_malloc(64)	159 ns	162 ns	159 ns	163 ns	178 ns	2,490 ns	195
cache_malloc(512)	165 ns	380 ns	165 ns	346 ns	6,572 ns	8,965 ns	456
cache_malloc(2048)	164 ns	768 ns	164 ns	8,350 ns	9,056 ns	13,435 ns	922
cache_free(8)	150 ns	169 ns	154 ns	166 ns	176 ns	8,244 ns	204
cache_free(64)	150 ns	150 ns	150 ns	150 ns	150 ns	2,246 ns	181
cache_free(512)	151 ns	292 ns	151 ns	2,025 ns	3,319 ns	7,455 ns	351
cache_free(2048)	150 ns	507 ns	155 ns	3,621 ns	4,035 ns	7,806 ns	608
<i>Kernel allocator (kmalloc/kfree)</i>							
kmalloc(8 bytes)	168 ns	197 ns	168 ns	168 ns	180 ns	8,220 ns	237
kmalloc(1p)	1,965 ns	4,401 ns	4,330 ns	6,494 ns	6,705 ns	14,962 ns	5,282
kmalloc(2p)	2,270 ns	4,627 ns	4,575 ns	6,767 ns	6,955 ns	10,843 ns	5,553
kmalloc(3p)	3,304 ns	8,355 ns	8,235 ns	12,958 ns	13,600 ns	28,666 ns	10,026
kmalloc(5p)	3,722 ns	8,792 ns	8,719 ns	13,465 ns	13,966 ns	36,445 ns	10,551
kfree(8 bytes)	165 ns	186 ns	169 ns	182 ns	191 ns	8,451 ns	224
kfree(1p)	1,609 ns	1,954 ns	1,898 ns	2,418 ns	2,697 ns	9,297 ns	2,345
kfree(2p)	1,762 ns	2,191 ns	2,130 ns	2,480 ns	2,707 ns	5,507 ns	2,630
kfree(3p)	2,512 ns	3,305 ns	3,286 ns	3,655 ns	4,208 ns	11,807 ns	3,966
kfree(5p)	2,829 ns	3,770 ns	3,788 ns	4,398 ns	4,734 ns	16,337 ns	4,524

significantly cheaper at 199 cycles (163 ns) when a `vma11oc_token` is provided, as it bypasses the $O(\log n)$ search and directly frees the node.

Cache allocator. The cache allocator shows a clear two-mode behavior across all size classes. The common call, finding a free slot in the current arena via bitfield scan costs 160–165 ns regardless of size class, confirming the $O(1)$ allocation. The high p99 values for larger classes (e.g. 6.5 μ s for 512-byte, 9 μ s for 2048-byte) correspond to arena fillings, where a new arena must be allocated from `raw_kma11oc`.

Kernel allocator. `kma11oc(8)` routes through the cache allocator and almost matches its cost at 237 cycles, with a very small overhead. Page-sized allocations go through `raw_kma11oc`, combining `page_ma11oc`, `vma11oc`, and an MMU map operation. The jump from 2p (5,553 cycles) to 3p (10,026 cycles) reflects the decomposition of non-power-of-two sizes into multiple `page_ma11oc` calls: 3 pages decompose into 2+1, relations and two MMU map calls instead of one. `kfree` follows the same pattern and has lower cost, as MMU unmapping and buddy merging are cheaper than the combined alloc path.

In summary, the cache allocator and `page_ma11oc` provide the fastest paths at under 200 ns in the common case. Large `kma11oc` allocations are the most expensive and scale with the number of buddy blocks required.

These results were mostly expected, however the efficiency of `cache_ma11oc` was expected to be worse, as it performs extremely well.

4.6 Experiments Conclusions

The results obtained across all benchmarks are surprisingly performant. While the benchmarks are designed to measure hot paths with instruction and data caches warm, even considering this, the measured latencies are considerably lower than initially expected.

The metrics reflect how effectively GCC [12] at `-O2` optimizes the kernel’s C code, collapsing critical paths such as the bitfield scan in the cache allocator or the free-list lookup in the buddy allocator into very efficient instructions.

It should be noted that real-world performance under cold instruction paths would produce higher latencies than the ones reported here. Nevertheless, the results prove solid performance baseline and confirm that the design decisions taken throughout the project result in efficient kernel operations.

Conclusions

THIS chapter summarizes the work done throughout this project, evaluates the objectives defined in Chapter 1, summarizes on the technical lessons learned during development, and defines directions for future work.

Summary

This project intended to design and implement a multithreaded, multicore operating system kernel for the Armv8-A architecture from scratch, running on real hardware. The result is a functional monolithic kernel targeting the AArch64 execution state of the NXP i.MX 8M Plus SoC, written primarily in C with Assembly used where required. Real userspace programs have been compiled and executed successfully.

The kernel boots from the state left by U-Boot, sets up a valid C execution environment in Assembly, and initializes all subsystems before activating secondary cores and entering the scheduling loop. From that point, every core works independently on its own runqueue, executing user processes that live under virtual memory isolation enforced by the MMU.

Memory management is handled by different allocators built in layers. A physical page allocator based on the buddy system, a virtual memory allocator, a cache allocator for fixed-size objects, and a general-purpose kernel memory allocator on top of the previous ones. Userspace memory is managed with per-process virtual address space structures, with `mmap` and `munmap` available to user programs via their own system call interfaces. Userspace physical memory assignments are deferred, and only committed when the user makes an attempt to read or write in the reserved virtual regions.

The scheduler is preemptive and multicore-aware, distributing threads across cores using a load balancing algorithm that ensures that any given task will have their threads divided among cores as perfectly as possible. A good global balance is obtained and has been validated through independent algorithm tests, showing an average global imbalance of approximately

3% for 100 threads.

Hardware is abstracted through a device registration system that defines a common interface per device class, making the rest of the kernel agnostic of platform-specific details. The I/O subsystem is fully interrupt-driven, avoiding polling or busy-waiting for peripherals, and exposes standard file descriptors to userspace through a minimal [VFS](#) layer.

Achieved Objectives

The objectives defined at the start of the project have been covered as follows:

- **Boot and initialization:** fully implemented. The kernel boot sequence is written in Assembly and correctly sets up the C execution environment from the state provided by U-Boot including the kernel boot sequence and initializing all subsystems in the correct order across both the primary and secondary cores.
- **Exception and interrupt handling:** fully implemented. The exception [Vector Table](#) is configured and exceptions are handled through the exception handler and the different subhandlers assigned to it. The drivers for the [GICv3](#) interrupt controller have been implemented, and is handled through a [Hardware Abstraction Layer](#).
- **Virtual memory:** fully implemented. The MMU is configured with multi-level page tables, providing kernel and per-process address space isolation. The kernel relocation to the high address space and dynamic page table management for user processes are both functional.
- **Memory management:** fully implemented. The layered allocators cover physical and virtual page management, and higher level allocators link them together to offer different sized dynamic allocations both for users and kernel space.
- **Process and thread execution:** fully implemented. User space execution runs at [EL0](#), with [context switching](#) between kernel and user space handled through the exception handler. The system call interface is functional and exposes a basic set of operations for user programs to interact with the kernel.
- **Multicore scheduler:** fully implemented. The preemptive scheduler distributes threads across all available cores and maintains the per-task and global balance.
- **Basic drivers:** fully implemented. Platform-specific drivers for [UART](#), timers, and the interrupt controller are implemented behind a [Hardware Abstraction Layer \(HAL\)](#) that keeps the kernel subsystems portable across Armv8-A platforms.

- **Performance analysis:** addressed in Chapter 4, [context switch](#) latency, system call overhead, and memory allocation costs are measured on real hardware and analyzed in relation to the design decisions taken throughout the project.

Lessons Learned

Developing this project has provided a bigger understanding of the Armv8-A architecture, well beyond what reading alone could achieve. Actions like the MMU implementation from scratch, working directly against the Architecture Reference Manual, required understanding how the architecture is expected to work, has taught me how the hardware works underneath, and how the hardware-software boundary works.

At the implementation level, the project has been a practical teacher in low-level programming. Bit manipulation, field extraction, and alignment tricks have been learnt. The data structures implemented throughout the kernel have also helped me understand what and how efficient data structures should be written. Code organization has also improved considerably, with the separation between different modules and subsystems, ability I will carry over following projects. The driver specific code was also a challenge, as understanding the different MMIO registers and peripheral behavior was required.

The early boot stage, before MMU enablement, was one of the hardest aspects of the project. Code executing in this stage runs under hard constraints, and a single incorrect assumption produces a silent failure with no useful diagnostics. Debugging it required working extensively with GDB, reasoning about machine state at the instruction level, which has produced a significantly deeper understanding of both the debugger and the machine.

Future Work

Several features remain open for future development of this kernel.

The most immediate easily implemented feature is thread sleeping, which is defined in the scheduler but not yet implemented. Supporting `THREAD_SLEEPING` would enable timer-based blocking and build the base for more complex features like blocking I/O.

The [VFS](#) layer is currently minimal. File descriptors are set at compile time via `vfs_bind()` and `vfs_open()` is not yet implemented. A full filesystem, for example implemented over FAT32, exFAT, or even a simple in-memory one, would allow user programs to open and close files dynamically and would make the system call interface significantly more useful.

The [Hardware Abstraction Layer](#) was designed from the start to support portability across Armv8-A platforms. Porting the kernel to a second board, such as the Raspberry Pi 5, would validate this design in practice and identify any remaining platform dependencies in the kernel

code.

Finally, expanding the driver set to cover additional peripherals would allow for further systems programming experimentation.

Personal Note

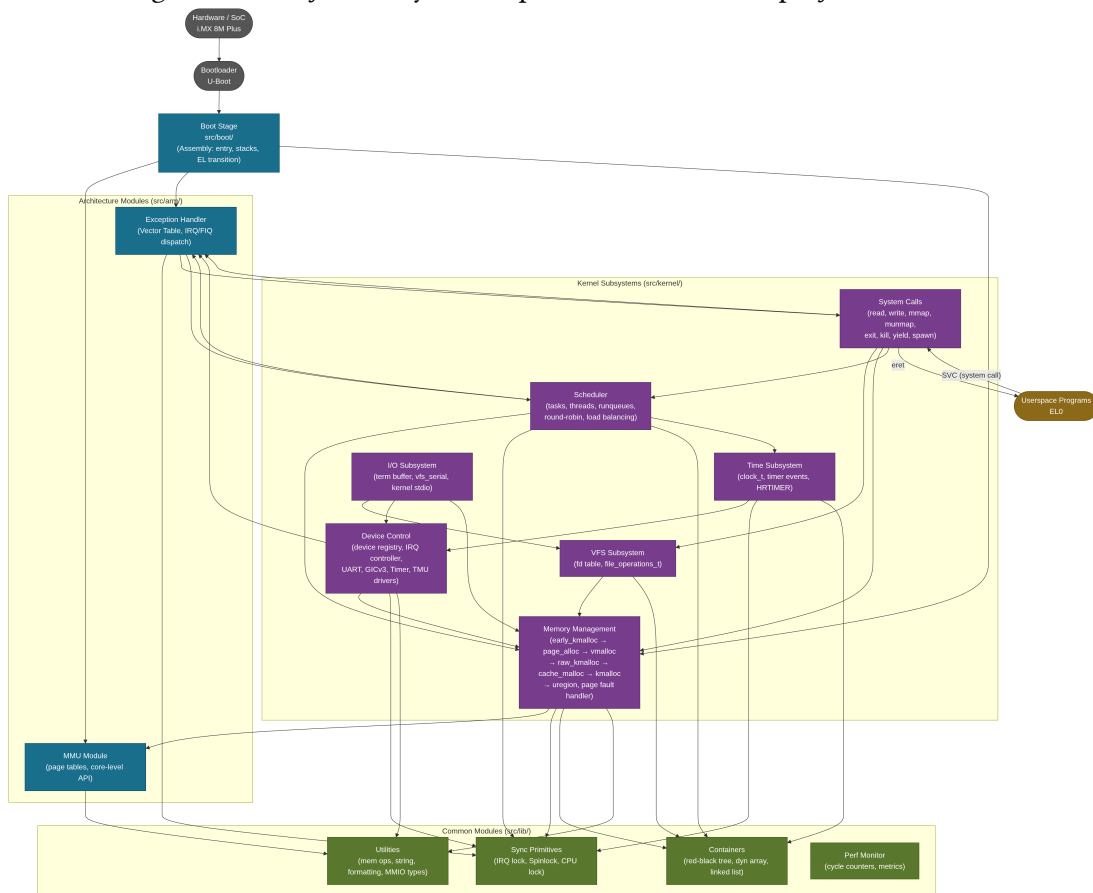
This project started with a personal motivation beyond the academic one. My background is in business, a field unrelated to computer science. When I accidentally discovered programming at the third year of my university degree, I knew it was what I wanted to do, and this master's degree has been the way to formalize that change. This project was, in that context, a personal challenge and a way to prove to myself that coming from outside the field, I was also capable of learning and building a big technical project.

This project holds a special place among everything I have developed. It has been, without a doubt, the hardest, most technically demanding, and the most enjoyable project I have worked on.

Appendix A

Figures

Figure A.1: Project subsystem dependencies related to project structure



Appendix B

Listings

Listing B.1: ectx_store and ectx_load assembly macros

```
#ifndef ECTX_IMPL
.error "this file should not be imported directly, import <arm/exceptions/ctx.h> instead"
#endif

.equ ECTX_SIZE,          ARM_STRUCT_ECTX_SIZE

.equ ECTX_OFFS_SP,       ARM_STRUCT_ECTX_OFFS_SP_ELX
.equ ECTX_OFFS_FPCR,     ARM_STRUCT_ECTX_OFFS_FPCR
.equ ECTX_OFFS_FPSR,     ARM_STRUCT_ECTX_OFFS_FPSR
.equ ECTX_OFFS_ELR,      ARM_STRUCT_ECTX_OFFS_ELR
.equ ECTX_OFFS_SPSR,     ARM_STRUCT_ECTX_OFFS_SPSR

.equ ECTX_OFFS_X,        ARM_STRUCT_ECTX_OFFS_X
.equ ECTX_OFFS_V,        ARM_STRUCT_ECTX_OFFS_V

.equ ECTX_SIZEOF_X,      ARM_STRUCT_ECTX_SIZEOF_X
.equ ECTX_SIZEOF_V,      ARM_STRUCT_ECTX_SIZEOF_V

// typedef struct {
//     sysreg_t fpcr;
//     sysreg_t fpsr;
//     sysreg_t elr;
//     sysreg_t spsr;

//     sp_t sp_elx;

//     gpr_t x[XREG_COUNT];    // x0-x30
//     vreg128_t v[VREG_COUNT]; // v0-v31
// } arm_ectx;

.macro ectx_store
    // store xregs
    sub sp, sp, #ECTX_SIZE

    str x0,          [sp, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 0)]
    stp x1, x2,      [sp, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 1)]
```

```

    stp x3, x4,    [sp, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 3)]
    stp x5, x6,    [sp, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 5)]
    stp x7, x8,    [sp, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 7)]
    stp x9, x10,   [sp, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 9)]
    stp x11, x12,  [sp, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 11)]
    stp x13, x14,  [sp, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 13)]
    stp x15, x16,  [sp, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 15)]
    stp x17, x18,  [sp, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 17)]
    stp x19, x20,  [sp, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 19)]
    stp x21, x22,  [sp, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 21)]
    stp x23, x24,  [sp, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 23)]
    stp x25, x26,  [sp, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 25)]
    stp x27, x28,  [sp, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 27)]
    stp x29, x30,  [sp, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 29)]

    // store sysregs
    mrs x0, fpcr
    mrs x1, fpsr
    stp x0, x1,    [sp, #ECTX_OFFS_FPCR]

    mrs x2, elr_el1
    mrs x3, spsr_el1
    stp x2, x3,    [sp, #ECTX_OFFS_ELR]

    // store sp
#ifdef DEBUG
    // check that we are using sp_el1
    mrs x0, spsel
    cbnz x0, 1f
0: wfi
    b 0b
1:
#endif

    tbnz x3, #0, 2f // goto store sp_el1 if bit[0] of spsr == 1

    // sp_el0
    mrs x4, sp_el0
    b 3f // store

2: // sp_el1
    add x4, sp, #ECTX_SIZE

3: // store sp_elx
    str x4, [sp, #ECTX_OFFS_SP]

    // store vregs
    add x0, sp, #ECTX_OFFS_V

    st1 {v0.16b, v1.16b}, [x0], #(2 * ECTX_SIZEOF_V)
    st1 {v2.16b, v3.16b}, [x0], #(2 * ECTX_SIZEOF_V)
    st1 {v4.16b, v5.16b}, [x0], #(2 * ECTX_SIZEOF_V)
    st1 {v6.16b, v7.16b}, [x0], #(2 * ECTX_SIZEOF_V)

```

```

    st1 {v8.16b, v9.16b}, [x0], #(2 * ECTX_SIZEOF_V)
    st1 {v10.16b, v11.16b}, [x0], #(2 * ECTX_SIZEOF_V)
    st1 {v12.16b, v13.16b}, [x0], #(2 * ECTX_SIZEOF_V)
    st1 {v14.16b, v15.16b}, [x0], #(2 * ECTX_SIZEOF_V)
    st1 {v16.16b, v17.16b}, [x0], #(2 * ECTX_SIZEOF_V)
    st1 {v18.16b, v19.16b}, [x0], #(2 * ECTX_SIZEOF_V)
    st1 {v20.16b, v21.16b}, [x0], #(2 * ECTX_SIZEOF_V)
    st1 {v22.16b, v23.16b}, [x0], #(2 * ECTX_SIZEOF_V)
    st1 {v24.16b, v25.16b}, [x0], #(2 * ECTX_SIZEOF_V)
    st1 {v26.16b, v27.16b}, [x0], #(2 * ECTX_SIZEOF_V)
    st1 {v28.16b, v29.16b}, [x0], #(2 * ECTX_SIZEOF_V)
    st1 {v30.16b, v31.16b}, [x0], #(2 * ECTX_SIZEOF_V)

    mov x0, sp
.endm

/// loads the ectx struct from the sp or from x0. The sp or x0 should point to the struct
.macro ectx_load reg_pt
.ifc \reg_pt, sp
.else
    .ifc \reg_pt, x0
    .else
        .error "reg_pt must be either sp or x0"
    .endif
.endif

    // load sysregs
    ldp x1, x2, [\reg_pt, #ECTX_OFFS_FPCR]
    msr fpcr, x1
    msr fpsr, x2

    ldp x3, x4, [\reg_pt, #ECTX_OFFS_ELR]
    msr elr_el1, x3
    msr spsr_el1, x4

    // load sp
    tbnz x4, #0, 0f

    ldr x5, [\reg_pt, #ECTX_OFFS_SP]
    msr sp_el0, x5

.ifc \reg_pt, x0
    b 2f
0:
    ldr x5, [\reg_pt, #ECTX_OFFS_SP]
    mov sp, x5
.else
    b 2f
0:
#   ifdef DEBUG
    // if sp_el1 is selected, the stack should already be the correct stack when adding the size of
    // the ectx struct size
    // so just ignore it (in DEBUG assert it)

```

```

    ldr x5, [\reg_pt, #ECTX_OFFS_SP]
    add x6, sp, #ECTX_SIZE
    cmp x5, x6
    b.eq 2f

1: wfi
    b 1b
# endif
.endif

2: // load vregs
    add x1, \reg_pt, #ECTX_OFFS_V

    ldr {v0.16b, v1.16b}, [x1], #(2 * ECTX_SIZEOF_V)
    ldr {v2.16b, v3.16b}, [x1], #(2 * ECTX_SIZEOF_V)
    ldr {v4.16b, v5.16b}, [x1], #(2 * ECTX_SIZEOF_V)
    ldr {v6.16b, v7.16b}, [x1], #(2 * ECTX_SIZEOF_V)
    ldr {v8.16b, v9.16b}, [x1], #(2 * ECTX_SIZEOF_V)
    ldr {v10.16b, v11.16b}, [x1], #(2 * ECTX_SIZEOF_V)
    ldr {v12.16b, v13.16b}, [x1], #(2 * ECTX_SIZEOF_V)
    ldr {v14.16b, v15.16b}, [x1], #(2 * ECTX_SIZEOF_V)
    ldr {v16.16b, v17.16b}, [x1], #(2 * ECTX_SIZEOF_V)
    ldr {v18.16b, v19.16b}, [x1], #(2 * ECTX_SIZEOF_V)
    ldr {v20.16b, v21.16b}, [x1], #(2 * ECTX_SIZEOF_V)
    ldr {v22.16b, v23.16b}, [x1], #(2 * ECTX_SIZEOF_V)
    ldr {v24.16b, v25.16b}, [x1], #(2 * ECTX_SIZEOF_V)
    ldr {v26.16b, v27.16b}, [x1], #(2 * ECTX_SIZEOF_V)
    ldr {v28.16b, v29.16b}, [x1], #(2 * ECTX_SIZEOF_V)
    ldr {v30.16b, v31.16b}, [x1], #(2 * ECTX_SIZEOF_V)

    // load xregs
    ldp x1, x2, [\reg_pt, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 1)]
    ldp x3, x4, [\reg_pt, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 3)]
    ldp x5, x6, [\reg_pt, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 5)]
    ldp x7, x8, [\reg_pt, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 7)]
    ldp x9, x10, [\reg_pt, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 9)]
    ldp x11, x12, [\reg_pt, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 11)]
    ldp x13, x14, [\reg_pt, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 13)]
    ldp x15, x16, [\reg_pt, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 15)]
    ldp x17, x18, [\reg_pt, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 17)]
    ldp x19, x20, [\reg_pt, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 19)]
    ldp x21, x22, [\reg_pt, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 21)]
    ldp x23, x24, [\reg_pt, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 23)]
    ldp x25, x26, [\reg_pt, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 25)]
    ldp x27, x28, [\reg_pt, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 27)]
    ldp x29, x30, [\reg_pt, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 29)]

    ldr x0, [\reg_pt, #(ECTX_OFFS_X + ECTX_SIZEOF_X * 0)]

.ifc \reg_pt, sp
    add sp, sp, #ECTX_SIZE
.endif
.endm

```

Listing B.2: Kernel exception vector configuration and exception context store/load

```
.global _el1_init_vector
_el1_init_vector:
    adr    x0, _el1_vector_table
    msr    VBAR_EL1, x0
    isb
    ret

.macro EL1_CALL_WITH_CONTEXT label
    ectx_store
    bl \label
    ectx_load sp
    eret
.endm

.macro EL1_CALL_WITH_CONTEXT label
    ectx_store
    bl \label
    ectx_load sp
    eret
.endm

/* ----- */
/*           Exception handlers           */
/* ----- */

.section .text
_el1_cur_sp0_sync:
    EL1_CALL_WITH_CONTEXT el1_cur_sp0_sync_handler

_el1_cur_sp0_irq:
    EL1_CALL_WITH_CONTEXT el1_cur_sp0_irq_handler

_el1_cur_sp0_fiq:
    EL1_CALL_WITH_CONTEXT el1_cur_sp0_fiq_handler

_el1_cur_sp0_serror:
    EL1_CALL_WITH_CONTEXT el1_cur_sp0_serror_handler

_el1_cur_spx_sync:
    EL1_CALL_WITH_CONTEXT el1_cur_spx_sync_handler

_el1_cur_spx_irq:
    EL1_CALL_WITH_CONTEXT el1_cur_spx_irq_handler

_el1_cur_spx_fiq:
    EL1_CALL_WITH_CONTEXT el1_cur_spx_fiq_handler

_el1_cur_spx_serror:
    EL1_CALL_WITH_CONTEXT el1_cur_spx_serror_handler

_el1_low_a64_sync:
    EL1_CALL_WITH_CONTEXT el1_low_a64_sync_handler
```

```

_el1_low_a64_irq:
    EL1_CALL_WITH_CONTEXT el1_low_a64_irq_handler

_el1_low_a64_fiq:
    EL1_CALL_WITH_CONTEXT el1_low_a64_fiq_handler

_el1_low_a64_serror:
    EL1_CALL_WITH_CONTEXT el1_low_a64_serror_handler

_el1_low_a32_sync:
    EL1_CALL_WITH_CONTEXT el1_low_a32_sync_handler

_el1_low_a32_irq:
    EL1_CALL_WITH_CONTEXT el1_low_a32_irq_handler

_el1_low_a32_fiq:
    EL1_CALL_WITH_CONTEXT el1_low_a32_fiq_handler

_el1_low_a32_serror:
    EL1_CALL_WITH_CONTEXT el1_low_a32_serror_handler

/* ----- */
/*          Vector table          */
/* ----- */

.macro SET_VENTRY label
.align 7
    b \label
.endm

.section .vectors, "ax"
.global _el1_vector_table
.align 11
_el1_vector_table:
// 0..3: Current EL, SP0
    SET_VENTRY _el1_cur_sp0_sync
    SET_VENTRY _el1_cur_sp0_irq
    SET_VENTRY _el1_cur_sp0_fiq
    SET_VENTRY _el1_cur_sp0_serror
// 4..7: Current EL, SPx
    SET_VENTRY _el1_cur_spx_sync
    SET_VENTRY _el1_cur_spx_irq
    SET_VENTRY _el1_cur_spx_fiq
    SET_VENTRY _el1_cur_spx_serror
// 8..11: Lower EL, AArch64
    SET_VENTRY _el1_low_a64_sync
    SET_VENTRY _el1_low_a64_irq
    SET_VENTRY _el1_low_a64_fiq
    SET_VENTRY _el1_low_a64_serror
// 12..15: Lower EL, AArch32
    SET_VENTRY _el1_low_a32_sync
    SET_VENTRY _el1_low_a32_irq
    SET_VENTRY _el1_low_a32_fiq

```

```
SET_VENTRY _el1_low_a32_serror
```

Listing B.3: Scheduler load and store functions

```
void scheduler_ectx_store(arm_ctx_t* ectx /* ectx stored at the stack, its value will be restored
after the exception handling is finished */)
{
    cpuid_t cpuid = get_cpuid();

    cpulock_acquire(&runqueue[cpuid].lock);

    // restore into sp_el0 the active thread
    thread_t* curr = runqueue[get_cpuid()].current_thread;
    DEBUG_ASSERT(curr == NULL || is_kva_ptr(curr));

    if (curr) {
        curr->ctx = *ectx;
    }

    sysreg_write(sp_el0, curr);

    arm_exceptions_enable_all();
}

void scheduler_ectx_load(arm_ctx_t* ectx)
{
    cpuid_t cpuid = get_cpuid();

    arm_exceptions_disable_all();

    thread_t* curr = schedule_is_required(cpuid)
        // schedule_required == true:
        // schedule a new thread
        ? runqueue_schedule()

        // schedule_required ==
        // false: continue with the
        // current thread
        : get_current_thread();

    runqueue[cpuid].current_thread = curr;

    if (unlikely(!curr))
        return scheduler_loop_cpu_exit();

    *ectx = curr->ctx;
    set_thread_mapping(curr);

    cpulock_release(&runqueue[cpuid].lock);
}
```

Listing B.4: runqueue_schedule function implementation

```

static thread_t* runqueue_schedule()
{
    const cpuid_t    cpuid = get_cpuid();
    thread_t* const  curr  = get_current_thread();
    thread_node* const node = node_from_thread(curr);

    ASSERT(curr);

    deferT(kvec(thread*), free_threads) to_free = kvec_new(thread_node*);

    DEBUG_ASSERT(curr && curr->sched_cpu == cpuid);

    // at this point the current thread is the thread that was being
    // executed by the core, its state should still be RUNNING, although it
    // could also be DEAD (killed by other thread) or SLEEPING

    /* --- handle the current thread's state --- */
    thread_state expected = THREAD_RUNNING; // most probable state is RUNNING,
                                           // then SLEEPING, then DEAD

    if (unlikely(!atomic_compare_exchange_strong(
        &curr->state,
        &expected,
        THREAD_READY))) {
        switch (expected) {
            case THREAD_SLEEPING:
                PANIC("TODO: implement THREAD_SLEEPING");
                break;
            case THREAD_DEAD:
                // we are the runqueue core so its safe to
                // remove the thread from the runqueue
                unqueue_thread(
                    node,
                    cpuid); // node is still valid memory even if unqued because
                            // the freeing is deferred, so the "next" pointer is
                            // valid
                kvec_push(&to_free, &node);
                break;
            default:
                PANIC();
        }
    }
}

if (runqueue[cpuid].list == NULL)
    goto null_return; // No threads remaining in the runqueue

thread_node* selected_node = node;

while (true) {
    // select next thread in the queue
    selected_node = selected_node->next;
    expected      = THREAD_READY;
}

```



```
// check that the thread's owner is not dying
task_state owner_state = atomic_load(&selected_node->th.owner->state);

DEBUG_ASSERT(
    owner_state != TASK_DEAD,
    "no threads of a dead task should be in the runqueue");

if (unlikely(owner_state != TASK_ALIVE)) {
    unqueue_thread(selected_node, cpuid);
    kvec_push(&to_free, &selected_node);

    if (runqueue[cpuid].list == NULL)
        goto null_return;

    continue;
}

bool scheduled_as_ready = atomic_compare_exchange_strong(
    &selected_node->th.state,
    &expected,
    THREAD_RUNNING);

if (likely(scheduled_as_ready))
    break; // exit the loop

// The thread was not READY, check the state it was in
switch (expected) {
    case THREAD_NEW: {
        // NEW threads cannot be executed as they are still
        // being configured, so just avoid it
    } break;

    case THREAD_DEAD: {
        // we own the lock and are the runqueue core so its safe
        // to remove the thread from the runqueue
        unqueue_thread(selected_node, cpuid);
        kvec_push(&to_free, &selected_node);

        if (runqueue[cpuid].list == NULL)
            goto null_return; // No threads remaining in the
                               // runqueue, set by unqueue_thread()
    } break;

    case THREAD_RUNNING: {
        PANIC(
            "no thread should be running because we are "
            "scheduling");
    }

    case THREAD_SLEEPING: {
        PANIC("TODO: implement THREAD_SLEEPING");
    }
}
```

```

        default: {
            PANIC();
        }
    }
}

runqueue[cpuid].preemptive_event = timer_create_event_delta(
    HRTIMER,
    event_preemptive_scheduling,
    NULL,
    atomic_load(&runqueue[cpuid].preemptive_duration_microsec) * 1000);

set_current_thread(&selected_node->th);
return &selected_node->th;

PANIC("unreachable");

null_return: {
    set_current_thread(NULL);
    return NULL;
}
}

```

Listing B.5: Device control public API exposed through device.h

```

typedef struct {
    uintptr_t    base;
    void** const state;
} driver_handle_t;

typedef enum {
    DEVICE_CLASS_GENERIC = 0,
    DEVICE_CLASS_IRQ_CTRL,
    DEVICE_CLASS_SERIAL,
    DEVICE_CLASS_TIMER,
    DEVICE_CLASS_CLOCKSOURCE,
    DEVICE_CLASS_THERMAL_SENSOR,

    DEVICE_CLASS_COUNT,
} device_class_t;

typedef struct {
    uint64_t    uid;
    const char* name;
    uint8_t     rank;
    device_class_t class_id;
    uintptr_t    base_pa;
    void*        driver_state;
    const void*  driver_ops;
} device_t;

void device_ctrl_init();
void device_register(

```

```
const char*    name,
device_class_t class_id,
uint8_t       rank,
puintptr_t    base,
const void*    driver_ops);
const device_t* device_get_by_name(device_class_t class_id, const char* name);
const device_t* device_get_by_uid(device_class_t class_id, uint64_t uid);
const device_t* device_get_primary(device_class_t class_id);
[[gnu::always_inline]] static inline driver_handle_t device_get_driver_handle(
    const device_t* device)
{
    return (driver_handle_t) {
        .base = mm_kernel_is_relocated() ? kpa_to_kva(device->base_pa)
                                           : device->base_pa,
        .state = (void**)&device->driver_state,
    };
}
```

Listing B.6: Context switch experiment userspace tested code

```
#![no_std]

use core::sync::atomic::AtomicU64;

use stl::{
    lock::Spinlock,
    printf,
    stdlib::yield_cpu,
    thread::{thread_create, thread_kill},
};

extern crate stl;

const THREADS: usize = 200;
static SPINLOCK: Spinlock<i32> = Spinlock::new(0);
static FINISHED: AtomicU64 = AtomicU64::new(0);

#[unsafe(no_mangle)]
unsafe extern "C" fn secondary_entry(thid: u64, arg: u64) {
    LOCK.locked(|_| printf!("Thread {} spawned!", arg));

    FINISHED.fetch_add(1, core::sync::atomic::Ordering::Release);

    thread_kill(thid).expect("could not kill self thread");
}

#[unsafe(no_mangle)]
extern "C" fn main() -> i32 {
    for i in 1..=THREADS {
        thread_create(secondary_entry, i as u64).unwrap();
    }

    while FINISHED.load(core::sync::atomic::Ordering::Relaxed) != THREADS as u64 {
```

```

        yield_cpu();
    }

    return 0;
}

```

Listing B.7: Syscall benchmarking userspace code

```

#![no_std]

use core::ffi::c_void;

use stl::{
    stdio::{print, read},
    stdlib::yield_cpu,
    syscall::{MmapFlags, MmapProt, syscall_kill, syscall_mmap, syscall_munmap, syscall_spawn},
};

extern crate stl;

const N: usize = 20_000;

#[unsafe(no_mangle)]
extern "C" fn main() -> i32 {
    bench_yield(N);
    bench_mmap_munmap(N);
    bench_spawn_kill(N);
    bench_write(N);
    bench_read(N, false);
    0
}

const PAGES: usize = 1;

fn bench_yield(n: usize) {
    for _ in 0..n {
        yield_cpu(); // calls syscall yield
    }
}

fn bench_mmap_munmap(n: usize) {
    for _ in 0..n {
        let ptr = syscall_mmap(
            0 as *mut c_void,
            PAGES * 4096,
            MmapProt::new(true, true, false), /* read && write */
            MmapFlags::new(true, false),    /* anonymous && !fixed */
            0,                               /* fd ignored (anonymous) */
            0,                               /* offset */
        )
        .expect("mmap should work with this parameters");

        // commit on demand test
    }
}

```

```
    for i in 0..PAGES {
        unsafe {
            let page_ptr = ptr.add(i * 4096) as *mut u8;
            core::ptr::write_volatile(page_ptr, 0xFF); // write 1 byte
        }
    }

    syscall_munmap(ptr, PAGES * 4096).unwrap();
}

unsafe extern "C" fn nop_thread_entry(_thid: u64, _arg: u64) {
    loop {}
}

fn bench_spawn_kill(n: usize) {
    for _ in 0..n {
        let tid = syscall_spawn(nop_thread_entry, 0).unwrap();
        syscall_kill(tid).unwrap();
    }
}

fn bench_write(n: usize) {
    for _ in 0..n {
        print("Hello from userspace!\n\r");
    }
}

fn bench_read(n: usize, with_data: bool) {
    let mut read_buf: [u8; 1000] = [0; 1000];

    for _ in 0..n {
        loop {
            let slice = read(&mut read_buf).expect("syscall should work");

            if with_data && slice.is_empty() {
                continue;
            }

            break;
        }
    }
}
```

Listing B.8: Syscall benchmarking kernel_entry function

```
noreturn void kernel_entry() // build configured for this benchmark with only 1 core available, no
    smp_init()
{
    if (get_cpuid() == 0) {
        if (!mm_kernel_is_relocated())
            kernel_early_init();
        else
```

```

        kernel_init();
    }

    printf("Hello from the %s!\n\r", "kernel");

    elf_load_result elf_res;
    uintptr_t        sysc_benchmarking_entry;

    task_t* sysc_benchmarking = task_new("bench syscalls");

    elf_res = elf_load(
        sysc_benchmarking,
        EMBEDDED_BINARY(sysc_benchmarking_elf),
        EMBEDDED_BINARY_SIZE(sysc_benchmarking_elf),
        &sysc_benchmarking_entry);
    ASSERT(elf_res == ELF_LOAD_OK);

    schedule_ready_thread(sysc_benchmarking, sysc_benchmarking_entry);

    scheduler_loop_cpu_enter();

    print("\n\rKERNEL FINISHED CORRECTLY\n\r");
    ksleep(
        60e9); // let irq driven I/O finish the prints before showing the results

    print_sysc_metrics();

    loop asm volatile("wfi");
}

```

Listing B.9: Syscall benchmarking logs

```

[CPU0][exit] N=1
  wall   min=0s.0ms.2us.500ns
         max=0s.0ms.2us.500ns
         avg=0s.0ms.2us.500ns
         std=0s.0ms.0us.0ns
         iqr=0s.0ms.0us.0ns
         p25=0s.0ms.2us.500ns
         p50=0s.0ms.2us.500ns
         p75=0s.0ms.2us.500ns
         p90=0s.0ms.2us.500ns
         p95=0s.0ms.2us.500ns
         p99=0s.0ms.2us.500ns
  cpu    min=0s.0ms.2us.527ns
         max=0s.0ms.2us.527ns
         avg=0s.0ms.2us.527ns
         std=0s.0ms.0us.0ns
         iqr=0s.0ms.0us.0ns
         p25=0s.0ms.2us.527ns
         p50=0s.0ms.2us.527ns
         p75=0s.0ms.2us.527ns
         p90=0s.0ms.2us.527ns

```

```
        p95=0s.0ms.2us.527ns
        p99=0s.0ms.2us.527ns
cycles  min=3033 max=3033 avg=3033
active  min=101% max=101% avg=101%
jitter  min=0s.0ms.0us.0ns
        max=0s.0ms.0us.0ns
        avg=0s.0ms.0us.0ns
exceptions total=0
samples_with_exceptions=0/1
[CPU0][read] N=20000
wall    min=0s.0ms.1us.0ns
        max=0s.0ms.4us.625ns
        avg=0s.0ms.1us.151ns
        std=0s.0ms.0us.105ns
        iqr=0s.0ms.0us.0ns
        p25=0s.0ms.1us.125ns
        p50=0s.0ms.1us.125ns
        p75=0s.0ms.1us.125ns
        p90=0s.0ms.1us.250ns
        p95=0s.0ms.1us.250ns
        p99=0s.0ms.1us.250ns
cpu      min=0s.0ms.0us.992ns
        max=0s.0ms.4us.542ns
        avg=0s.0ms.1us.151ns
        std=0s.0ms.0us.92ns
        iqr=0s.0ms.0us.41ns
        p25=0s.0ms.1us.129ns
        p50=0s.0ms.1us.150ns
        p75=0s.0ms.1us.170ns
        p90=0s.0ms.1us.186ns
        p95=0s.0ms.1us.196ns
        p99=0s.0ms.1us.214ns
cycles  min=1191 max=5451 avg=1382
active  min=89% max=112% avg=99%
jitter  min=0s.0ms.0us.0ns
        max=0s.0ms.3us.375ns
        avg=0s.0ms.0us.51ns
exceptions total=0
samples_with_exceptions=0/20000
[CPU0][write] N=20000
wall    min=0s.0ms.1us.375ns
        max=0s.0ms.24us.250ns
        avg=0s.0ms.2us.123ns
        std=0s.0ms.0us.856ns
        iqr=0s.0ms.1us.625ns
        p25=0s.0ms.1us.625ns
        p50=0s.0ms.1us.625ns
        p75=0s.0ms.3us.250ns
        p90=0s.0ms.3us.500ns
        p95=0s.0ms.3us.625ns
        p99=0s.0ms.3us.750ns
cpu      min=0s.0ms.1us.466ns
        max=0s.0ms.24us.213ns
```

```

    avg=0s.0ms.2us.123ns
    std=0s.0ms.0us.854ns
    iqr=0s.0ms.1us.679ns
    p25=0s.0ms.1us.615ns
    p50=0s.0ms.1us.644ns
    p75=0s.0ms.3us.294ns
    p90=0s.0ms.3us.526ns
    p95=0s.0ms.3us.599ns
    p99=0s.0ms.3us.803ns
cycles min=1760 max=29056 avg=2548
active min=92% max=108% avg=99%
jitter min=0s.0ms.0us.0ns
    max=0s.0ms.22us.625ns
    avg=0s.0ms.1us.0ns
exceptions total=0
samples_with_exceptions=0/20000
[CPU0][spawn] N=20000
wall   min=0s.0ms.0us.875ns
        max=0s.0ms.3us.875ns
        avg=0s.0ms.1us.74ns
        std=0s.0ms.0us.356ns
        iqr=0s.0ms.0us.0ns
        p25=0s.0ms.1us.0ns
        p50=0s.0ms.1us.0ns
        p75=0s.0ms.1us.0ns
        p90=0s.0ms.1us.125ns
        p95=0s.0ms.1us.125ns
        p99=0s.0ms.3us.0ns
cpu     min=0s.0ms.0us.882ns
        max=0s.0ms.3us.877ns
        avg=0s.0ms.1us.74ns
        std=0s.0ms.0us.353ns
        iqr=0s.0ms.0us.37ns
        p25=0s.0ms.0us.993ns
        p50=0s.0ms.1us.10ns
        p75=0s.0ms.1us.30ns
        p90=0s.0ms.1us.51ns
        p95=0s.0ms.1us.72ns
        p99=0s.0ms.2us.990ns
cycles min=1059 max=4653 avg=1289
active min=89% max=114% avg=99%
jitter min=0s.0ms.0us.0ns
    max=0s.0ms.2us.375ns
    avg=0s.0ms.0us.159ns
exceptions total=0
samples_with_exceptions=0/20000
[CPU0][kill] N=20000
wall   min=0s.0ms.0us.125ns
        max=0s.0ms.2us.250ns
        avg=0s.0ms.0us.214ns
        std=0s.0ms.0us.58ns
        iqr=0s.0ms.0us.125ns
        p25=0s.0ms.0us.125ns

```



```

    p50=0s.0ms.0us.250ns
    p75=0s.0ms.0us.250ns
    p90=0s.0ms.0us.250ns
    p95=0s.0ms.0us.250ns
    p99=0s.0ms.0us.250ns
cpu   min=0s.0ms.0us.162ns
      max=0s.0ms.2us.180ns
      avg=0s.0ms.0us.214ns
      std=0s.0ms.0us.22ns
      iqr=0s.0ms.0us.26ns
      p25=0s.0ms.0us.202ns
      p50=0s.0ms.0us.215ns
      p75=0s.0ms.0us.228ns
      p90=0s.0ms.0us.236ns
      p95=0s.0ms.0us.242ns
      p99=0s.0ms.0us.253ns
cycles min=195 max=2617 avg=258
active  min=65% max=198% avg=109%
jitter  min=0s.0ms.0us.0ns
      max=0s.0ms.2us.125ns
      avg=0s.0ms.0us.50ns
exceptions total=0
samples_with_exceptions=0/20000
[CPU0][yield] N=20000
wall   min=0s.0ms.0us.250ns
      max=0s.0ms.1us.0ns
      avg=0s.0ms.0us.370ns
      std=0s.0ms.0us.23ns
      iqr=0s.0ms.0us.0ns
      p25=0s.0ms.0us.375ns
      p50=0s.0ms.0us.375ns
      p75=0s.0ms.0us.375ns
      p90=0s.0ms.0us.375ns
      p95=0s.0ms.0us.375ns
      p99=0s.0ms.0us.375ns
cpu    min=0s.0ms.0us.370ns
      max=0s.0ms.1us.15ns
      avg=0s.0ms.0us.370ns
      std=0s.0ms.0us.5ns
      iqr=0s.0ms.0us.0ns
      p25=0s.0ms.0us.370ns
      p50=0s.0ms.0us.370ns
      p75=0s.0ms.0us.370ns
      p90=0s.0ms.0us.370ns
      p95=0s.0ms.0us.370ns
      p99=0s.0ms.0us.380ns
cycles min=445 max=1219 avg=445
active  min=76% max=148% avg=99%
jitter  min=0s.0ms.0us.0ns
      max=0s.0ms.0us.500ns
      avg=0s.0ms.0us.8ns
exceptions total=0
samples_with_exceptions=0/20000
```

```

[CPU0][mmap] N=20001
  wall   min=0s.0ms.0us.375ns
         max=0s.0ms.2us.875ns
         avg=0s.0ms.0us.595ns
         std=0s.0ms.0us.90ns
         iqr=0s.0ms.0us.125ns
         p25=0s.0ms.0us.500ns
         p50=0s.0ms.0us.625ns
         p75=0s.0ms.0us.625ns
         p90=0s.0ms.0us.625ns
         p95=0s.0ms.0us.625ns
         p99=0s.0ms.0us.750ns
  cpu    min=0s.0ms.0us.465ns
         max=0s.0ms.2us.854ns
         avg=0s.0ms.0us.594ns
         std=0s.0ms.0us.72ns
         iqr=0s.0ms.0us.43ns
         p25=0s.0ms.0us.567ns
         p50=0s.0ms.0us.586ns
         p75=0s.0ms.0us.610ns
         p90=0s.0ms.0us.638ns
         p95=0s.0ms.0us.665ns
         p99=0s.0ms.0us.748ns
cycles  min=559 max=3425 avg=714
active  min=82% max=125% avg=100%
jitter  min=0s.0ms.0us.0ns
         max=0s.0ms.2us.250ns
         avg=0s.0ms.0us.61ns
exceptions total=0
samples_with_exceptions=0/20001
[CPU0][munmap] N=20000
  wall   min=0s.0ms.3us.750ns
         max=0s.0ms.8us.0ns
         avg=0s.0ms.4us.225ns
         std=0s.0ms.0us.197ns
         iqr=0s.0ms.0us.250ns
         p25=0s.0ms.4us.125ns
         p50=0s.0ms.4us.250ns
         p75=0s.0ms.4us.375ns
         p90=0s.0ms.4us.500ns
         p95=0s.0ms.4us.500ns
         p99=0s.0ms.4us.625ns
  cpu    min=0s.0ms.3us.724ns
         max=0s.0ms.7us.908ns
         avg=0s.0ms.4us.225ns
         std=0s.0ms.0us.190ns
         iqr=0s.0ms.0us.307ns
         p25=0s.0ms.4us.67ns
         p50=0s.0ms.4us.228ns
         p75=0s.0ms.4us.374ns
         p90=0s.0ms.4us.459ns
         p95=0s.0ms.4us.503ns
         p99=0s.0ms.4us.597ns

```

```
cycles min=4469 max=9490 avg=5070
active min=96% max=103% avg=99%
jitter min=0s.0ms.0us.0ns
      max=0s.0ms.3us.875ns
      avg=0s.0ms.0us.112ns
exceptions total=0
samples_with_exceptions=0/20000
[CPU0][commit_on_demand] N=20001
wall  min=0s.0ms.12us.125ns
      max=0s.0ms.33us.125ns
      avg=0s.0ms.12us.741ns
      std=0s.0ms.0us.287ns
      iqr=0s.0ms.0us.250ns
      p25=0s.0ms.12us.625ns
      p50=0s.0ms.12us.750ns
      p75=0s.0ms.12us.875ns
      p90=0s.0ms.12us.875ns
      p95=0s.0ms.13us.0ns
      p99=0s.0ms.13us.375ns
cpu    min=0s.0ms.12us.140ns
      max=0s.0ms.33us.138ns
      avg=0s.0ms.12us.742ns
      std=0s.0ms.0us.282ns
      iqr=0s.0ms.0us.179ns
      p25=0s.0ms.12us.630ns
      p50=0s.0ms.12us.705ns
      p75=0s.0ms.12us.809ns
      p90=0s.0ms.12us.931ns
      p95=0s.0ms.12us.996ns
      p99=0s.0ms.13us.405ns
cycles min=14568 max=39766 avg=15290
active min=99% max=101% avg=99%
jitter min=0s.0ms.0us.0ns
      max=0s.0ms.12us.625ns
      avg=0s.0ms.0us.133ns
exceptions total=0
samples_with_exceptions=0/20001
```

Listing B.10: Allocator benchmarking logs

```
[CPU0][page_malloc] N=40000
wall  min=0s.0ms.0us.0ns
      max=0s.0ms.2us.250ns
      avg=0s.0ms.0us.105ns
      std=0s.0ms.0us.82ns
      iqr=0s.0ms.0us.125ns
      p25=0s.0ms.0us.0ns
      p50=0s.0ms.0us.125ns
      p75=0s.0ms.0us.125ns
      p90=0s.0ms.0us.250ns
      p95=0s.0ms.0us.250ns
      p99=0s.0ms.0us.250ns
cpu    min=0s.0ms.0us.58ns
```

```

max=0s.0ms.2us.270ns
avg=0s.0ms.0us.105ns
std=0s.0ms.0us.61ns
iqr=0s.0ms.0us.57ns
p25=0s.0ms.0us.66ns
p50=0s.0ms.0us.78ns
p75=0s.0ms.0us.123ns
p90=0s.0ms.0us.189ns
p95=0s.0ms.0us.225ns
p99=0s.0ms.0us.279ns
cycles min=70 max=2724 avg=126
active min=0% max=198% avg=58%
jitter min=0s.0ms.0us.0ns
max=0s.0ms.2us.250ns
avg=0s.0ms.0us.99ns
exceptions total=0
samples_with_exceptions=0/40000
[CPU0][page_free] N=40000
wall min=0s.0ms.0us.500ns
max=0s.0ms.3us.125ns
avg=0s.0ms.0us.810ns
std=0s.0ms.0us.142ns
iqr=0s.0ms.0us.125ns
p25=0s.0ms.0us.750ns
p50=0s.0ms.0us.750ns
p75=0s.0ms.0us.875ns
p90=0s.0ms.1us.0ns
p95=0s.0ms.1us.125ns
p99=0s.0ms.1us.250ns
cpu min=0s.0ms.0us.535ns
max=0s.0ms.3us.104ns
avg=0s.0ms.0us.809ns
std=0s.0ms.0us.133ns
iqr=0s.0ms.0us.137ns
p25=0s.0ms.0us.727ns
p50=0s.0ms.0us.781ns
p75=0s.0ms.0us.864ns
p90=0s.0ms.0us.978ns
p95=0s.0ms.1us.56ns
p99=0s.0ms.1us.202ns
cycles min=643 max=3725 avg=971
active min=83% max=124% avg=99%
jitter min=0s.0ms.0us.0ns
max=0s.0ms.2us.375ns
avg=0s.0ms.0us.162ns
exceptions total=0
samples_with_exceptions=0/40000
[CPU0][vmalloc] N=40000
wall min=0s.0ms.0us.500ns
max=0s.0ms.5us.375ns
avg=0s.0ms.1us.972ns
std=0s.0ms.0us.788ns
iqr=0s.0ms.1us.375ns

```

```

    p25=0s.0ms.1us.250ns
    p50=0s.0ms.2us.0ns
    p75=0s.0ms.2us.625ns
    p90=0s.0ms.3us.0ns
    p95=0s.0ms.3us.250ns
    p99=0s.0ms.3us.375ns
cpu   min=0s.0ms.0us.521ns
      max=0s.0ms.5us.434ns
      avg=0s.0ms.1us.971ns
      std=0s.0ms.0us.786ns
      iqr=0s.0ms.1us.365ns
      p25=0s.0ms.1us.294ns
      p50=0s.0ms.1us.970ns
      p75=0s.0ms.2us.659ns
      p90=0s.0ms.3us.60ns
      p95=0s.0ms.3us.201ns
      p99=0s.0ms.3us.315ns
cycles min=626 max=6521 avg=2366
active  min=83% max=119% avg=99%
jitter  min=0s.0ms.0us.0ns
        max=0s.0ms.4us.625ns
        avg=0s.0ms.0us.61ns
exceptions total=0
samples_with_exceptions=0/40000
[CPU0][vfree] N=40000
wall   min=0s.0ms.0us.125ns
      max=0s.0ms.2us.500ns
      avg=0s.0ms.0us.165ns
      std=0s.0ms.0us.67ns
      iqr=0s.0ms.0us.125ns
      p25=0s.0ms.0us.125ns
      p50=0s.0ms.0us.125ns
      p75=0s.0ms.0us.250ns
      p90=0s.0ms.0us.250ns
      p95=0s.0ms.0us.250ns
      p99=0s.0ms.0us.250ns
cpu    min=0s.0ms.0us.138ns
      max=0s.0ms.2us.460ns
      avg=0s.0ms.0us.165ns
      std=0s.0ms.0us.37ns
      iqr=0s.0ms.0us.30ns
      p25=0s.0ms.0us.150ns
      p50=0s.0ms.0us.163ns
      p75=0s.0ms.0us.180ns
      p90=0s.0ms.0us.184ns
      p95=0s.0ms.0us.195ns
      p99=0s.0ms.0us.220ns
cycles min=166 max=2953 avg=199
active  min=55% max=196% avg=109%
jitter  min=0s.0ms.0us.0ns
        max=0s.0ms.2us.375ns
        avg=0s.0ms.0us.35ns
exceptions total=0
```

```

    samples_with_exceptions=0/40000
[CPU0][cache_malloc_8] N=40000
    wall    min=0s.0ms.0us.125ns
            max=0s.0ms.8us.125ns
            avg=0s.0ms.0us.183ns
            std=0s.0ms.0us.319ns
            iqr=0s.0ms.0us.125ns
            p25=0s.0ms.0us.125ns
            p50=0s.0ms.0us.125ns
            p75=0s.0ms.0us.250ns
            p90=0s.0ms.0us.250ns
            p95=0s.0ms.0us.250ns
            p99=0s.0ms.0us.250ns
    cpu      min=0s.0ms.0us.160ns
            max=0s.0ms.8us.150ns
            avg=0s.0ms.0us.183ns
            std=0s.0ms.0us.315ns
            iqr=0s.0ms.0us.0ns
            p25=0s.0ms.0us.160ns
            p50=0s.0ms.0us.160ns
            p75=0s.0ms.0us.160ns
            p90=0s.0ms.0us.160ns
            p95=0s.0ms.0us.160ns
            p99=0s.0ms.0us.172ns
    cycles  min=192 max=9780 avg=220
    active  min=64% max=177% avg=110%
    jitter  min=0s.0ms.0us.0ns
            max=0s.0ms.8us.0ns
            avg=0s.0ms.0us.72ns
    exceptions total=0
    samples_with_exceptions=0/40000
[CPU0][cache_malloc_64] N=40000
    wall    min=0s.0ms.0us.125ns
            max=0s.0ms.2us.500ns
            avg=0s.0ms.0us.162ns
            std=0s.0ms.0us.83ns
            iqr=0s.0ms.0us.125ns
            p25=0s.0ms.0us.125ns
            p50=0s.0ms.0us.125ns
            p75=0s.0ms.0us.250ns
            p90=0s.0ms.0us.250ns
            p95=0s.0ms.0us.250ns
            p99=0s.0ms.0us.250ns
    cpu      min=0s.0ms.0us.159ns
            max=0s.0ms.2us.490ns
            avg=0s.0ms.0us.162ns
            std=0s.0ms.0us.61ns
            iqr=0s.0ms.0us.4ns
            p25=0s.0ms.0us.159ns
            p50=0s.0ms.0us.159ns
            p75=0s.0ms.0us.163ns
            p90=0s.0ms.0us.163ns
            p95=0s.0ms.0us.163ns

```

```
        p99=0s.0ms.0us.178ns
cycles  min=191 max=2988 avg=195
active  min=63% max=188% avg=109%
jitter  min=0s.0ms.0us.0ns
        max=0s.0ms.2us.375ns
        avg=0s.0ms.0us.31ns
exceptions total=0
samples_with_exceptions=0/40000
[CPU0][cache_malloc_512] N=40000
wall    min=0s.0ms.0us.125ns
        max=0s.0ms.8us.875ns
        avg=0s.0ms.0us.379ns
        std=0s.0ms.1us.83ns
        iqr=0s.0ms.0us.125ns
        p25=0s.0ms.0us.125ns
        p50=0s.0ms.0us.125ns
        p75=0s.0ms.0us.250ns
        p90=0s.0ms.0us.250ns
        p95=0s.0ms.0us.375ns
        p99=0s.0ms.6us.625ns
cpu     min=0s.0ms.0us.165ns
        max=0s.0ms.8us.965ns
        avg=0s.0ms.0us.380ns
        std=0s.0ms.1us.81ns
        iqr=0s.0ms.0us.0ns
        p25=0s.0ms.0us.165ns
        p50=0s.0ms.0us.165ns
        p75=0s.0ms.0us.165ns
        p90=0s.0ms.0us.170ns
        p95=0s.0ms.0us.346ns
        p99=0s.0ms.6us.572ns
cycles  min=198 max=10759 avg=456
active  min=66% max=191% avg=110%
jitter  min=0s.0ms.0us.0ns
        max=0s.0ms.8us.750ns
        avg=0s.0ms.0us.446ns
exceptions total=0
samples_with_exceptions=0/40000
[CPU0][cache_malloc_2048] N=40000
wall    min=0s.0ms.0us.125ns
        max=0s.0ms.13us.375ns
        avg=0s.0ms.0us.768ns
        std=0s.0ms.2us.131ns
        iqr=0s.0ms.0us.125ns
        p25=0s.0ms.0us.125ns
        p50=0s.0ms.0us.125ns
        p75=0s.0ms.0us.250ns
        p90=0s.0ms.0us.500ns
        p95=0s.0ms.8us.375ns
        p99=0s.0ms.9us.0ns
cpu     min=0s.0ms.0us.164ns
        max=0s.0ms.13us.435ns
        avg=0s.0ms.0us.768ns
```

```

        std=0s.0ms.2us.130ns
        iqr=0s.0ms.0us.0ns
        p25=0s.0ms.0us.164ns
        p50=0s.0ms.0us.164ns
        p75=0s.0ms.0us.164ns
        p90=0s.0ms.0us.465ns
        p95=0s.0ms.8us.350ns
        p99=0s.0ms.9us.56ns
cycles min=197 max=16122 avg=922
active min=65% max=192% avg=108%
jitter min=0s.0ms.0us.0ns
        max=0s.0ms.13us.250ns
        avg=0s.0ms.1us.198ns
exceptions total=0
samples_with_exceptions=0/40000
[CPU0][cache_free_8] N=40000
wall   min=0s.0ms.0us.125ns
        max=0s.0ms.8us.250ns
        avg=0s.0ms.0us.170ns
        std=0s.0ms.0us.216ns
        iqr=0s.0ms.0us.0ns
        p25=0s.0ms.0us.125ns
        p50=0s.0ms.0us.125ns
        p75=0s.0ms.0us.125ns
        p90=0s.0ms.0us.250ns
        p95=0s.0ms.0us.250ns
        p99=0s.0ms.0us.250ns
cpu    min=0s.0ms.0us.150ns
        max=0s.0ms.8us.244ns
        avg=0s.0ms.0us.169ns
        std=0s.0ms.0us.209ns
        iqr=0s.0ms.0us.7ns
        p25=0s.0ms.0us.150ns
        p50=0s.0ms.0us.154ns
        p75=0s.0ms.0us.157ns
        p90=0s.0ms.0us.157ns
        p95=0s.0ms.0us.166ns
        p99=0s.0ms.0us.176ns
cycles min=181 max=9893 avg=204
active min=60% max=158% avg=108%
jitter min=0s.0ms.0us.0ns
        max=0s.0ms.8us.125ns
        avg=0s.0ms.0us.45ns
exceptions total=0
samples_with_exceptions=0/40000
[CPU0][cache_free_64] N=40000
wall   min=0s.0ms.0us.125ns
        max=0s.0ms.2us.250ns
        avg=0s.0ms.0us.151ns
        std=0s.0ms.0us.63ns
        iqr=0s.0ms.0us.0ns
        p25=0s.0ms.0us.125ns
        p50=0s.0ms.0us.125ns

```



```

        p75=0s.0ms.0us.125ns
        p90=0s.0ms.0us.250ns
        p95=0s.0ms.0us.250ns
        p99=0s.0ms.0us.250ns
cpu    min=0s.0ms.0us.150ns
        max=0s.0ms.2us.246ns
        avg=0s.0ms.0us.150ns
        std=0s.0ms.0us.37ns
        iqr=0s.0ms.0us.0ns
        p25=0s.0ms.0us.150ns
        p50=0s.0ms.0us.150ns
        p75=0s.0ms.0us.150ns
        p90=0s.0ms.0us.150ns
        p95=0s.0ms.0us.150ns
        p99=0s.0ms.0us.150ns
cycles min=181 max=2696 avg=181
active  min=60% max=163% avg=107%
jitter  min=0s.0ms.0us.0ns
        max=0s.0ms.2us.125ns
        avg=0s.0ms.0us.8ns
exceptions total=0
samples_with_exceptions=0/40000
[CPU0][cache_free_512] N=40000
wall    min=0s.0ms.0us.125ns
        max=0s.0ms.7us.375ns
        avg=0s.0ms.0us.293ns
        std=0s.0ms.0us.613ns
        iqr=0s.0ms.0us.125ns
        p25=0s.0ms.0us.125ns
        p50=0s.0ms.0us.125ns
        p75=0s.0ms.0us.250ns
        p90=0s.0ms.0us.250ns
        p95=0s.0ms.2us.0ns
        p99=0s.0ms.3us.375ns
cpu    min=0s.0ms.0us.151ns
        max=0s.0ms.7us.455ns
        avg=0s.0ms.0us.292ns
        std=0s.0ms.0us.611ns
        iqr=0s.0ms.0us.0ns
        p25=0s.0ms.0us.151ns
        p50=0s.0ms.0us.151ns
        p75=0s.0ms.0us.151ns
        p90=0s.0ms.0us.151ns
        p95=0s.0ms.2us.25ns
        p99=0s.0ms.3us.319ns
cycles min=182 max=8947 avg=351
active  min=60% max=197% avg=107%
jitter  min=0s.0ms.0us.0ns
        max=0s.0ms.7us.250ns
        avg=0s.0ms.0us.224ns
exceptions total=0
samples_with_exceptions=0/40000
[CPU0][cache_free_2048] N=40000
```

```

wall    min=0s.0ms.0us.125ns
        max=0s.0ms.7us.750ns
        avg=0s.0ms.0us.507ns
        std=0s.0ms.1us.0ns
        iqr=0s.0ms.0us.125ns
        p25=0s.0ms.0us.125ns
        p50=0s.0ms.0us.125ns
        p75=0s.0ms.0us.250ns
        p90=0s.0ms.2us.0ns
        p95=0s.0ms.3us.625ns
        p99=0s.0ms.4us.0ns
cpu     min=0s.0ms.0us.150ns
        max=0s.0ms.7us.806ns
        avg=0s.0ms.0us.507ns
        std=0s.0ms.0us.999ns
        iqr=0s.0ms.0us.6ns
        p25=0s.0ms.0us.150ns
        p50=0s.0ms.0us.155ns
        p75=0s.0ms.0us.156ns
        p90=0s.0ms.2us.25ns
        p95=0s.0ms.3us.621ns
        p99=0s.0ms.4us.35ns
cycles  min=181 max=9368 avg=608
active  min=60% max=192% avg=107%
jitter  min=0s.0ms.0us.0ns
        max=0s.0ms.7us.625ns
        avg=0s.0ms.0us.525ns
exceptions total=0
samples_with_exceptions=0/40000
[CPU0][kmallocc_8] N=40000
wall    min=0s.0ms.0us.125ns
        max=0s.0ms.8us.250ns
        avg=0s.0ms.0us.197ns
        std=0s.0ms.0us.408ns
        iqr=0s.0ms.0us.125ns
        p25=0s.0ms.0us.125ns
        p50=0s.0ms.0us.125ns
        p75=0s.0ms.0us.250ns
        p90=0s.0ms.0us.250ns
        p95=0s.0ms.0us.250ns
        p99=0s.0ms.0us.250ns
cpu     min=0s.0ms.0us.168ns
        max=0s.0ms.8us.220ns
        avg=0s.0ms.0us.197ns
        std=0s.0ms.0us.404ns
        iqr=0s.0ms.0us.0ns
        p25=0s.0ms.0us.168ns
        p50=0s.0ms.0us.168ns
        p75=0s.0ms.0us.168ns
        p90=0s.0ms.0us.168ns
        p95=0s.0ms.0us.168ns
        p99=0s.0ms.0us.180ns
cycles  min=202 max=9864 avg=237

```

```
active min=67% max=176% avg=110%
jitter min=0s.0ms.0us.0ns
      max=0s.0ms.8us.125ns
      avg=0s.0ms.0us.100ns
exceptions total=0
samples_with_exceptions=0/40000
[CPU0][kmallocl_1p] N=40000
wall  min=0s.0ms.1us.875ns
      max=0s.0ms.15us.0ns
      avg=0s.0ms.4us.401ns
      std=0s.0ms.1us.362ns
      iqr=0s.0ms.2us.250ns
      p25=0s.0ms.3us.250ns
      p50=0s.0ms.4us.375ns
      p75=0s.0ms.5us.500ns
      p90=0s.0ms.6us.250ns
      p95=0s.0ms.6us.500ns
      p99=0s.0ms.6us.750ns
cpu    min=0s.0ms.1us.965ns
      max=0s.0ms.14us.962ns
      avg=0s.0ms.4us.401ns
      std=0s.0ms.1us.361ns
      iqr=0s.0ms.2us.305ns
      p25=0s.0ms.3us.235ns
      p50=0s.0ms.4us.330ns
      p75=0s.0ms.5us.540ns
      p90=0s.0ms.6us.262ns
      p95=0s.0ms.6us.494ns
      p99=0s.0ms.6us.705ns
cycles min=2359 max=17955 avg=5282
active min=94% max=105% avg=99%
jitter min=0s.0ms.0us.0ns
      max=0s.0ms.8us.875ns
      avg=0s.0ms.0us.169ns
exceptions total=0
samples_with_exceptions=0/40000
[CPU0][kmallocl_2p] N=40000
wall  min=0s.0ms.2us.250ns
      max=0s.0ms.10us.875ns
      avg=0s.0ms.4us.627ns
      std=0s.0ms.1us.362ns
      iqr=0s.0ms.2us.250ns
      p25=0s.0ms.3us.500ns
      p50=0s.0ms.4us.625ns
      p75=0s.0ms.5us.750ns
      p90=0s.0ms.6us.500ns
      p95=0s.0ms.6us.750ns
      p99=0s.0ms.7us.0ns
cpu    min=0s.0ms.2us.270ns
      max=0s.0ms.10us.843ns
      avg=0s.0ms.4us.627ns
      std=0s.0ms.1us.361ns
      iqr=0s.0ms.2us.324ns
```

```

        p25=0s.0ms.3us.457ns
        p50=0s.0ms.4us.575ns
        p75=0s.0ms.5us.781ns
        p90=0s.0ms.6us.530ns
        p95=0s.0ms.6us.767ns
        p99=0s.0ms.6us.955ns
cycles min=2725 max=13012 avg=5553
active min=95% max=105% avg=99%
jitter min=0s.0ms.0us.0ns
        max=0s.0ms.6us.625ns
        avg=0s.0ms.0us.146ns
exceptions total=0
samples_with_exceptions=0/40000
[CPU0][kmallo_3p] N=40000
wall   min=0s.0ms.3us.250ns
        max=0s.0ms.28us.750ns
        avg=0s.0ms.8us.355ns
        std=0s.0ms.3us.38ns
        iqr=0s.0ms.5us.0ns
        p25=0s.0ms.5us.750ns
        p50=0s.0ms.8us.250ns
        p75=0s.0ms.10us.750ns
        p90=0s.0ms.12us.500ns
        p95=0s.0ms.13us.0ns
        p99=0s.0ms.13us.625ns
cpu     min=0s.0ms.3us.304ns
        max=0s.0ms.28us.666ns
        avg=0s.0ms.8us.355ns
        std=0s.0ms.3us.38ns
        iqr=0s.0ms.5us.46ns
        p25=0s.0ms.5us.764ns
        p50=0s.0ms.8us.235ns
        p75=0s.0ms.10us.810ns
        p90=0s.0ms.12us.500ns
        p95=0s.0ms.12us.958ns
        p99=0s.0ms.13us.600ns
cycles min=3965 max=34400 avg=10026
active min=96% max=103% avg=99%
jitter min=0s.0ms.0us.0ns
        max=0s.0ms.19us.625ns
        avg=0s.0ms.0us.292ns
exceptions total=0
samples_with_exceptions=0/40000
[CPU0][kmallo_5p] N=40000
wall   min=0s.0ms.3us.750ns
        max=0s.0ms.36us.500ns
        avg=0s.0ms.8us.792ns
        std=0s.0ms.3us.9ns
        iqr=0s.0ms.5us.0ns
        p25=0s.0ms.6us.250ns
        p50=0s.0ms.8us.750ns
        p75=0s.0ms.11us.250ns
        p90=0s.0ms.13us.0ns

```

```

        p95=0s.0ms.13us.500ns
        p99=0s.0ms.14us.0ns
cpu    min=0s.0ms.3us.722ns
        max=0s.0ms.36us.445ns
        avg=0s.0ms.8us.792ns
        std=0s.0ms.3us.9ns
        iqr=0s.0ms.5us.57ns
        p25=0s.0ms.6us.218ns
        p50=0s.0ms.8us.719ns
        p75=0s.0ms.11us.275ns
        p90=0s.0ms.12us.960ns
        p95=0s.0ms.13us.465ns
        p99=0s.0ms.13us.966ns
cycles min=4467 max=43735 avg=10551
active  min=97% max=103% avg=99%
jitter  min=0s.0ms.0us.0ns
        max=0s.0ms.23us.375ns
        avg=0s.0ms.0us.264ns
exceptions total=0
samples_with_exceptions=0/40000
[CPU0][kfree_8] N=40000
wall    min=0s.0ms.0us.125ns
        max=0s.0ms.8us.500ns
        avg=0s.0ms.0us.187ns
        std=0s.0ms.0us.238ns
        iqr=0s.0ms.0us.125ns
        p25=0s.0ms.0us.125ns
        p50=0s.0ms.0us.125ns
        p75=0s.0ms.0us.250ns
        p90=0s.0ms.0us.250ns
        p95=0s.0ms.0us.250ns
        p99=0s.0ms.0us.250ns
cpu    min=0s.0ms.0us.165ns
        max=0s.0ms.8us.451ns
        avg=0s.0ms.0us.186ns
        std=0s.0ms.0us.230ns
        iqr=0s.0ms.0us.7ns
        p25=0s.0ms.0us.165ns
        p50=0s.0ms.0us.169ns
        p75=0s.0ms.0us.172ns
        p90=0s.0ms.0us.175ns
        p95=0s.0ms.0us.182ns
        p99=0s.0ms.0us.191ns
cycles min=199 max=10142 avg=224
active  min=66% max=172% avg=111%
jitter  min=0s.0ms.0us.0ns
        max=0s.0ms.8us.250ns
        avg=0s.0ms.0us.78ns
exceptions total=0
samples_with_exceptions=0/40000
[CPU0][kfree_1p] N=40000
wall    min=0s.0ms.1us.625ns
        max=0s.0ms.9us.250ns
```

```

    avg=0s.0ms.1us.954ns
    std=0s.0ms.0us.273ns
    iqr=0s.0ms.0us.125ns
    p25=0s.0ms.1us.875ns
    p50=0s.0ms.1us.875ns
    p75=0s.0ms.2us.0ns
    p90=0s.0ms.2us.250ns
    p95=0s.0ms.2us.375ns
    p99=0s.0ms.2us.625ns
cpu    min=0s.0ms.1us.609ns
        max=0s.0ms.9us.297ns
        avg=0s.0ms.1us.954ns
        std=0s.0ms.0us.268ns
        iqr=0s.0ms.0us.170ns
        p25=0s.0ms.1us.830ns
        p50=0s.0ms.1us.898ns
        p75=0s.0ms.2us.0ns
        p90=0s.0ms.2us.192ns
        p95=0s.0ms.2us.418ns
        p99=0s.0ms.2us.697ns
cycles min=1931 max=11157 avg=2345
active  min=93% max=107% avg=99%
jitter  min=0s.0ms.0us.0ns
        max=0s.0ms.7us.375ns
        avg=0s.0ms.0us.206ns
exceptions total=0
samples_with_exceptions=0/40000
[CPU0][kfree_2p] N=40000
wall    min=0s.0ms.1us.750ns
        max=0s.0ms.5us.500ns
        avg=0s.0ms.2us.192ns
        std=0s.0ms.0us.217ns
        iqr=0s.0ms.0us.375ns
        p25=0s.0ms.2us.0ns
        p50=0s.0ms.2us.125ns
        p75=0s.0ms.2us.375ns
        p90=0s.0ms.2us.500ns
        p95=0s.0ms.2us.500ns
        p99=0s.0ms.2us.750ns
cpu     min=0s.0ms.1us.762ns
        max=0s.0ms.5us.507ns
        avg=0s.0ms.2us.191ns
        std=0s.0ms.0us.210ns
        iqr=0s.0ms.0us.374ns
        p25=0s.0ms.2us.43ns
        p50=0s.0ms.2us.130ns
        p75=0s.0ms.2us.417ns
        p90=0s.0ms.2us.427ns
        p95=0s.0ms.2us.480ns
        p99=0s.0ms.2us.707ns
cycles min=2115 max=6609 avg=2630
active  min=93% max=106% avg=99%
jitter  min=0s.0ms.0us.0ns

```

```
        max=0s.0ms.3us.500ns
        avg=0s.0ms.0us.243ns
exceptions total=0
samples_with_exceptions=0/40000
[CPU0][kfree_3p] N=40000
wall   min=0s.0ms.2us.500ns
        max=0s.0ms.11us.875ns
        avg=0s.0ms.3us.305ns
        std=0s.0ms.0us.461ns
        iqr=0s.0ms.0us.375ns
        p25=0s.0ms.3us.125ns
        p50=0s.0ms.3us.250ns
        p75=0s.0ms.3us.500ns
        p90=0s.0ms.3us.625ns
        p95=0s.0ms.3us.625ns
        p99=0s.0ms.4us.250ns
cpu     min=0s.0ms.2us.512ns
        max=0s.0ms.11us.807ns
        avg=0s.0ms.3us.305ns
        std=0s.0ms.0us.459ns
        iqr=0s.0ms.0us.328ns
        p25=0s.0ms.3us.115ns
        p50=0s.0ms.3us.286ns
        p75=0s.0ms.3us.443ns
        p90=0s.0ms.3us.573ns
        p95=0s.0ms.3us.655ns
        p99=0s.0ms.4us.208ns
cycles min=3015 max=14169 avg=3966
active min=95% max=104% avg=99%
jitter min=0s.0ms.0us.0ns
        max=0s.0ms.8us.750ns
        avg=0s.0ms.0us.246ns
exceptions total=0
samples_with_exceptions=0/40000
[CPU0][kfree_5p] N=40000
wall   min=0s.0ms.2us.750ns
        max=0s.0ms.16us.250ns
        avg=0s.0ms.3us.770ns
        std=0s.0ms.0us.498ns
        iqr=0s.0ms.0us.500ns
        p25=0s.0ms.3us.500ns
        p50=0s.0ms.3us.750ns
        p75=0s.0ms.4us.0ns
        p90=0s.0ms.4us.250ns
        p95=0s.0ms.4us.375ns
        p99=0s.0ms.4us.750ns
cpu     min=0s.0ms.2us.829ns
        max=0s.0ms.16us.337ns
        avg=0s.0ms.3us.770ns
        std=0s.0ms.0us.495ns
        iqr=0s.0ms.0us.498ns
        p25=0s.0ms.3us.504ns
        p50=0s.0ms.3us.788ns
```

```

        p75=0s.0ms.4us.2ns
        p90=0s.0ms.4us.254ns
        p95=0s.0ms.4us.398ns
        p99=0s.0ms.4us.734ns
cycles min=3395 max=19605 avg=4524
active min=96% max=104% avg=99%
jitter min=0s.0ms.0us.0ns
        max=0s.0ms.12us.625ns
        avg=0s.0ms.0us.237ns
exceptions total=0
samples_with_exceptions=0/40000

```

Listing B.11: Allocator benchmark source code

```

#define N_OUTER 200
#define N_INNER 200
#define N_TOTAL (N_OUTER * N_INNER)

typedef enum {
    METRIC_PAGE_MALLOC = 0,
    METRIC_PAGE_FREE,
    METRIC_VMALLOC,
    METRIC_VFREE,
    METRIC_CACHE_MALLOC_8,
    METRIC_CACHE_MALLOC_64,
    METRIC_CACHE_MALLOC_512,
    METRIC_CACHE_MALLOC_2048,
    METRIC_CACHE_FREE_8,
    METRIC_CACHE_FREE_64,
    METRIC_CACHE_FREE_512,
    METRIC_CACHE_FREE_2048,
    METRIC_KMALLOC_8,
    METRIC_KMALLOC_1P, // 1 page (4096 bytes)
    METRIC_KMALLOC_2P, // 2 pages (8192 bytes)
    METRIC_KMALLOC_3P, // 3 pages (12288 bytes)
    METRIC_KMALLOC_5P, // 5 pages (20480 bytes)
    METRIC_KFREE_8,
    METRIC_KFREE_1P,
    METRIC_KFREE_2P,
    METRIC_KFREE_3P,
    METRIC_KFREE_5P,
    METRIC_COUNT,
} metric_id_e;

static const char* metric_names[METRIC_COUNT] = {
    [METRIC_PAGE_MALLOC] = "page_malloc",
    [METRIC_PAGE_FREE] = "page_free",
    [METRIC_VMALLOC] = "vmalloc",
    [METRIC_VFREE] = "vfree",
    [METRIC_CACHE_MALLOC_8] = "cache_malloc_8",
    [METRIC_CACHE_MALLOC_64] = "cache_malloc_64",
    [METRIC_CACHE_MALLOC_512] = "cache_malloc_512",
    [METRIC_CACHE_MALLOC_2048] = "cache_malloc_2048",

```



```

[METRIC_CACHE_FREE_8]      = "cache_free_8",
[METRIC_CACHE_FREE_64]     = "cache_free_64",
[METRIC_CACHE_FREE_512]    = "cache_free_512",
[METRIC_CACHE_FREE_2048]   = "cache_free_2048",
[METRIC_KMALLOC_8]         = "kmalloc_8",
[METRIC_KMALLOC_1P]        = "kmalloc_1p",
[METRIC_KMALLOC_2P]        = "kmalloc_2p",
[METRIC_KMALLOC_3P]        = "kmalloc_3p",
[METRIC_KMALLOC_5P]        = "kmalloc_5p",
[METRIC_KFREE_8]           = "kfree_8",
[METRIC_KFREE_1P]          = "kfree_1p",
[METRIC_KFREE_2P]          = "kfree_2p",
[METRIC_KFREE_3P]          = "kfree_3p",
[METRIC_KFREE_5P]          = "kfree_5p",
};

static kvec(monitor_data_t) metrics[METRIC_COUNT];

static inline void record(metric_id_e id)
{
    performance_monitor_fence();
    monitor_data_t in = performance_monitor_read_fence(IMX8MP_CPU_FREQ);
    kvec_push(&metrics[id], &in);
}

static void bench_page_allocator()
{
    puintptr_t ptrs[N_INNER];
    for (size_t i = 0; i < N_OUTER; i++) {
        irqlocked()
        {
            for (size_t j = 0; j < N_INNER; j++) {
                performance_monitor_fence();
                ptrs[j] = page_malloc(0, (mm_page_data) {});
                record(METRIC_PAGE_MALLOC);
            }
            for (size_t j = 0; j < N_INNER; j++) {
                performance_monitor_fence();
                page_free(ptrs[j]);
                record(METRIC_PAGE_FREE);
            }
        }
    }
}

static void bench_vmalloc()
{
    vmalloc_token vtokens[N_INNER];
    for (size_t i = 0; i < N_OUTER; i++) {
        irqlocked()
        {
            for (size_t j = 0; j < N_INNER; j++) {
                performance_monitor_fence();
                vmalloc(1, "", (vmalloc_cfg) {}, &vtokens[j]);
            }
        }
    }
}

```

```

        record(METRIC_VMALLOC);
    }
    for (size_t j = 0; j < N_INNER; j++) {
        performance_monitor_fence();
        vfree(vtokens[j], NULL);
        record(METRIC_VFREE);
    }
}
}

static void bench_cache_malloc()
{
    const cache_malloc_size sizes[4] = {
        CACHE_8,
        CACHE_64,
        CACHE_512,
        CACHE_2048};

    const metric_id_e metr[4] = {
        METRIC_CACHE_MALLOC_8,
        METRIC_CACHE_MALLOC_64,
        METRIC_CACHE_MALLOC_512,
        METRIC_CACHE_MALLOC_2048,
    };

    const metric_id_e metrfree[4] = {
        METRIC_CACHE_FREE_8,
        METRIC_CACHE_FREE_64,
        METRIC_CACHE_FREE_512,
        METRIC_CACHE_FREE_2048,
    };

    for (size_t c = 0; c < 4; c++) {
        for (size_t i = 0; i < N_OUTER; i++) {
            irqlocked()
            {
                void* ptrs[N_INNER];
                for (size_t j = 0; j < N_INNER; j++) {
                    performance_monitor_fence();
                    ptrs[j] = cache_malloc(sizes[c]);
                    record(metr[c]);
                }
                for (size_t j = 0; j < N_INNER; j++) {
                    performance_monitor_fence();
                    cache_free(ptrs[j]);
                    record(metrfree[c]);
                }
            }
        }
    }
}

```

```
static void bench_kmalloc()
{
    size_t sizes[5] = {
        8, // to test the cache malloc path
        PAGE_SIZE * 1,
        PAGE_SIZE * 2,
        PAGE_SIZE * 3,
        PAGE_SIZE * 5,
    };

    static const metric_id_e kmalloc_metrics[5] = {
        METRIC_KMALLOC_8,
        METRIC_KMALLOC_1P,
        METRIC_KMALLOC_2P,
        METRIC_KMALLOC_3P,
        METRIC_KMALLOC_5P,
    };

    static const metric_id_e kfree_metrics[5] = {
        METRIC_KFREE_8,
        METRIC_KFREE_1P,
        METRIC_KFREE_2P,
        METRIC_KFREE_3P,
        METRIC_KFREE_5P,
    };

    void* ptrs[N_INNER];
    for (size_t sz = 0; sz < 5; sz++) {
        for (size_t i = 0; i < N_OUTER; i++) {
            irqlocked()
            {
                for (size_t j = 0; j < N_INNER; j++) {
                    performance_monitor_fence();
                    ptrs[j] = kmalloc(sizes[sz]);
                    record(kmalloc_metrics[sz]);
                }
                for (size_t j = 0; j < N_INNER; j++) {
                    performance_monitor_fence();
                    kfree(ptrs[j]);
                    record(kfree_metrics[sz]);
                }
            }
        }
    }
}

static void print_metrics()
{
    for (size_t i = 0; i < METRIC_COUNT; i++) {
        performance_monitor_print_metrics(
            &metrics[i],
            IMX8MP_CPU_FREQ,
            metric_names[i]);
    }
}
```

```
    }  
}  
  
noreturn void kernel_entry()  
{  
    if (get_cpuid() == 0) {  
        if (!mm_kernel_is_relocated())  
            kernel_early_init();  
        else  
            kernel_init();  
    }  
  
    if (get_cpuid() != 0)  
        loop asm volatile("wfi");  
  
    for (size_t i = 0; i < METRIC_COUNT; i++)  
        metrics[i] = kvec_new(monitor_data_t);  
  
    bench_page_allocator();  
    bench_vmalloc();  
    bench_cache_malloc();  
    bench_kmalloc();  
  
    ksleep(1e9);  
    print_metrics();  
  
    loop asm volatile("wfi");  
}
```

Acronyms

AIPS Advanced IP Bus Switch. 20, 21

bss block starting symbol. 28, 29, 49, 51, 76

DAIF Debug, [SError](#), [IRQ](#), [FIQ](#) (exception mask bits). 11, 27, 32, 39, 46

DDR Double Data Rate (category of RAM). ix, 19–21, 27, 47, 48, 51, 55, 56, 59, 96

DMA Direct Memory Access. 17, 48

EL0 Exception Level 0 (User-space privileged level). 2, 9, 10, 15, 31, 41–43, 47, 75, 76, 78–82, 90, 108

EL1 Exception Level 1 (Kernel/Supervisor privileged level). 9, 10, 15, 29–34, 41, 42, 47, 49, 75, 79–82, 90

EL2 Exception Level 2 (Hypervisor privileged level). 9, 10, 27, 29, 31, 33, 49

EL3 Exception Level 3 (Firmware privileged level). 9, 84

ELF Executable and Linkable Format. 47, 76

FIQ Fast Interrupt Request. 10–12, 27, 36, 42

GDB GNU Debugger. 21, 27

GIC Generic Interrupt Controller. 2, 10, 20, 21, 36, 42, 87, 108

IRQ Interrupt Request. 10–12, 27, 36, 41, 42, 46, 69, 97, 100, 102, 103

ISA Instruction Set Architecture. 7

JTAG Joint Test Action Group. 21

- MMIO** Memory-Mapped I/O. ix, 17, 19–21, 45, 51, 53, 55, 56, 59, 63, 85, 86
- MMU** Memory Management Unit. i, ii, v, ix, 1, 2, 7–10, 12–15, 17, 18, 27, 29, 31, 34–38, 46, 47, 53–55, 57, 58, 63, 65–68, 72–75, 79, 84, 102, 104, 106–108
- NPU** Neural Processing Unit. 19
- OCRAM** On-Chip RAM. 20, 55
- PE** Processing Element. i, 8, 11, 22, 80, 82, 97
- PFN** Page Frame Number. 59, 60
- PSCI** Power State Coordination Interface [20]. 84
- RTOS** Real-Time Operating System. 6, 7
- SError** System Error. 10, 11, 27, 36, 42
- SIMD** Single Instruction, Multiple Data. 18, 21, 28, 31, 35, 45, 54, 69, 102
- SMCCC** SMC Calling Convention [21]. 25, 84
- SMP** Symmetric Multiprocessing. 69
- SoC** System On a Chip. 1, 6, 7, 18, 19, 23, 24, 27, 48, 85, 87, 107
- SWD** Serial Wire Debug. 21
- TF-A** Trusted Firmware-A [18]. 48
- TRM** Technical Reference Manual. 19, 20
- UART** Universal Asynchronous Receiver-Transmitter. 10, 21, 22, 85–87, 89, 95, 103, 108
- UID** Unique Identifier. 91–93
- VFS** Virtual File System. ii, 88, 89, 91, 108, 109

Glossary

Alignment fault A synchronous exception thrown when a memory access is not aligned to its natural boundary and the architecture or configuration requires alignment. For example, accessing a 64-bit value at an address not divisible by 8. [10](#), [17](#), [35](#), [54](#)

Armv8-A ARM architecture [\[4\]](#) targeting application-class processors. It supports two execution states: AArch64, a 64-bit state with 31 general-purpose registers and a revised A64 instruction set, and AArch32, a 32-bit state providing backwards compatibility with the Armv7-A architecture. The architecture defines four exception levels, a hardware memory management unit with multi-level page tables, TrustZone security extensions, and an advanced SIMD and floating-point unit. [1](#), [7](#), [8](#), [10](#), [12](#), [15](#), [17](#), [21](#)

Bootloader Software that runs immediately after power-on and is responsible for initializing hardware and loading the operating system kernel into memory before transferring control to it. [2](#), [9](#), [20](#), [25](#), [27](#), [86](#)

Canonical A virtual address is canonical if its upper bits are a sign extension of the highest address bit used by the translation regime. Accessing a non-canonical address causes a fault.. [13](#), [47](#)

Context switch The act of saving the execution state of a running thread and restoring the state of another, allowing the processor to switch between threads. Involves saving and restoring general-purpose registers, the program counter, the stack pointer, and any floating-point or SIMD state.. [2](#), [24](#), [41](#), [53](#), [72](#), [76](#), [78](#), [79](#), [108](#), [109](#)

Device Memory Memory type in the Armv8-A memory model used for memory-mapped I/O regions. Accesses to device memory are strongly ordered and never speculatively fetched, ensuring that side effects such as clearing an interrupt flag occur in the expected order. [17](#), [49](#), [51](#), [55](#), [59](#)

Direct-mapped memory A region of the kernel virtual address space where all physical memory is mapped at a fixed offset, allowing the kernel to access any physical address through a simple arithmetic conversion without requiring a dynamic virtual memory allocation. [54](#), [57](#), [63](#), [64](#), [66–71](#), [74](#), [75](#)

Exception Level Privilege level in the Armv8-A architecture. EL0 is unprivileged userspace, EL1 is the operating system kernel, EL2 is the hypervisor, and EL3 is the secure monitor. Higher levels have access to a wider set of system registers and can configure the behavior of lower levels. [1](#), [8–10](#), [31](#), [78](#)

Exception masking and unmasking The act of enabling or disabling the delivery of exceptions to the processor. In Armv8-A, the DAIF bits in PSTATE control masking of Debug, SError, IRQ, and FIQ exceptions independently. [11](#), [27](#), [32](#), [36](#), [46](#), [82](#), [89](#)

Fragmentation Condition in which available memory is divided into small, non-contiguous regions that cannot satisfy large allocation requests even if the total free memory is sufficient. Internal fragmentation refers to wasted space within an allocated block due to size rounding, while external fragmentation refers to free memory scattered across many small gaps between allocated blocks. [47](#), [59](#), [60](#), [62](#), [64](#), [69](#)

Hardware Abstraction Layer A software layer that defines a common interface for a class of hardware devices, hiding platform-specific implementation details from the rest of the kernel. Drivers register themselves against the interface, making the kernel subsystems that use them portable across platforms. [2](#), [85](#), [108](#), [109](#)

Identity mapping A virtual memory mapping in which a virtual address maps directly to the same physical address. Used during early boot to allow the kernel to enable the MMU without immediately remapping its own code, since the program counter continues to hold a physical address that remains valid under the identity mapping. [ix](#), [27](#), [29](#), [35](#), [37](#), [54–57](#)

Normal Memory Memory type in the Armv8-A memory model used for ordinary RAM. The processor may reorder, merge, or speculatively access normal memory, allowing significant performance optimizations that would be unsafe for device memory. [11](#), [17](#), [18](#), [55](#)

OP-TEE Open Portable Trusted Execution Environment. An open-source Trusted OS running in the Armv8-A Secure World, providing a TEE infrastructure for trusted applications to execute in isolation from the Normal World kernel. [48](#), [51](#)

PSTATE Abstraction of the processor state in Armv8-A, holding condition flags, execution state information, and exception mask bits. Individual fields of PSTATE are accessible through dedicated system registers such as DAIF. 8

Secure World Execution environment in Armv8-A provided by [TrustZone](#) technology, isolated from the Normal World by hardware. The Secure World runs at EL3 and Secure EL1, and can access resources that are hidden from the Normal World kernel and hypervisor. 9, 10, 48, 87

Sysreg System register in the Armv8-A architecture, accessed via the MRS and MSR instructions. System registers control and expose the configuration of processor features such as the MMU, exception handling, timers, and performance monitors. 8–11, 13, 18, 27, 31, 45, 58, 75, 97

System Call A system call (syscall) is a controlled entry point into the operating system kernel that enables an application to access privileged services unavailable in user space.. 104

Translation fault A synchronous exception raised by the MMU when a virtual address cannot be translated because no valid page table descriptor exists for it at the required level of the page table walk. 9, 10, 13, 67

TrustZone Hardware security extension of the Armv8-A architecture that partitions the processor into two isolated execution environments: the Normal World, which runs the general-purpose operating system, and the Secure World, which runs trusted software such as a Trusted OS. The isolation is enforced by hardware, preventing Normal World software from accessing [Secure World](#) memory, peripherals, or registers. Transitions between worlds are mediated by the Secure Monitor running at EL3 via the SMC instruction. 10, 15

Vector Table Table of code entry points that the processor jumps to when an exception or interrupt is taken. In Armv8-A, the base address of the vector table is configured via the VBAR_EL1 system register, and each entry handles a specific combination of exception type and source exception level. vii, ix, 2, 10, 11, 27, 30, 31, 33, 41, 42, 48, 58, 78, 108

Bibliography

- [1] L. Torvalds, “Linux: a portable operating system,” *Master’s thesis, University of Helsinki*, 1997.
- [2] R. Cox, M. F. Kaashoek, and R. Morris, “xv6: A Simple, Unix-like Teaching Operating System,” <https://pdos.csail.mit.edu/6.828/2025/xv6.html>, 2026, mIT PDOS group.
- [3] seL4 Foundation, “The seL4 Microkernel,” <https://sel4.systems/>, 2026.
- [4] Arm Limited, *Arm® Architecture Reference Manual for A-profile Architecture*, <https://developer.arm.com/documentation/ddi0487/mb/>, 2026, document ID DDI0487M.b.
- [5] —, *Arm® v8-M Architecture Reference Manual*, <https://developer.arm.com/documentation/ddi0553/bz/>, 2026, document ID DDI0553B.z.
- [6] —, *Arm Generic Interrupt Controller Architecture Specification, GIC Architecture Version 3 and Version 4*, <https://developer.arm.com/documentation/ihl0069/hb/>, 2024, document ID IHI0069H.b.
- [7] —, *Armv8-A Address Translation*, <https://developer.arm.com/documentation/100940/latest>, 2019, document ID 100940_0100_en.
- [8] NXP Semiconductors, *i.MX 8M Plus Applications Processor Reference Manual*, https://www.nxp.com/docs/en/preview/PREVIEW_IMX8MPRM.pdf, 2021, full download requires an NXP account.
- [9] Arm Limited, *Arm Cortex-A53 MPCore Processor Technical Reference Manual*, <https://developer.arm.com/documentation/ddi0500/j>, 2018, document ID DDI0500J.
- [10] NXP Semiconductors, *AN14367: Debugging Cortex-A U-Boot and Native RTOS on i.MX 8M Plus and i.MX 93 EVKs*, <https://docs.nxp.com/bundle/AN14367/page/topics/introduction.html>, 2024, application Note AN14367.
- [11] U. Iborra Albéniz, “ARMv8_kernel,” https://github.com/unaiiborra/ARMv8_kernel, 2026.

- [12] Arm Limited, “GNU Arm Embedded Toolchain Downloads,” <https://developer.arm.com/downloads/-/gnu-rm>, 2026.
- [13] U-Boot Contributors, “Das U-Boot – the Universal Boot Loader,” <https://u-boot.org/>, 2026.
- [14] Arm Limited, *Bare-metal Boot Code for Armv8-A Processors*, <https://developer.arm.com/documentation/dai0527/a/>, 2017, application Note 527, Document ID DAI0527A.
- [15] —, *ARM System Memory Management Unit Architecture Specification*, <https://developer.arm.com/documentation/ih0062/b/>, 2012, document ID IHI0062B.
- [16] R. Landley, “Red-black trees (rbtree) in Linux — The Linux Kernel documentation,” <https://docs.kernel.org/core-api/rbtree.html>, 2007.
- [17] L. Torvalds and Linux Kernel Contributors, “rbtree.c – Linux kernel source,” <https://github.com/torvalds/linux/blob/master/lib/rbtree.c>, 2026.
- [18] Trusted Firmware Project, “Trusted Firmware-A (TF-A),” <https://www.trustedfirmware.org/projects/tf-a/>, 2026.
- [19] TIS Committee, *Tool Interface Standard (TIS) Executable and Linking Format (ELF) Specification, Version 1.2*, <https://refspecs.linuxfoundation.org/elf/elf.pdf>, 1995, tool Interface Standards Committee.
- [20] Arm Limited, *Arm Power State Coordination Interface: Platform Design Document*, <https://developer.arm.com/documentation/den0022/fb/>, 2024, document ID DEN0022F.b.
- [21] —, *SMC Calling Convention*, <https://developer.arm.com/documentation/den0028/h/>, 2026, document ID DEN0028H.